

Mesterséges intelligencia alapjai

Hogy mérik és mire használják?

Intelligencia

- **Webster's Dictionary (szótár):**

"Az a képesség, hogy hatékonyan tudjon a tapasztalásból tanulni, vagy megérteni, kinyerni és megőrizni a tudást; mentális (szellemi) képesség; egy új szituációhoz való gyors és sikeres alkalmazkodás képessége; a következtetés használatának a képessége a problémamegoldásban, a viselkedés irányítása, stb.,,

- **Christopher F. Chabris:**

"Mondhatnánk azt: Nem tudom megmondani, mi az intelligencia, de megismerem, ha találkozok vele. Az intelligencia olyan fogalom, amely jelentését a kontextusból, alkalmazási környezetéből nyeri, nem pedig egy felállított modellből, vagy kritériumrendszerből."

- **David Wechsler:**

"Az intelligencia az egyénnek az az összesített, vagy globális képessége, amely lehetővé teszi, hogy célszerűen cselekedjen, hogy racionálisan gondolkodjon és eredményesen bántson a környezetével."

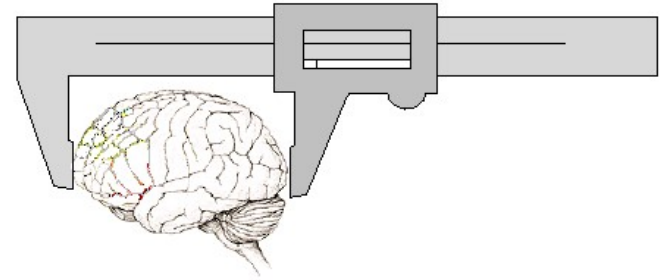
- **Alfred Binet és Teophile Simon:**

"Úgy tűnik, hogy az intelligenciában van egy alapvető tényező, amelynek megléte, illetve hiánya oly döntő a mindennapi életben. Ez az ítéletek, a józan ész képessége, a gyakorlati érzék, a kezdeményezőkézség és a körülményekhez való alkalmazkodás képessége. A jó döntés, a jó felfogás és a jó okfejtés az intelligencia lényege."

Mesterséges intelligencia

- Az Artificial Intelligence (AI) elnevezést McCarthy alkalmazta először 1956-ban
- Wikipédia szerint
 - „Mesterséges intelligenciának egy gép, program vagy mesterségesen létrehozott tudat által megnyilvánuló intelligenciát nevezzük. „
- Megközelítések:
 1. A mesterségesen létrehozott tárgy állandó emberi beavatkozás nélkül képes legyen válaszolni környezeti behatásokra (automatizáltság) – az egyszerű szoftverágens ilyen;
 2. A mesterségesen létrehozott tárgy képes legyen hasonlóan viselkedni, mint egy természetes intelligenciával rendelkező élőlény, még ha az azonos viselkedés mögött eltérő mechanizmus is húzódik meg (TI szimuláltság – ilyen értelemben beszélhetünk pl. a számítógépes játékok gép irányította karaktereinek „intelligenciájáról”);
 3. Végül, a mesterségesen létrehozott tárgy képes legyen viselkedését célszerűen és megismételhető módon változtatni (tanulás) – ez utóbbi jelentés az, ami a modern MI-kutatásban előtérbe került, és jelenleg az MI fogalmával legjobban azonosítható.

Az intelligencia mérése

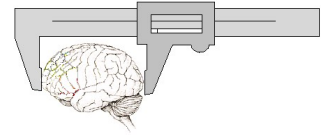


- Már korábban felmerült az egzakt mérés igénye
 - Mivel az intelligencia definiálása nem egységes, az intelligencia mérése is többféle megközelítéssel lehetséges → tesztek
- Tesztek fajtái
 - Teljesítménytesztek: elsajátított képességeket mérjen, jelenleg mit tudunk teljesíteni
 - Képességtesztek: prognosztizálni, jósolni, hogy gyakorlás után mire leszünk képesek
 - Az intelligencia teszt is képességteszt
- Követelmények:
 - Érvényesség: azt mérje, amit mérni szeretnénk
 - Megbízhatóság: ismételve közel ugyanolyan eredményt adjon.
- Kortól, kultúrától való függés

Intelligencia tesztek készítése



- Sir Francis Galton
 - Szerinte az intelligencia kivételes érzékelési és észlelési készségek kérdése, melyek öröklöttek
 - Mérésekkel összekötött vizsgálatokat végzett
 - Ezek viszont nem igazolták elképzeléseit
 - Érdeme: a korrelációs együtttható alkalmazásának bevezetése.
Korreláció: egymást kölcsönösen feltételező dolgok, vagy fogalmak viszonya, dolgoknak egymástól való függése, ill. egymásnak való megfelelése.
- J. Cattel:
 - Első intelligenciateszt, 1890.
 - Két csoportra osztotta az emberi képesség tág spektrumát (57 változó)
 - Elsődleges faktorok: alapvető emberi képességek
 - Másodlagos faktorok:
 - Folyékony intelligencia (Fluid): a megértés ereje, a problémahelyzethez való rugalmas alkalmazkodás, öröklés által meghatározott
 - Kristályos intelligencia (Rögzült): már megszerzett tudás és képességek alkalmazása
 - Ő a tesztműszerek atyja.

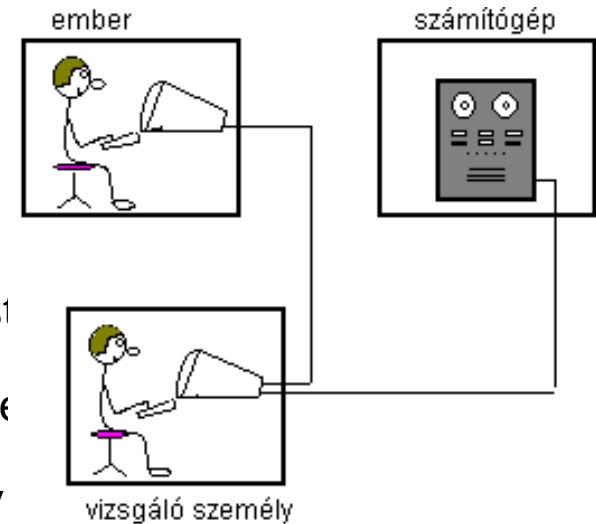


- Alfred Binet, Teophile Simon

- Teszt gyerekek iskolaérettségének vizsgálatára (3-15 éves kor között), 1905.
 - Francia kormány kérte a tesztet, amelynek segítségével a lassú felfogású gyermekeket szűrte ki, akik nehezen tudnak lépést tartani a többiekkel a tananyag elsajátításában
- Gondolkodási és problémamegoldási feladatok.
 - Például hónapok nevei, évszakok jellemzői, számemlékezet
 - A legkönnyebb feladatban a gyermeknek a szemével követnie kellett egy fénypontot
 - Ha a feladatot egy adott korú vizsgált gyermekek 85-90%-a megold, akkor ez jellemző az adott életkorra
- Mentális kor fogalma, mentális kor skála.
 - Egy gyermek abból a szempontból jellemezhető, hogy milyen életkornak megfelelő feladatokat képes megoldani -> Mentális szint
 - A legmagasabb szintet, ahol sikeresen elvégezte a gyermek a feladatot, azt a gyermek mentális korának tekintették
- A legelterjedtebb tesztek egyike

A Turing teszt „Imitation Game”

- A tesztet Alan Turing fogalmazta meg a(z igazi) mesterséges intelligencia minősítésére.
- Bíráló billentyűzet és monitor közvetítésével kérdéseket tesz fel a két tesztalanynak
- Az egyik tesztalany valóban ember, míg a másik egy gép
- Cél: Meggyőzni a bírálót arról, hogy gondolkodó emberek
- Ha 5 perc faggatás után sem tudja megállapítani, hogy melyik alany a gép, akkor a gép sikeresen teljesítette a tesztet
- Kritikák:
 - Nem minden ember tudná teljesíteni
 - A párbeszéd szimulálása kevésbé tekinthető az intelligencia jelének
 - Attól még lehet intelligens egy gép, hogy nem képes emberi módon kommunikálni
 - A teszten olyan ember is megbukhat, aki nem hajlandó együttműködni, és ez a gép részéről sincs kizárva
 - Kellően nagy adatbázissal az előre eltárolt kérdés-válaszok miatt is átmehet egy gép a teszten
- A tesztet általánosították az emberhez hasonlóan gondolkodó gépből kiindulva az emberhez hasonlóan cselekvő gép irányába.



- Turing azt feltételezte, hogy legkésőbb 2000-re lehetségessé válik egy olyan program megalkotása, amelynél öt perc „beszélgetés” után az átlagos felhasználó már csak 70%-os eséllyel tehet különbséget ember és gép között.
2014. június 7-én – Turing halálának 60. évfordulóján – egy Eugene Goostman nevű beszélgetőprogram egy kísérletben 33%-os arányban volt képes meggyőzni a felhasználókat arról, hogy ő egy Ukrajnában élő 13 éves gyerek (A vizsgálat Londonban történt)
- GPT-4 2023-ban a felhasználók 54%-át tudta meggyőzni, hogy ember (Az emberek 67%-ot értek el)

A mesterséges intelligencia alkalmazási területei

- logikai játékok (logical games)
- tételbizonyítás (theorem proving)
- automatikus programozás (automated programming)
- szimbolikus számítás (symbolic algebraic computation)
- látás, képfeldolgozás (vision)
- robotika (robotics)
- beszédfelismerés (voice recognition)
- természetes nyelvek feldolgozása (natural language processing)
- korlátozás kielégítés (constraint satisfaction)
- cselekvési tervek generálása (planning)
- szakértőrendszerek (expert systems)
- mesterséges neurális hálózatok (artificial neural nets).
- adatbányászat (data mining)
- ágensek, multi-ágensek (agents, multi-agents).

Logikai játékok (logical games)



Mottó: "A játék nem játék!"

- A játék lényege:
 - Ellenérdekű résztvevők felváltva érvényesített stratégiái a végső nyereség érdekében
 - Rendelkezik gazdasági, katonai vetülettel
 - A véletlennek nincs szerepe
- Motiváció: az ember szereti a szellemi kihívásokat (különösen a férfiak szeretnek versenyezni)
- Fő kérdés: Lehet-e a gépnek esélye az ember ellen?

Tételbizonyítás

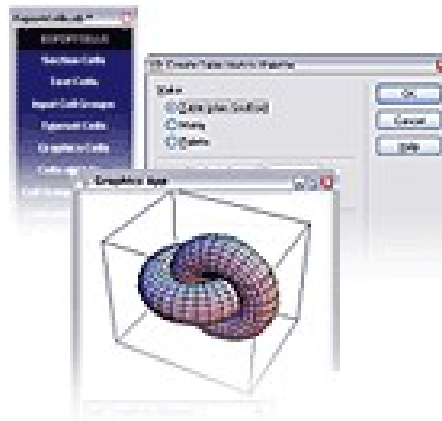
- Matematikai tételek bizonyítása az alapaxiómákból kiindulva, pl. kijelentés (propozíciós, nulladrendű predikátum-) kalkulust, illetve elsőrendű predikátumkalkulust használó MI programokkal, a rezolúció módszerével. A rezolúció lényege: lássuk be, hogy a tényekből, szabályállításokból és a bizonyítandó állítás negáltjából álló halmaz kielégíthetetlen, ellentmondásos. Ha sikerül, akkor a bizonyítandó állítás csak igaz lehet. (lásd később)
- A tételbizonyítók automatikus következtetések levonására alkalmasak alapaxiómákból, alapinformációkból kiindulva, következtetési szabályokat alkalmazva.
- A tételbizonyítók esetében, szemben a logikai programozási nyelvekkel (pl. Prolog), nem csak a logika, azaz a kiinduló ismeretek, tények és a szabályok megadása hárul a felhasználóra, hanem a vezérlést, azaz a következtetés, bizonyítás menetének módszerét is nyújtania kell. A tételbizonyítók kutatása elsősorban általánosan alkalmazható vezérlők megtalálását célozza.
- Konkrét alkalmazások: QA1, QA2, QA3 programnyelvek, a QA4 programnyelv procedurális reprezentációt is alkalmaz.

Automatikus programozás

- Cél: a szoftverkészítés munkájának automatizálása, olyan eszközök létrehozásával, melyeknél elegendő a megoldandó feladatot specifikálni, a megoldás algoritmusát és programját automatikusan készülni.
- Az automatikus programozás esetén a szoftver specifikációja értelem szerűen kisebb és könnyebben megadható, mint maga a program lenne valamilyen programnyelven

Szimbolikus számítás

- Matematikai levezetések, algebrai manipulációk, deriválás, integrálás azonosságainak, trigonometrikus, logaritmikus, stb. azonosságoknak az alkalmazása szimbolikus alakban adott feladatok megoldására.
- Ismertebb szimbolikus algebrai szoftverek: MACSYMA, REDUCE, CAMAL, LAM, ALTRAN, FORMAC, SYMBOL, MATHEMATICA



- A MATHEMATICA szimbolikus számítási program
 - Egyaránt végez numerikus és szimbolikus számításokat

`In[1]:= 3 + 62 - 1` `In[2]:= 3x - x + 2`

`Out[1]= 64`

`Out[2]= 2 + 2 x`

- Automatikusan végzi az alapvető egyszerűsítéseket:

`In[4]:= x^2 + x - 4 x^2`

`Out[4]= x - 3 x^2`

- Összetett kifejezésekre addig alkalmazza az átalakítási szabályokat, amíg a kifejezés tovább nem változik, egyszerűsödik

Gépi látás, képfeldolgozás



- Az ember a külvilágból szerzett információknak a legnagyobb részét látással szerzi meg. Ezért fontos az MI számára a mesterséges látás.
- A feladat: Adott egy kétdimenziós bit-térkép, ebből kiindulva meg kell adni a kép leírását, beleértve az alakzatok, méretek, színek, helyzetek paramétereit. Lényegében a nagyon alacsony szintű vizuális adatból egy magas szintű absztrakciót kell elérni, mely megfelel a képen látható objektumoknak.
- Fontos mozzanatok: az emberi látás jellegzetességeinek modellezése, pl.: élek, kontúrok detektálása.
- Az elért eredmények magukba foglalják az elektronikus regehártyát is, mely a regehártya sok funkcióját, köztük a látvány elsődleges feldolgozását is modellezi. Létrehozása megkönnyíti a mesterséges neurális hálók bemenőjeleinek előállítását.

- Input: kétdimenziós bitmap fájl alakjában a látvány, a kép.
- Output: a felismert objektumok és térbeli viszonyuk, fizikai jellemzőik.
- A képfeldolgozás lépései:
 - Élek detektálása
 - Mélység meghatározása
 - Alak meghatározása az árnyékoltságból
 - Vonalak címkézése
 - Objektum beazonosítás, helyzet meghatározás



Robotika

- A szó jelentése a rabota (kemény munka) szóból ered
- Mit értünk alatta:
 - Önálló tevékenységre képes
 - Mozgásra képes
 - Többnyire nyílt kinematikai láncú gép
 - A környezetével interakcióban van
 - Ismétlődő, az ember számára fáradságos vagy veszélyes munkát végez
- Célja: Az ember helyettesítése
 - Így egyre több emberi vonást mutatnak, melyek között a humán megjelenés, az emberszerű felépítés és a szándékos cselekvésre emlékeztető viselkedés egyaránt megjelennek
- A robotvezérlő intelligenciájának növelésével nemcsak emberhez hasonlóan gondolkodó, Turing tesztet teljesítő gépi intelligencia jöhet létre, hanem az emberhez hasonlóan cselekvő, humanoid, emberszerű gépi intelligencia is.
- Az ily módon létrejövő gépi intelligencia hasznosíthatósága (és veszélyessége) is megnő.



Beszédfelismerés

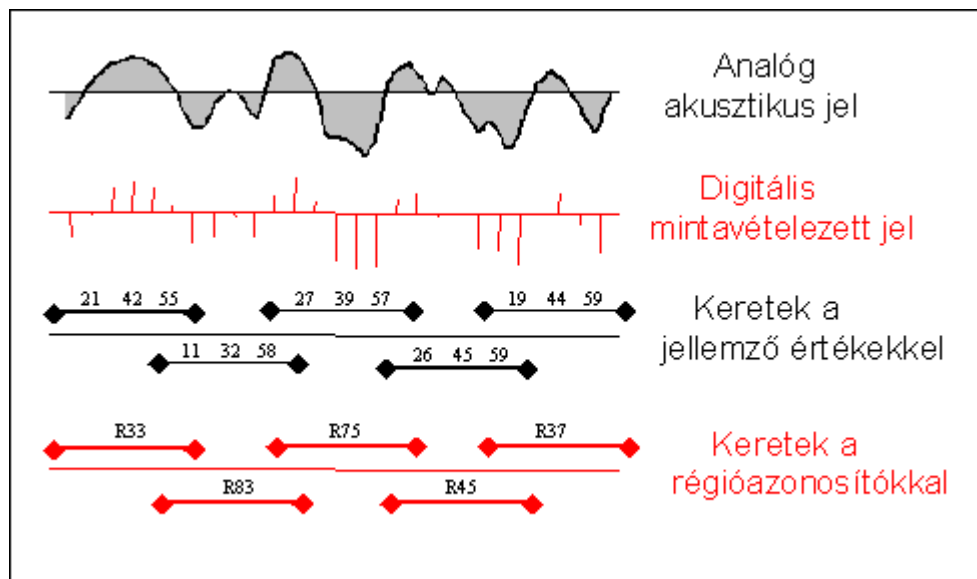
- Cél: Az emberi beszéd gép által kezelhető, szöveges formára alakítása, végső célként, a nyelvfeldolgozással egyesítve a beszélt nyelv gépi megértése céljából.
- Kezdeti eredmények: szűk szókincs, vagy ugyanazon beszélő esetén nagyobb szókincs felismerése.
- Az emberi nyelvek kb. 50 fonémát, beszédhangot különböztetnek meg.
- Szegmentálás, szavakra bontás → gond: a beszéd szavai egybefolynak.
- A fonémák és a szavak leírására használt betűk között nincs teljesen egyértelmű megfeleltetés → gond. (Pl: eltérő fonémák, azonos betűk: Mondd, Zsanett, kell findzsa? Vagy: azonos fonémák, eltérő betűk: Mi hájjal kenjük a kenyeret, Mihály.)

A hangjel feldolgozása

- Cél: az információ csökkentése és a jellemzők kiemelése.

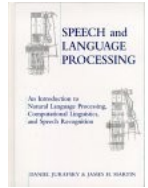
Lépések:

- Mintavételezés, kvantálás
- Jellemzők kinyerése, keretekben, azonos időintervallumokban
- Vektorkvantálás: a jellemzők hiperterének régióihoz rendelése a keretek jellemzővektorainak.



- A **beszédfelismerés** célja a beszélőktől független, de a beszédre jellemző paraméterek kinyerése
- A **beszélőfelismerés** ezzel szemben éppen a beszélőkre jellemző paraméterek kinyerését jelenti.
- A szavak egyértelmű felismerése további **statisztikai**, illetve **valószínűségi** adatok felhasználását kívánja meg.
- A legjobb rendszerek a szavak több, mint 95%-át jól ismerik fel.
- A beszédfelismerő rendszerek statisztikai, valószínűségi adatait általában betanítással adják meg.
- (Próbálják ki a Word diktálás funkcióját)

Számítógépes fordítás



- A természetes nyelvek feldolgozása nagy jelentőségű a számítógépes fordítás, tolmácsolás megoldásához is.
- Az emberi nyelv összetettsége miatt a megoldást csak az emberi intelligenciával, tudással összemérhető képességű géptől várhatjuk.
- A természetes nyelvek megértésének problematikájára jól rávilágít egy mondat értelmezésének 4 szintje:
 - Szintaktikai
 - Szemantikai
 - Pragmatikus
 - Intencionális.

- A mondat jelentésének négy szintje

Az egyes szintek tartalmát legjobban egy példa kapcsán mutathatjuk be:

- *Éva: Tudod, hogy Viktor ugyanúgy dohányzott, mint te?*
 - *Imre: Nem. Miért, mi van vele?*
 - *Éva: Tüdőrák. Feldobta a bocskorát.*
 - *Imre: Szomorúan hallom.*
- Figyeljük azt a mondatot: "**Feldobta a bocskorát.**"
 - Szintaktikailag (formailag) egy múlt idejű állítmányt és egy tárgyat figyelhetünk meg.
 - Szemantikailag (tartalmilag) azt jelenti: Felhajította a lábbelijét.
 - Pragmatikus (valóságos) jelentése: Meghalt.
 - A mondat intencionális (szándékolt) jelentése, célja a szövegkörnyezettel együtt Imre figyelmét felhívni, hogy ne dohányozzon annyit.

Korlátozás kielégítés

- A korlátozás kielégítési feladat a benne szereplő változók értékeit korlátozza. A korlátok megadhatók az értékek felsorolásával, explicit módon, vagy egy kifejezéssel, implicit módon.
A változók által felvehető értékek száma véges.
- A feladat megoldása alatt a változók olyan értékhalmozát értjük, melyek kielégítik az összes korlátozást.
- Megelégszünk egyetlen megoldással, bár gyakran több megoldás is lehetséges. Egyes esetekben az összes megoldást meg kell találnunk.
- A korlátozás kielégítés kedvenc feladata volt a 8 vezér probléma: úgy helyezzünk el a sakktáblán 8 vezért, hogy ne üssék egymást.
- A megoldó módszerek egyike a lehetőségtér állapotait tartalmazó fagráf „mélységben először” technikával történő bejárása.
- A megoldáskeresés gyorsítható a korlátozás propagáló, azaz a korlátokon kívül eső, megoldást biztosan nem adó pontok kizárásával.

Cselekvési tervek generálása

- A cselekvéstervezés lényege hatékony célirányos tevékenységsorozat generálása valamilyen, a világban felmerülő feladat megoldására.
- A megoldás meghatározására általános, azaz a problémától független valamint alkalmazás-specifikus módszerek közül választhatunk.
- Megoldási módszerek:
 - keresés - nehezíti a nagy elágazási tényező, s a talált megoldások a cselekvések egyszerű szekvenciái lehetnek csak.
 - Szituációkalkulus - nehezen irányítható és könnyen adódnak nem megfelelő lépések is. (A szituációkalkulus az elsőrendű logika módszerét alkalmazza a világ egy adott állapotára, azaz egy szituációra.)
- A cselekvési terv a két módszer együttes alkalmazásával áll elő, a finomító tervezési szakasz előre/hátra láncoló technikájánál a keresés jut szerephez.
- Végeredményként egy olyan cselekvésegyüttes jön létre, amely végrehajtható és a világot a megadott korlátokat kielégítő új állapotba viszi át.

Szakértőrendszerek

- Egy szakértőrendszer olyan eszköz, amely probléma specifikus ismeret megértésére képes és intelligensen használja a tématerület ismeretanyagát egy tevékenység különböző megvalósítási útjainak felvetéséhez.
- A szakértőrendszerek nem csak az ismeretátadás technikáit alkalmazzák, hanem analitikus, elemző eszközöket is az ismeret kiértékelésére, valamint tanulási technikákat.
- A szakértő személyeknél jobban vagy hozzájuk hasonló színvonalon tudják megoldani a legnehezebb problémákat.
- A szakértők által bevett gyakorlatok alapján heurisztikus magyarázatot szolgáltatnak.
- A felhasználókkal való interakció természetes nyelven és megfelelő módon történik.

- Sikeresen kezeli a bizonytalanságot bizonyos szabályok esetében, ahol ezeknek megfelelő adatokat használ.
- Képes „mérlegelni” a nagyszámú, egymással versengő, szimultán hipotézisek között.
- Meg tudja indokolni a következtetéseit.
- Egy szakértőrendszer általában egy tudásbázison és egy következtetési mechanizmuson alapul. A tudásbázis tartalmazza a szakértői tudást, azaz az információkat, amelyeket a rendszer használ a problémák megoldásához. A következtetési mechanizmus pedig az adott tudásbázis alapján dolgozza fel a bemenő adatokat, és hoz döntéseket, javaslatokat.
- Előnyei: Emberi hibák minimalizálása, az idő és költség megtakarítása, tudás megőrzése és megosztása

Példák olyan területekre, ahol szakértőrendszert alkalmaznak:

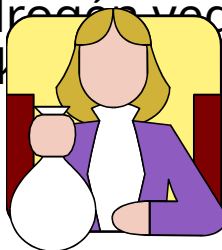
- Repülés: repülőgépmotor-diagnosztika: helikopter javítás, Pl.: NAVEX,
- Mezőgazdaság: Almáskertek gondozása (POMME),
- Kémia: Szerkezetértelmezés (DENDRAL),
- Oktatás: Tervezők oktatása konstrukciós tervezés ellenőrzésére (DECGUIDE),
- Vállalatvezetés: Üzlet hatékonyságelemzése (GURU),
- Egészségügy: Fertőző betegségek diagnózisa (MYCIN),
- Mérnöki technika, gépészet: Motoralkatrészek tervezése, stb.

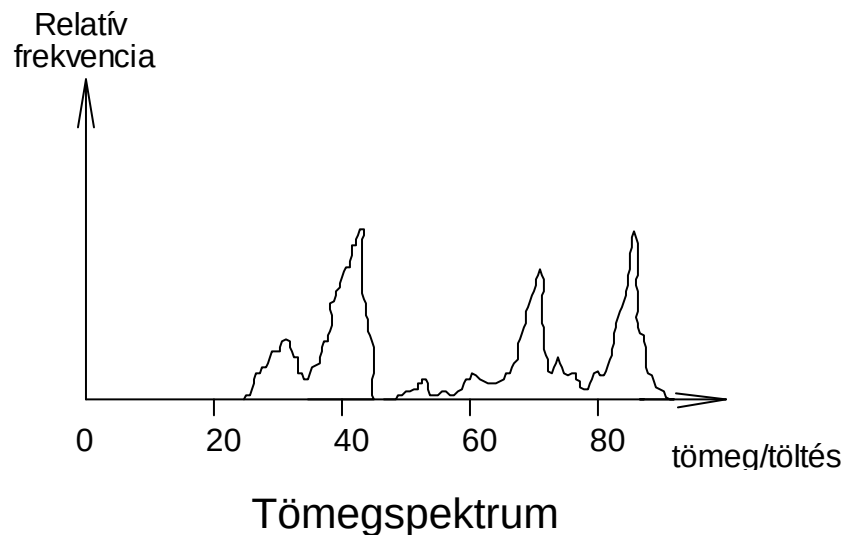
Szakértőrendszerek: a DENDRAL

- Ma is használt szakértőrendszer, több, mint 50 tudományos cikket írtak a vele elért felfedezésekről. (Feigenbaum, 1965)
- **Célja:** Hipotézis felállítása egy vegyület lehetséges molekuláris felépítésére, atomszerkezetére.
- Ha egy vegyész egy ismeretlen vegyülettel találkozik, első feladata, hogy meghatározza az összetevő atomokat és azok relatív arányát. Ehhez analitikai tesztekre és kísérletekre van szüksége. Az egyik gyakran alkalmazott műszer a tömegspektrométer. A DENDRAL is felhasználja a vegyület tömegspektrumát.
- Egy vegyület kémiai képletét egyszerű megtalálni, de **szerkezeti képletének** meghatározása speciális tudást igényel.

- **A DENDRAL három részből áll:**

- **Szerkezetgeneráló** rész, az összegképletből a lehetséges szerkezeti képletet generálja;
- **Spektrumjósító**, a szerkezeti képletből a megfelelő jósolt tömegspektrumot származtatja, és összehasonlítja az aktuális adatokkal;
- **Szerkezetjósító**, amely részszerkezeteket származtat le az adatokból és lehetetlen szerkezeteket kizár. Ez a rész felel meg a szakértő tudásának (1. + 2. elvileg elég, de kombinatorikus robbanás miatt nem lehet az összes szerkezetet generálni és vizsgálni. pl.: egyes szénhidrogén vegyületek 10000 lehetséges szerkezettel is rendelkeznek)





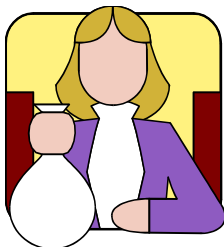
i. szabály: ha **létezik egy magas csúcs a 71 tömeg/töltés helyen és**

ha létezik egy magas csúcs a 43 tömeg/töltés helyen és

ha létezik egy magas csúcs a 86 tömeg/töltés helyen és

ha van néhány csúcs az 58 tömeg/töltés helyen

akkor az N-propil-keeton 3 szerkezet jelenlétének feltételezhető.



Hol találkozhatunk a mesterséges intelligenciával?

- E-kereskedelem

- Személyreszabott vásárlás

- A mesterséges intelligenciát ajánlómotorok létrehozására használják
 - Cél a jobb kapcsolatba lépés az ügyféllel, és a márkahűség javítása
 - Figyelembe veszi a böngészési előzményeket, a preferenciákat és az érdeklődési köröket

- Mesterséges intelligencia alapú asszisztensek

- A virtuális vásárlási asszisztensek és chatbotok javítják a felhasználói élményt az online vásárlás során
 - Természetes nyelvfeldolgozást arra használják, hogy a beszélgetések a lehető legemberibb és legszemélyesebb hangzásúak legyenek

- Csalásmegelőzés

- Hitelkártya-csalás és hamis értékelés megakadályozására
 - Használati szokásminták figyelembevételével a mesterséges intelligencia csökkenti a hitelkártya-csalás lehetőségét (Repülőtéren a táskánkhoz nyomott terminál általi csalást is akár)

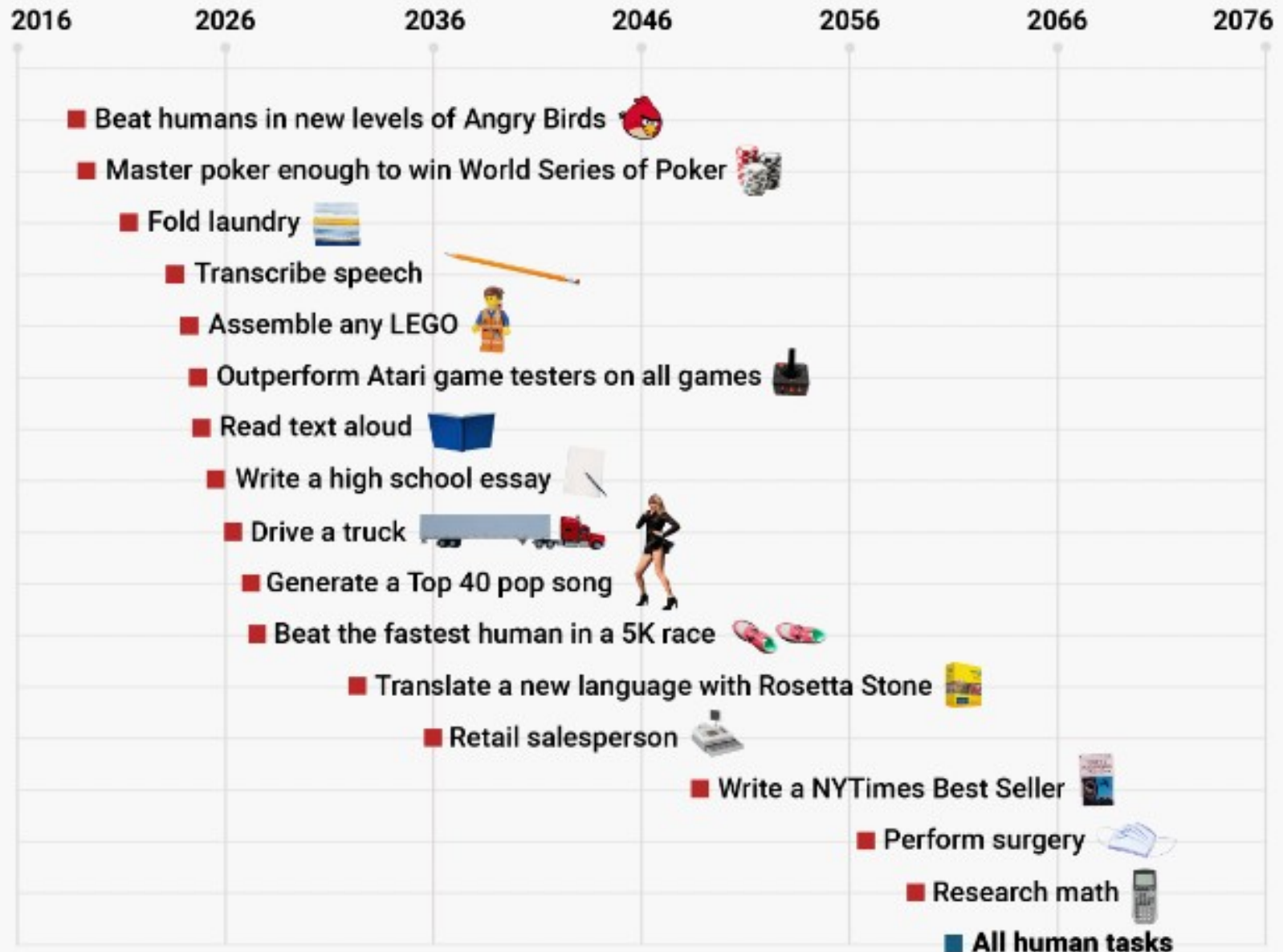
- Önvezető járművek
 - Az autógyártó vállalatok gépi tanulást alkalmaznak arra, hogy a számítógépeket úgy tanítsák meg, hogy az emberekhez hasonlóan gondolkodjanak és fejlődjenek bármely környezetben történő vezetés során és detektáljanak objektumokat az ütközések elkerülése érdekében
- Spam szűrő
 - Levélszemét kiszűrésére használnak mesterséges intelligenciát
 - A Gmail szűrési kapacitása körülbelül 99.9% (*Van-e tapasztalat más levelezőrendszerekkel?*)
- Arcfelismerés
 - A kedvenc eszközeink (mobiltelefon, laptop, stb) arcfelismerési technikákat alkalmaznak hogy növeljék az eszköz biztonságát
 - Alkalmas megfigyelőkamerák képe alapján bűnözők, eltűnt emberek beazonosítására
 - A koronavírus fertőzés alatt a maszkot nem viselő embereket is beazonosította
 - Alkalmas bizonyos genetikus betegségek diagnosztizálására is
 - Alkohol vásárlásakor a személy korának meghatározására

- Robotika

- Dohányboltban, bevásárlóközpontokban
- Takarítórobot, robotporszívó
- Kórházi robotok, sebészeti robotok
- Robotpincér
- Fűnyírórobot
- Ipari robotok, szerelőrobotok (például Kuka)



WHEN JOBS/TASKS WILL BE TAKEN OVER BY MACHINES



Mesterséges intelligencia alapjai

Tudásbázis

Tudás

- Központi fogalom
- tudás = az elvégzendő feladat végrehajtásában hasznosnak bizonyuló bármely ismeret
- Bár az intelligenciát a vele foglalkozó tudósok sokféleképpen definiálják, abban egyetértenek, hogy az emberi intelligenciával összemérhető képességű gépi intelligenciának rendelkeznie kell az ember általános tudásával.
- Az ilyen tudással, ismerethalmazzal bíró mesterséges rendszer intelligenciája ezen ismerethalmaz szervezésétől függ.

A tudásszemléltetés szükségessége

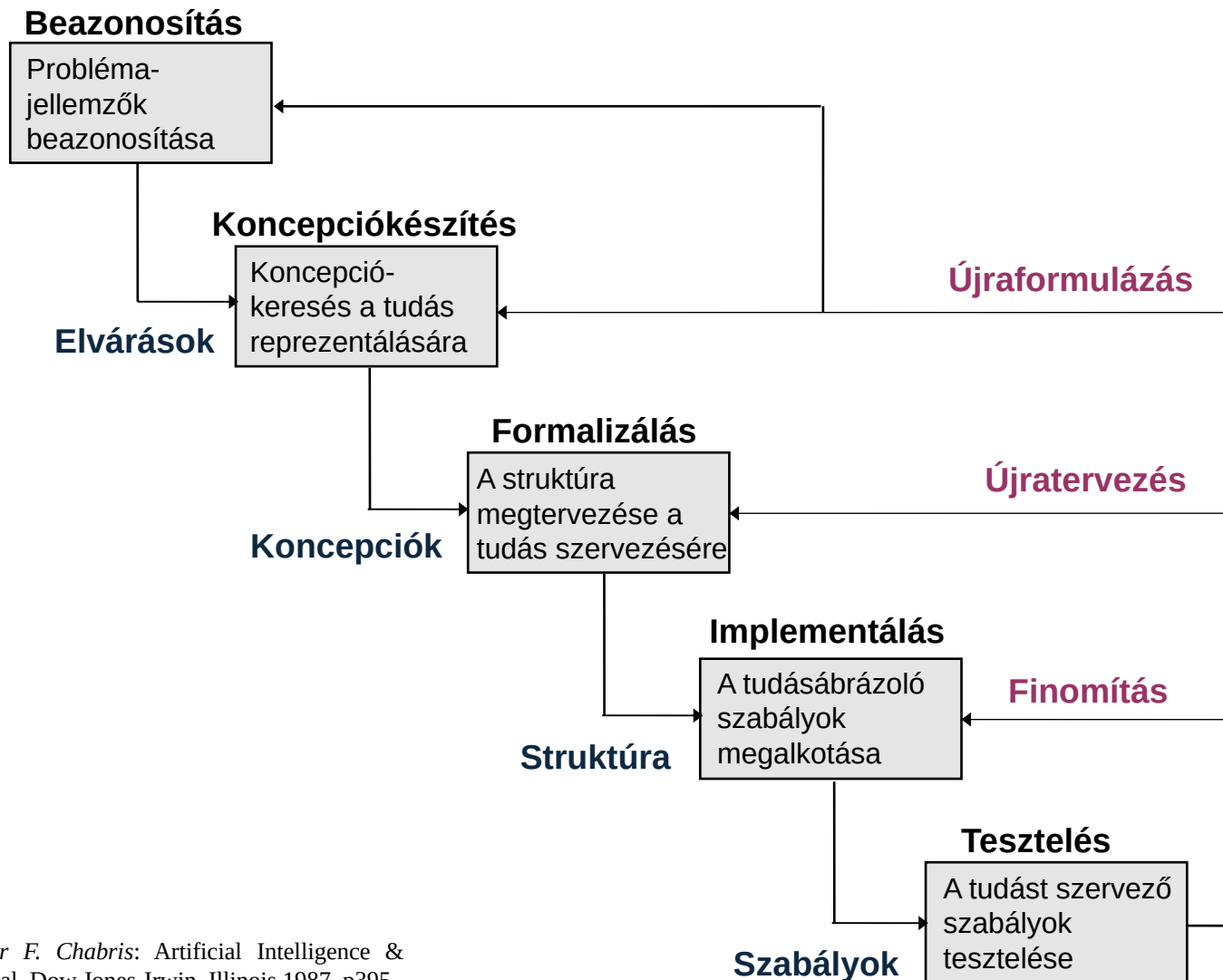
- A valós világ objektumairól, azok viszonyáról, kapcsolataikról rendelkezésre álló ismeret ritkán adódik a számítógép által kezelhető formában.
- Ahhoz, hogy a számítógép az ismereteket tárolni, kezelni tudja, azok kódolására van szükség.
- A kódolás módja nagyban kihat a gép általi feldolgozás gyorsaságára, hatékonyságára, a tárolt tudáson alkalmazható gépi műveletekre, mint pl. keresések, illesztések, összehasonlítások, láncolások, kapcsolatok kialakítása, stb.
- A megfelelő ismeretstruktúrát alkalmazó tudásszemléltetés az MI kulcskérdése. Oka: korlátos gépidő és tárkapacitás

- Megfelelő színvonalú működés reálisan csak párhuzamos hardverrendszerektől remélhető.
 - Súlypont áthelyezés a statikus tudásszemléltetés, tudásszemléltető modellek megalkotásáról és használatáról
- 
- Dinamikus, önszervező ismeretábrázolási modellek
 - Automatikus ismeretszerző és ismeretstruktúráló tevékenysége ugyanolyan fontos része a működésnek, mint az ismeret alkalmazása
 - Öntanulóak, az ismeret megszerzéséhez nem igényelnek emberi közreműködést
 - Előtérbe kerül a szimbolikus jelleg
 - A fogalom jelentése: mindaz, ami hozzá asszociálódott
 - Térbeli, időbeli, minőségi, stb vonatkozásban

A tudásszemléltetés elvárt jellemzői

1. A fontos dolgokat világosan adja meg.
2. Fedje fel a természetes korlátokat, megkönnyítve a számítások néhány fajtáját.
3. Legyen teljes.
4. Legyen tömör.
5. Legyen átlátható számunkra.
6. Legyen alkalmas gyors feldolgozásra.
7. Rejtse el a részleteket, de tegye elérhetővé azokat szükség esetén.
8. Létezzen rá számítógépi eljárás.

A tudásfeldolgozási folyamat vízesésmodellje



Tudásfeldolgozás

- A tudásfeldolgozás az a folyamat, amelyben a szintetizált tudást a számítógépbe juttatjuk abból a célból, hogy a problémákat a tudásbázison végzett elektronikus szimbolikus manipulációval és következtetéssel megoldjuk
- A szakterület szakértője nem elegendő egyedül, hogy létrehozza a tudásbázist, együtt kell működnie az úgynevezett „tudásmérnökkel” (knowledge engineer), aki megteremti a szakértőrendszer számára fogyasztható tudásreprezentációt, ez lesz a tudásfeldolgozás
- A gazdaságos és hatékony ismeretalapú rendszer létrehozásának kulcskérdése a megfelelő tudásfeldolgozás
- A tudásalapú rendszerek fejlesztésének egyik első teendője a *tudásgyűjtés* (Knowledge Acquisition). Egyben ez a tudástechnológus munkájának egyik legfontosabb részese is

Tudásgyűjtés

- *Tudásgyűjtés, tudáskinyerés* az a folyamat, amelynek feladata beazonosítani, kinyerni, dokumentálni és elemezni a szakterület szakértőjének információ-feldolgozó tevékenységét abból a célból, hogy meghatározásra kerüljön egy szakértőrendszer tudásbázisa és következtető automatája.
- A tudáskinyerés négy fő szakasza:
 1. Az előzetes tudás és problématerület feltárása
 2. Az információforrások beazonosítása
 3. A részletes tudás kinyerése a forrásokból
 4. A kinyert tudás elemzése, kódolása és dokumentálása.

Előzetes tudásfeltárás

- Célja:

- a rendszer által kezelendő problémák tartományának behatárolása,
- a problématartomány jellemzőinek behatárolása,
- a tématerület tudásmennyiségének behatárolása,
- a felhasználók elvárásainak megismerése,
- néhány tipikus következtetési fordulat megismerése,
- a szakértők szakterülettel kapcsolatos elképzeléseinek megismerése,
- fő szabályok és koncepciók definiálása.

Eszközei:

- előzetes archívum-kutatások,
- konzultációk szakértőkkel.

Tudás kézikönyv

A kezdeti feltáró munka végén a tudástechnológusnak egy dokumentummal kell rendelkeznie, egy kézikönyv formájában, melynek a következőket kell tartalmaznia:

- Az általános problémaleírást
- Kik a felhasználók és milyen elvárásokkal
- A problémákra és résztartományokra való lebontást a későbbi tudáskinyeréshez
- Egy részletesebb leírást a résztartományokról, melyet fel lehet használni a prototípusfejlesztésben
- A referencia dokumentumok nyilvántartását
- A szakterület terminológiájának, elveinek, szakkifejezéseinek listáját
- A prototípus kifejlesztésében közreműködő szakértők nevét
- Néhány ésszerű teljesítménymutatót a rendszer számára
- A tipikus következtetési módszer leírását.

Információforrások

- A szakterület szakértőjének kiválasztása
 - A szakértő az, aki speciális szakértelmet birtokol, hordoz magában, vagy mutat, illetve olyan tudást, melyet gyakorlással, vagy tapasztalással szerzett
- Szakértő annyira rutinszerűen végezhetnek egy tevékenységet, hogy nem tudatosul bennük, mit miért csinálnak
- A tapasztalatok heurisztikaként is reprezentálhatóak
- A tartomány szakértőjének hitelessége
 - Hitelesnek kell lennie az alábbi szerepkörök előtt:
 - Felhasználók
 - Projekt team
 - Szakértők közössége
 - Szervezet vezetése

- Szakértő fajtái:
 - Akadémikus:
 - Fontos számára az elméleti háttér
 - Szakterületüket logikusan felépített szerkezetűnek tartják
 - Tevékenységének része a dolgok tisztázása, más tanítása, így rengeteget beszélnek a szakterületről
 - Esetenként távol állhatnak a valódi, mindennapi problémamegoldástól
 - Gyakorlati szakértő:
 - Napról-napra részt vesznek a szakterület problémáinak gyakorlati megoldásában
 - Gyakorlatuk sokszor implicit
 - Céljuk egy olyan megoldás, amely működik a megszorítások és korlátozott források mellett
 - Heurisztikák dominálnak, sokszor nincs jelen megfelelő elméleti háttér
 - Szamuráj:
 - Tisztán a végrehajtás szakértője
 - Egyetlen célja a tevékenység véghezvitele úgy, hogy optimális eredményt kapjunk
 - Képzése sokszor kizárólag gyakorlati jellegű
 - Válaszai gyakran automatikusak
- A szakértő motiválása
 - Ez a tudáskinyerés záloga
 - A szakértők gyakran érzik veszélyeztetve a projekt miatt a pozíciójukat vagy státuszukat
 - A klasszikus mondás, „a tudás hatalom” megfordul, a tudás elvesztése a hatalom elvesztése
 - Meg kell különböztetni helyi és kozmopolita szakértőt
 - A helyi szakértő lojalitása elsődlegesen a cégéhez fűződik
 - A kozmopolita szakértő a szakma legjobbjaihoz méri magát
 - Eltérő motiválást igényelnek

Tudáskinyerés

- A tudáskinyerés tárgyat két csoportra bontjuk:
 - Előzetes tudás
 - A bevezető interjúk és konzultációk alatt:
 - Az alap-, gyakran primitívnek nevezett terminusok és elvek beazonosítása
 - A rendszer tipikus inputjainak és outputjainak behatárolása
 - A tipikus megoldások, vagy megoldási osztályok behatárolása
 - A rendszer kezdeti implementációjához problémakezelő stratégiák találása
 - Részletes tudás

Részletes tudás

- A szakértő privát tudása, amely tudás több éves tanulást és tapasztalatot tükröz
 - A változatos adatok és szabályok közötti viszonyok beazonosítását
 - A szabályok hierarchiájának beazonosítását
 - Az adatok relatív értékének és fontosságának megítélését
 - Az adatok bizonyosságának és relatív valószínűségének megítélését
 - A szakértő közelítései alapjának meghatározását
 - A feladatok prioritásának és sorrendjének megítélését
 - A szabályok és a következmények közötti konfliktusok megoldásának meghatározását
 - A problémamegoldás alternatív lehetőségeinek felismerését
 - A következtetésekben elvégezhető ésszerűsítések meghatározását
 - Részletes válaszokat a várt és a váratlan szituációkra
 - A különféle szabályokban, eredményekben és adatokban való hit fokának megadását
 - A különféle célok és részcélok input követelményeinek megértését
 - Megfelelő teljesítménymércék meghatározását.

Tudáskinyerési technikák

- Cél minimalizálni a begyűjtésre és begyűjtött adatok analizálására fordított munkát, valamint a drága és nehezen elérhető szakértővel töltött időt, mellette maximalizálni a szerzett és használható ismeretek mennyiségét

1. Interjú

- Különösen az előzetes információ kinyerésére előnyös.
- Két típusa:
 - Strukturálatlan
 - nyitott-végű.
- A strukturálatlan interjúnál a tudástechnológus engedi a szakértőnek, hogy elveket, terminológiát mutasson be és meghatározza az interjú menetét. A tudástechnológus csak rögzít. Alacsonyabb hatékonyságú. Szakértő feleslegesen sokat beszélhet, esetleg eltúlozza egy általa fontosnak tartott téma jelentőségét.
- A nyitott-végű interjú feltételezi, hogy a tudástechnológus rendelkezik előzetes háttér-információkkal. A tudástechnológus irányítja az interjú menetét kérdésekkel, de a szakértő szabadon válaszolhat, vagy kitérhet a válaszadás elől.

<i>Kód</i>	<i>Kérdés sablonja</i>	<i>Hatás</i>
K1	Miért csinálná ezt?	Szabállyá alakít egy kijelentést.
K2	Hogyan csinálná ezt?	Alacsonyabb szintű szabályokat generál.
K3	Mikor csinálná ezt? Igaz, hogy mindig <a szabály> van érvényben?	Rámutat a szabály általánosságára, és újabb szabályokat generálhat.
K4	Milyen alternatívái vannak <a leírt döntésnek/ cselekvésnek>?	Újabb szabályokat generál.
K5	Mi lenne, ha nem lenne igaz <egy aktuálisan igaz állapot>?	Szabályokat generál olyan esetekre, melyeknél nem állnak fenn a jelenlegi feltételek.
K6	Tudna többet mondani a <bármely eddig megemlített téma>-ról?	További párbeszédet eredményez, ha kezdenénk kifogyni a témákból.

SZ: Megvizsgáltam a számítógép portját.

TM: (K1) Miért vizsgálta meg a portot?

SZ: Ha előtte villámlás volt, akkor általában jó ötlet megvizsgálni a gép portjait, mert a villámlás kárt szokott tenni bennük.

TM: (K4) Vannak más alternatívái ennek a problémának?

SZ: Igen, a fentit meg kell előznie egy kérdésnek, hogy volt e több furcsán viselkedő billentyű – nem feltétlenül mind, de kettőnél több.

TM: (K1) Miért kell, hogy kettőnél több billentyűről legyen szó?

SZ: Ha csak egy vagy két billentyű viselkedne furcsán, akkor meg kellene vizsgálni, jók-e a kontaktusaik. A sebesség minden billentyűre kihatna, a paritás kb. a billentyűk felére.

2. Protokoll elemzés

- Leggyakrabban alkalmazott tudáskinyerési módszer a részletes tudás kinyerésére
- A protokoll a szakértő információ-feldolgozó tevékenységének és döntéshozó viselkedésének lépésről-lépésre történő rögzítése
- Online: Szakértőről problémamegoldás közben készül felvétel, melyhez ezzel párhuzamosan megjegyzések is készülnek. Ha a szakértő kommentálja, akkor önbeszámoló, ha egy másik szakértő, akkor árnyékbeszámoló
- Offline: A problémamegoldás folyamata után a szakemberek visszamenőleg teszik meg a kommentárjaikat
- Figyelmesen kell megválasztani a protokoll elemzésre kerülő problémákat az idő kötöttsége miatt, azok legyenek reprezentatívak.

2. Végigvezetés

- Általánosan alkalmazzák
- A tudásmérnök megkéri a szakértőt, hogy vezesse őt végig a feladat megoldásában és felügyelje

2. Kérdőívek

- Szabad-végű: Akkor válaszol, amikor ráér és kedve van
- Rövid válaszós: Leíró jellegű, tömör válaszok adására van mód, gyorsan elkészül
- Erőltetett válaszós: Válaszlehetőségek közül csak választania kell

5.Szakértői beszámoló

- A szakértő saját írásos összegzése a problémamegoldó viselkedéséről. Kétféle forma:

a) A szakértő megadja:

- feladat megközelítési módszerét
- stratégiáját
- a szabályokat, melyeket használ
- a szükséges adatokat
- az eredményt
- A tudástechnológus:
 - feldolgozza a beszámolót,
 - kinyeri az ismereteket.

b) A szakértő formálisan, például folyamatábra alakjában adja meg az általa követett döntéshozás lépéseit, mentesítve ezen feladat alól a tudástechnológust.

Kinyert tudás elemzése, kódolása, dokumentálása

- Ebben a fázisban a következőket kell elvégezni: formális módszerekkel elemezni kell a kinyert tudást, dokumentálni kell a problématerületet érintő szabályokat, hálózatokat, viszonylatokat, jellemzőket.
- A dokumentálás helye a Tudás Kézikönyv.
- Elemzés négy tipikus lépése:

1.Átírás

- A szóbeli információk első írásos alakjának letisztázása, minden egyéb járulékos információt, például a technikai segédszemélyzet által rögzített vizuális anyagokat is beleértve

2.Szövegrész-indexelés

- Lényege az átirat tagolása rövid szövegrészekre, melyeket indexekkel látnak el
- Egy ilyen egység célszerűen megfelel egy tudáselemnek, azaz egy elemi feladatnak, állításnak vagy adategyüttesnek

3. Tudáskódolás

- Elemzik a tudást aszerint, hogy
 - mivel foglalkozik,
 - milyen típusú operátorokat tartalmaz,
 - milyen kiértékelési kritériumot tartalmaz
 - A választásokat és döntéseket megalapozó következtetéseket tartalmaz
- Két fő kategóriája:
 - Descriptív (leíró) tudás
 - Procedurális (eljárásokra vonatkozó) tudás
- Például: „A sikerre 75% esély van” tudáselemhez a „Megoldás kiértékelés/kritériumbeállítás(valószínűség)” megjegyzés kerül feljegyzése

3. Dokumentálás

- Helye a Tudás Kézikönyv
- Részei:
 - Átfogó tartománylista
 - Descriptív (leíró) tudás
 - Procedurális (eljárásokra vonatkozó) tudás
 - Szójegyzék

Gépi tanulás

- **Feleslegessé válik a szakértő is.**
- A szakértőrendszerek on-line adatbázisokat fognak letapogatni, könyveket, folyóiratokat és magazinokat fognak digitalizálni. Más számítógépes rendszereken tárolt adatokat elektronikus vonalon lehívnak, hogy származtassák, vagy aktualizálják az ismeretbázist, mindezt emberi közreműködés nélkül.
- Az automatikus tudáskinyerés potenciális **előnyei** a következők:
 - Jobb eredményeket érhetnek el, mint az emberek, különösen amiatt, hogy az információkezelés egyre nagyobb méretű és az információ egyre komplexebb, és ily módon nagyobb szakértőrendszereket hozhatnak létre.
 - Csökkentik a magas költségű emberi munkaerő iránti igényt és a tudásbázisok kifejlesztéséhez szükséges időt.

Előállító szabályok, szabályalapú tudásszemléltetés

- ***Előállító szabály (production rule):***

- egy IF-THEN, feltétel-következmény szerkezet, melyet egyaránt alkalmaznak a problémák megoldásához szükséges deklaratív (leíró) és procedurális (eljárásokon alapuló) tudás ábrázolására. A logikában neve: implikáció, modus ponens: $\frac{\alpha, \alpha \rightarrow \beta}{\beta}$

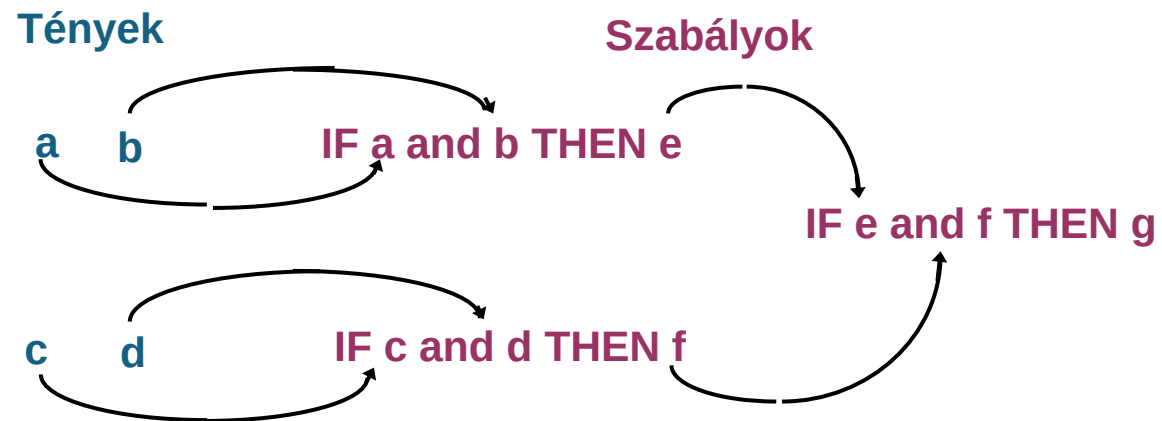
- **Deklaratív** következmény: kikövetkeztetett tény
- **Procedurális** következmény: akció.
- **Példa: IF hőmérséklet>30 THEN hűtés**

- **Szabályláncolás - a következtetési folyamat megvalósulása**

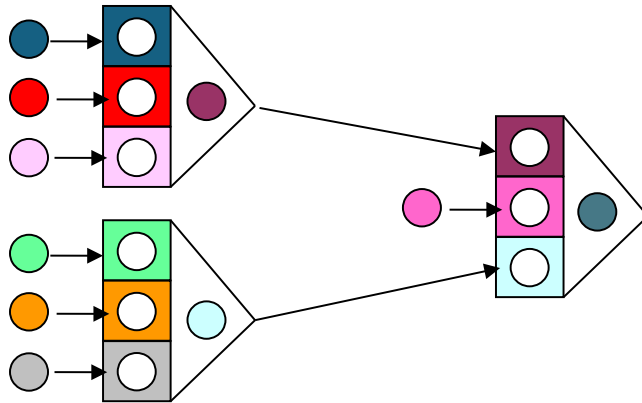
***szabályláncolódás* = a szabály következményrésze illeszkedik más szabály feltételrésszére**

- Ha az összes kapcsolódást megvalósítanánk, előállna a szabálybázis által reprezentált összes következtetési fa, melyek egymáshoz rendelnék a leveleiken szereplő tényeket és a fák csúcsán szereplő végkövetkeztetéseket.
- A következtetési **fa teljes előállítás** **gyakorlatilag lehetetlen**, ezért egy konkrét esetre a következtetési feladat végigvitele a **következtető automatára** hárul.

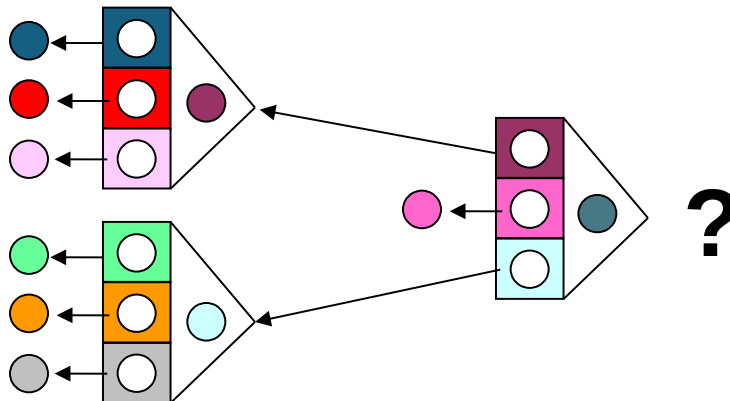
Szabályláncolás



- **Előrehaladó láncolás** (forward chaining): a végkonklúzió megtalálása a feladat, a megadott tényekből kiindulva

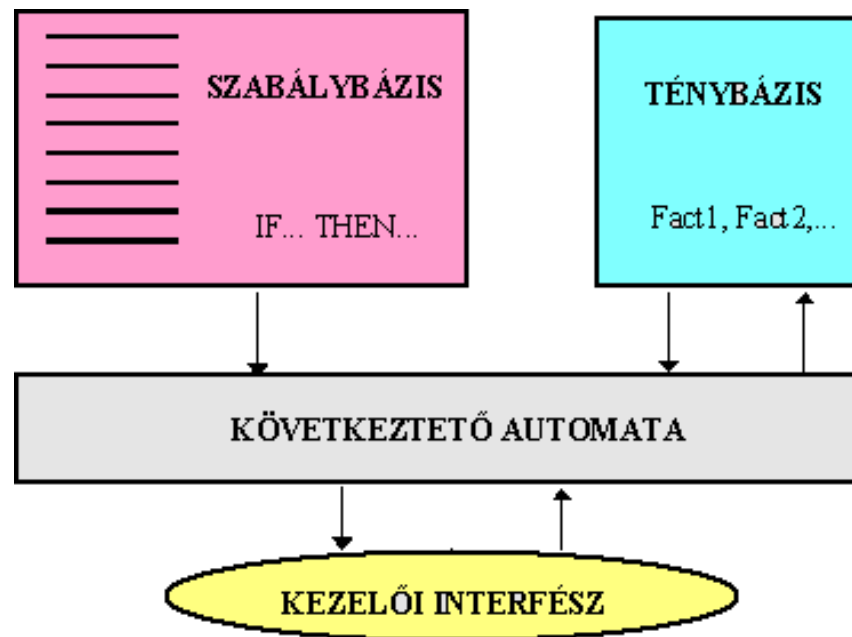


- **Hátrafelé haladó láncolás** (backward chaining): az elérendő cél adott és a szükséges előfeltételek megtalálása, ill a megadottak elégséges voltának megállapítása a cél.



- **Előnyös alkalmazási terület** (Christopher Chabris):
 - **Előrehaladó láncolás:**
ha a megoldandó problémával kapcsolatosan nagyszámú összegyűjtött adattal rendelkezünk, de a megoldást illetően nincs jó elképzelésünk. A szabályalapú rendszer el fogja végezni az összes végrehajtható következtetést és **elő fogja állítani a megadott tényekből következő összes szóbajöhető megoldást.**
 - **Hátrafelé haladó láncolás:** amikor egy, vagy több **hipotézissel rendelkezünk** a problémánk megoldására vonatkozóan és azt kívánjuk, hogy a szabályalapú rendszerünk tesztelje ezeket. A hipotézistől visszafelé indulva a rendszer bekéri a hipotézis teljesüléséhez szükséges adatokat.
- Az **előreláncolást** alkalmazó rendszert **deduktív rendszernek** is szokták nevezni utalva a sorozatban végzett deduktív logikai következtető műveletre (modus ponens).

Szabályalapú tudásszemléltetést és következtető automatát alkalmazó szakértőrendszer felépítése



Szabályalapú következtetés működése előrehaladó láncolásnál

- **Feladat:** A szabálybázist és az aktuális feladat tényeit felhasználva meg kell próbálni elérni a célállapotot, mely a problémára adott válasz, vagy egy elfogadható választ reprezentáló rész cél elérése lehet.
- **Lépések:**
 1. *Mintaillesztés:* ki kell keresni a szabálybázisból az összes olyan szabályt, melynek a feltételrésze a ténybázisbeli tényekkel igaz.
 2. *Konfliktusfeloldás:* az 1. lépésben megtalált szabályok alkotják a konfliktus-halmazt, mert bármelyik felhasználható, de csak egyet alkalmazhatunk.
 3. *Szabályalkalmazás:* végre kell hajtani a kiválasztott szabály következmény-részeiben szereplő tevékenységet, ill. fel kell használni az új tényt. (Új tény: az adott feladatra nézve igaz, kikövetkeztetett.)
 4. *Célállapot tesztelése:* meg kell vizsgálni, eljutottunk-e a

- **A szabályalapú rendszerek előnyei**

- Modularitás: egyedi előállító szabályok hozzáadhatók, törölhetők, megváltoztathatók
- Egyöntetűség: homogén ábrázolás, könnyű megértés
- Természetesség: emberi problémamegoldáshoz hasonló.

- **A szabályalapú rendszerek hátrányos tulajdonságai**

- Merevek: nem nyújtanak különféle absztrakciós szinteket
- Nem hatékonyak: ténybázis-szabálybázis illesztés, kombinatorikus robbanás.

A szabályalapú tudásszemléltetés **rendkívül elterjedt** és számtalan szakértőrendszernek és szakértőrendszer-váznak alkotja az alapját.

Mesterséges intelligencia alapjai

Tudásrepresentáció

Szemantikus háló alapú tudásszemléltetés

Ross Quillian, 1968

- **Biológiai ihletés:** az agy fogalomábrázolásának feltételezett formája
 - objektumok, koncepciók, fogalmak - csomópontok
 - viszonyaik, kapcsolataik - gráfélek.
- **Hierarchikus modell** oka: az ember kognitív (megismeréssel kapcsolatos) működésére vonatkozó kísérletek az objektumok specifikus jellemzőinél gyorsabb válaszidőket eredményeztek, mint az általános, magasabb szintű kategóriákhoz tartozó jellemzőknél (*Kanári - énekel? Kanári - repül? Kanári - bőre van?*)
Következtetés: hierarchikus egyed - alosztály - osztály kapcsolat valószínű.
- **Cél:** Az emberi információátárolás és visszakeresés modellezése.

- Szoftveralkalmazás, célja: a természetes nyelvek megértésének gépi modellezése.
- **Quillian: "egy szó jelentését meg lehet kapni a szóhoz társított szövegek halmazaként"**
- Programjával az ember gondolkodási folyamatának azt a részét modellezte, amelyet az ember akkor végez, amikor egy **lexikon két szava között** keres kapcsolatot. Egy szemantikus háló jól definiált eljárásainak segítségével képes volt összeállítani egy választ bármelyik, a szótárban megtalálható szópár esetében azok összevetésére és szembeállítására.
- Megkereste a szavakhoz kötődő ismeretek közös részeit, kapcsolódási pontjaikat. Emiatt szokták **asszociatív hálónak** is nevezni a szemantikus hálót.

- Példák a Quillian által elkészített program működésére szavak közötti kapcsolat keresésekor:

1.Összevetés: *Sírni, Kényelem*

A. Kapcsolat: *Szomorú*

(1) A *sírás* azon dolgok közül való, amelyek *szomorú* hanggal párosulnak.

(2) A *kényelem* érdekében valamit kevésbé *szomorúvá* tehetünk.

2.Összevetés: *Növény, Élő*

A. 1.Kapcsolat: *Élő*

(1) A *növény élő* szervezet.

B. 2.Kapcsolat: *Élő*

(2) A *növény* olyan szervezet, amely a levegőből is vesz magához táplálékot.

Ez a táplálék olyan dolog, amely kell az *életéhez*.

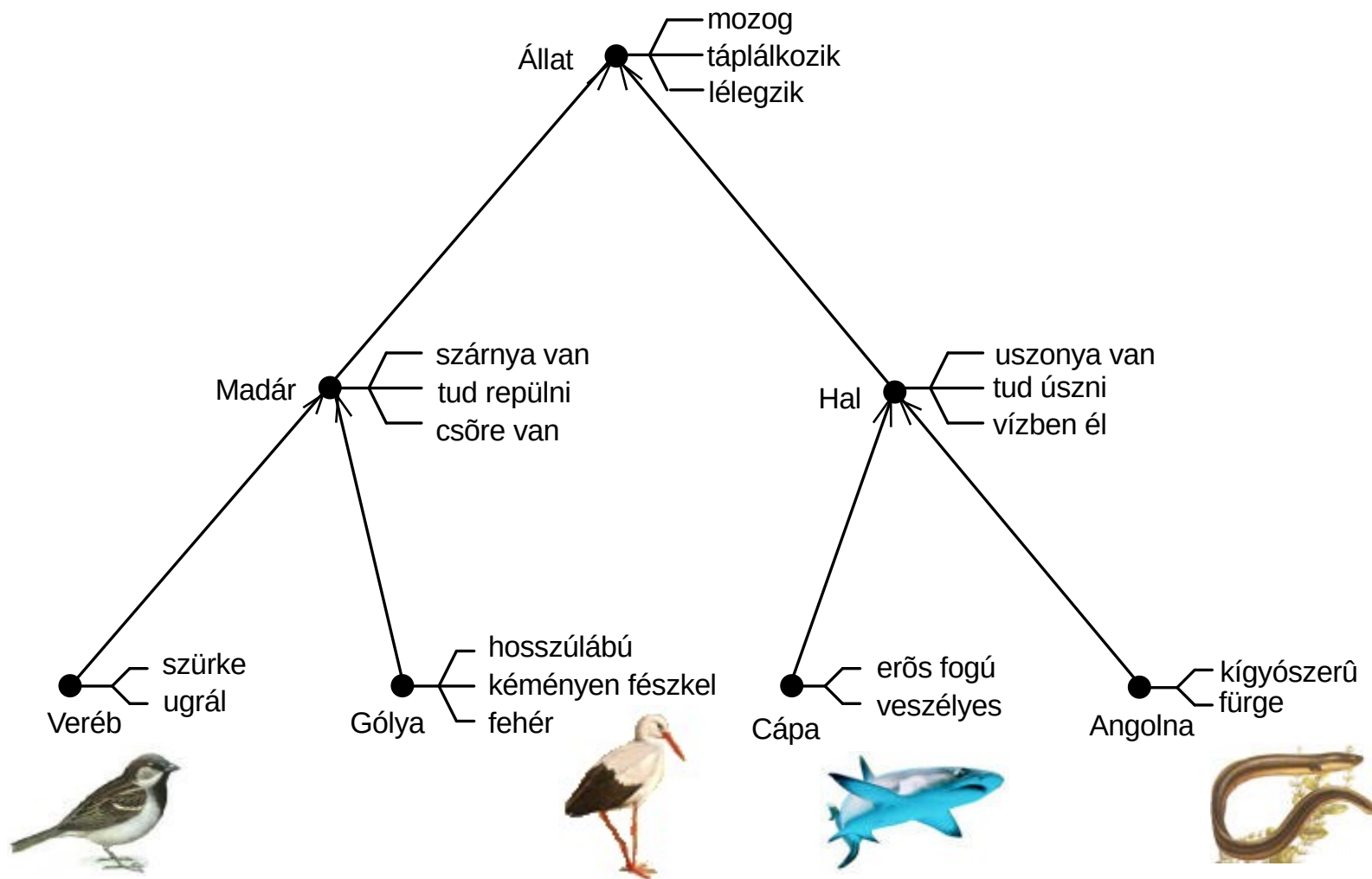
3.Összevetés: *Növény, Ember*

A. Kapcsolat: *Emlős*

(1) A *növény* nem *emlős* szervezet.

(2) Az *ember* egy *emlős*

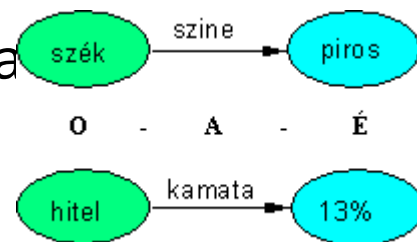
Példa



- A nyilak az **egyed-alosztály** (instance_of) és az **alosztály - osztály** (is_a) kapcsolatot jelzik, a többi gráfél a **tulajdonsága** (featured__by) kapcsolat megadására szolgál. További lehetséges kapcsolatformák: *típusa; van_neki; tud; azonos_vele; stb.*
- Léteznek gazdagabb kapcsolatkészlettel dolgozó szemantikus háló reprezentációk is.
- **Következtetés, feladatmegoldás szemantikus hálóval**
Egy szemantikus hálóval ábrázolhatjuk egy tématerület ismereteit. A tématerülettel kapcsolatos ismeretek alapján megválaszolható kérdést szintén egy neki megfelelő szemantikus hálóval adjuk meg (**célháló**), majd ezt a kisebb hálót illesztjük a tématerület hálójának azonos csomópontokat tartalmazó részére. A kérdésre a választ a tématerület hálójának illeszkedő része hordozza. Általában a hálókezelő algoritmusra ennél több feladat hárul, elő kell állítania például az öröklött tulajdonságokat is.

Az egyed - alosztály - osztály kapcsolat

- **Egyed:** van olyan tulajdonsága, amely csak rá igaz
- **Osztály:** olyan kategória, melynek jellemzői több egyedre, vagy alsóbb osztályra is igazak, így ezek az egyedek, vagy alosztályok közös tulajdonságaik alapján egy magasabb szintű osztályba sorolhatók.
- A közös tulajdonságot csak az osztálynál kell tárolni, az egyedekre, vagy alosztályokra **öröklődik**.
- A tulajdonságörökítés (inheritance) általánosan értendő: nemcsak a tulajdonság jellegű kapcsolatok öröklődnek, hanem például a birtoklást, valamilyen érzelem irányulását, stb. is beleértjük.
- A tulajdonságörökítést a hálókezelő programnak végeznie.
- A taxonomikus kapcsolat: osztálybatartozás.
- Másik fő kapcsolati forma: tulajdonság hozzárendelés,



Szemantikus háló előnyei

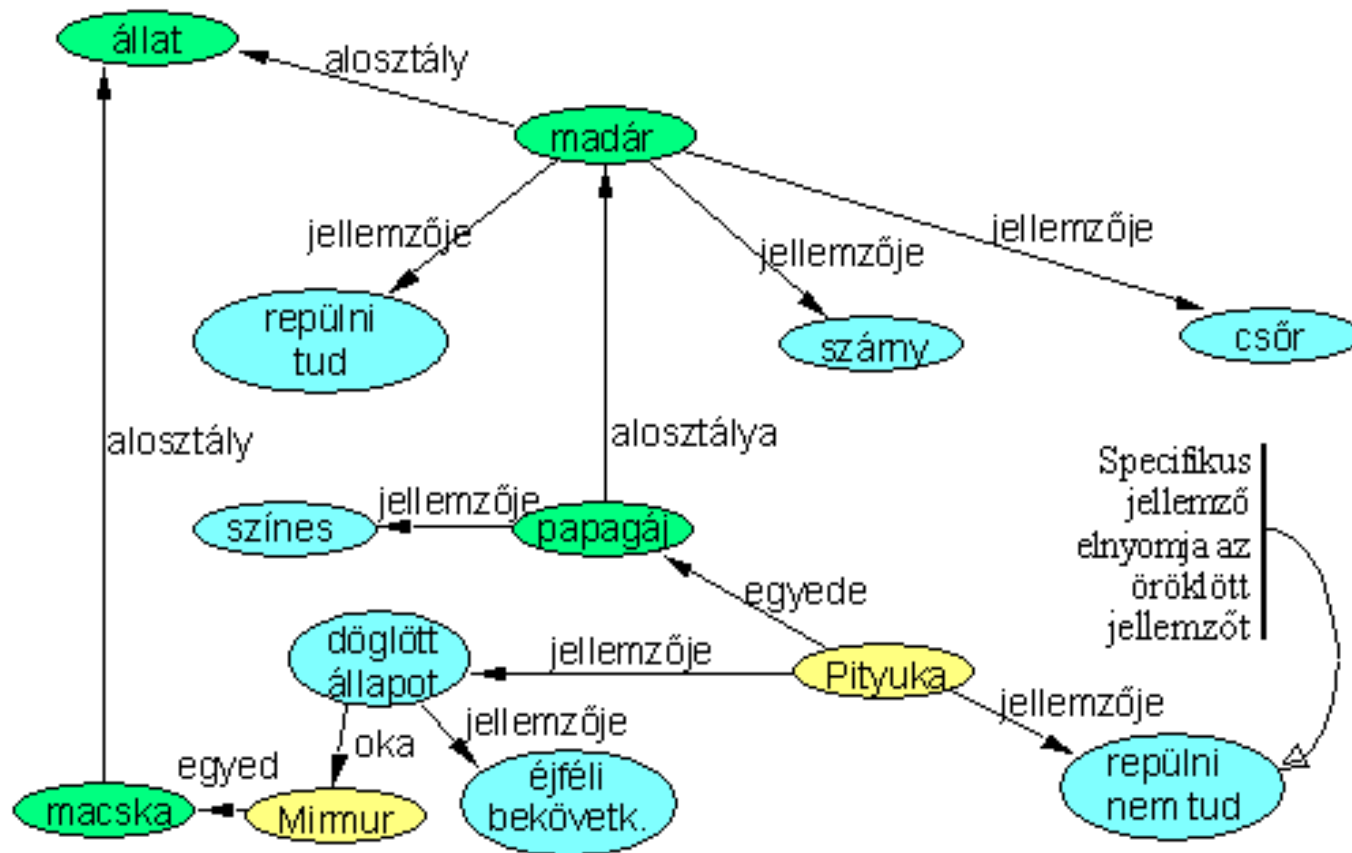
- Az **osztályhierarchia** a tulajdonságok hatékony tárolását is segíti: az osztály minden alosztályára, illetve egyedére egyaránt érvényes tulajdonságok a legmagasabb, legáltalánosabb szinten kerülnek tárolásra, azonban érvényesek a becsatlakozó alsóbb osztályokra és egyedekre is - működik az **öröklődés**. Kiküszöböli a redundanciát, ellentmondás-mentességet eredményez.
- A **grafikus ábrázolás** szemléletes és könnyű érthetőséget jelent és az emberi gondolkodáshoz közel áll.
- **Gyors** számítógépi reprezentációt tesz lehetővé: a csomópontok memóriaterületekre, az élek **mutatókra** képezhetők le. Elmarad a listák elemeinek kimerítő illesztése, mely a szabály- és logika alapú ismeretszemléltetést jellemezte. Az objektumok megtalálása ún. *hash* táblák segítségével gyorsan megtörténhet, a kapcsolatoknak megfelelő mutatók pedig meghatározzák a kapcsolódó ismeretelemek helyét

A jelentés tárolása

- A szemantikus háló **önmagában nem hordozza** a teljes jelentést.
A kapcsolatok értelmezése a szemantikus hálót szemlélő emberre, ill. a hálót kezelő **algoritmusra** hárul.
- Például mondhatjuk azt, hogy a Cápa a Hal osztály egyede, de jelenthetné a kapcsolat azt is, hogy a Cápa a Hal objektum mellett él.
- Az ábra által sugallt értelmezéstől eltérő értelmezés alkalmazása az ember számára **a fogalmakhoz kötődő tudása miatt erőltetettnek tűnhet**, de a számítógép számára csak annyi, az objektumokhoz kötődő ismeret áll rendelkezésre, amennyit a háló kapcsolatrendszeré és az azt kezelő algoritmus megtestesít.
- Előnyös lenne az algoritmus szempontjából, ha **az összes kapcsolatot**, amely egy szemantikus hálóval történő tudásszemléltetésben előfordulhat, az algoritmus elkészítése előtt néhány csoportba sorolhatnánk. Ez lehetővé tenné a szemantikus háló működtető, értelmező

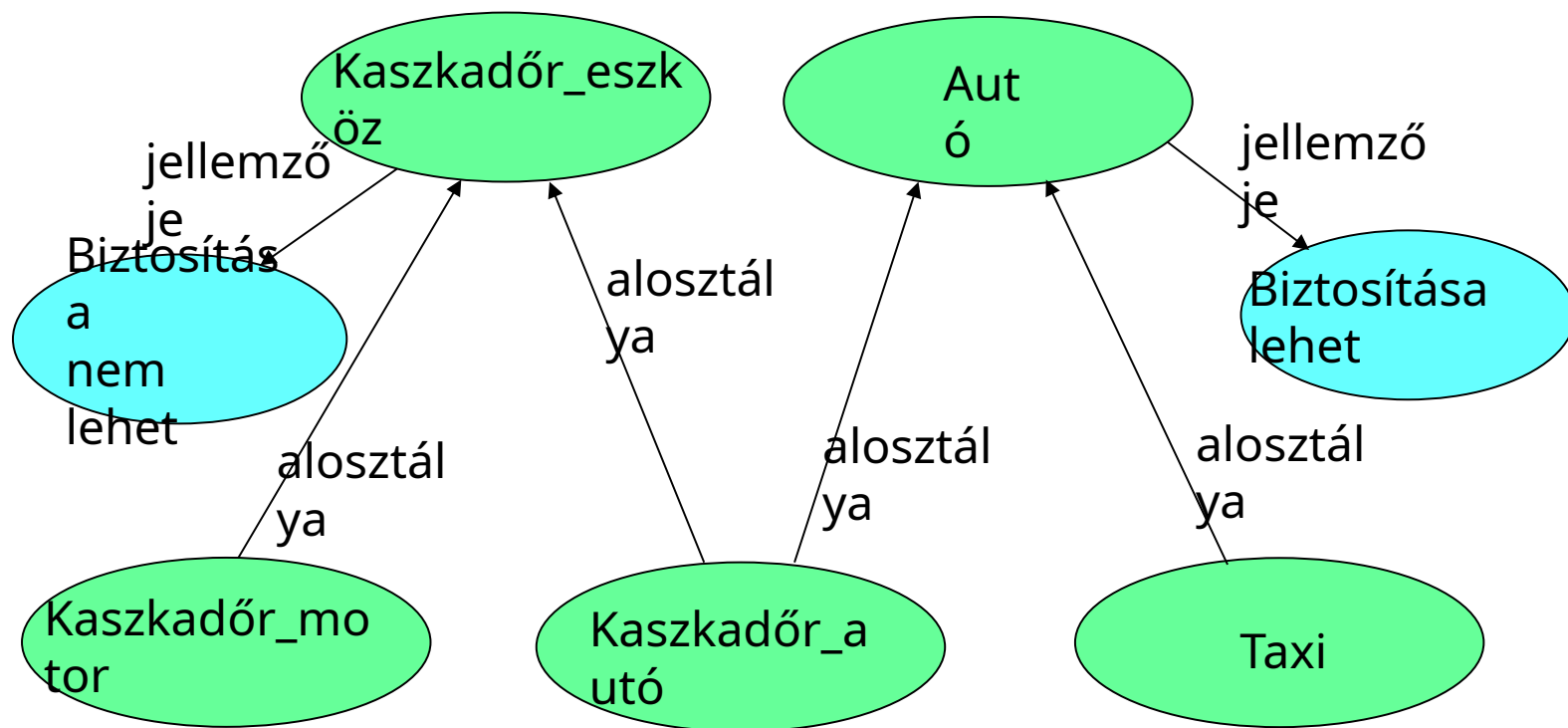
Az öröklés problémái

Kivétel: Pityuka repülni nem tud.



Az öröklés problémái ..

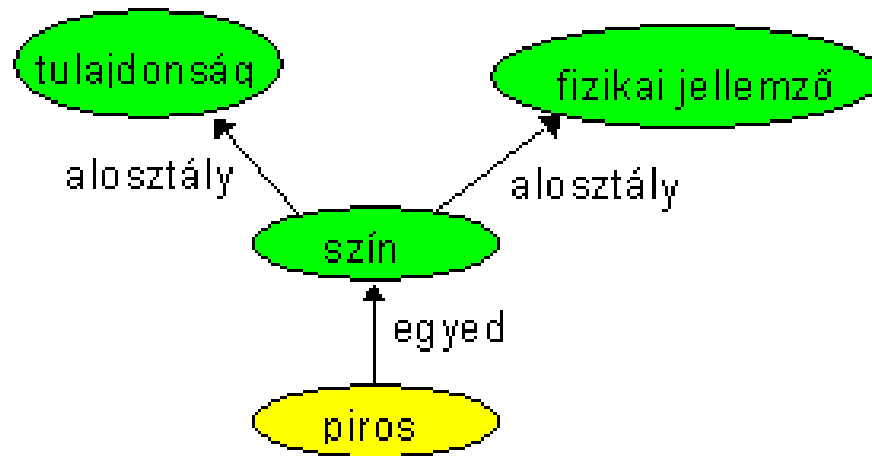
Ellentmondás: Kaszkadőr autóknak lehet, vagy sem biztosítása?



Feloldási módszerek: alapértelmezés; prioritás alkalmazása.

A szemantikus háló egyéb problémái

A típus/egyed megkülönböztetés szükségessége (Type/Token Distinction).



A piros egy szín

A szín egy fizikai jellemző.

A piros egy fizikai jellemző?

A probléma ott érezhető, hogy a piros esetében az örökölt fizikai jellemző mivolta nem tűnik helyesnek. Ennek magyarázata abban van, hogy a fizikai jellemző tulajdonság a színhez, mint osztályhoz (type) kötődik, és

A szemantikus háló egyéb problémái ..

- Másik érzékenységet igénylő probléma az **intencionális és az extencionális** értelmezés eltérése ugyanazon dolognál. Ugyanazon dolog intencionális leírása a lényegret ragadja meg, míg az extencionális megadás a dolog leírása által megadott valós objektumok halmaza. Pl. a *kutya* fogalom extencionális jelentése az összes kutya, intencionális jelentése mindazok a gondolatok, melyek a kutyákhoz kapcsolódnak, azaz a "kutyaság" maga.
- Egyes szerzők a szemantikus háló hibájául rótták fel, hogy a logikai szemléltetéssel ellentétben nem tud olyan fogalmakat kezelni, mint: legalább egy nem specifikált objektum, összes objektum, stb.

Keret alapú tudásszemléltetés

- **A keretek**

A keretek a valós világra vonatkozó ismereteket oly módon reprezentálják, hogy **egyesítik** az objektumokra, tevékenységekre és eseményekre vonatkozó **deklaratív leírást** azon információk előállítására vonatkozó **eljárások megadásával**, melyek célok elérésének, információk megszerzésének módjára vonatkoznak, ily módon túllépve több olyan problémán, mely a szemantikus hálót jellemezte.

- A keret tudásábrázolási forma bevezeti a **prototípus** fogalmát, mely azt a felismerést tükrözi, hogy az emberi ismerettárolás sok **sztereotip leképezést** tartalmaz.
- A valós, vagy elvont objektumokat, fogalmakat reprezentáló keretek **hierarchikus keretrendszerre** kapcsolódnak össze az ismeretábrázolás során.
- A keret alapú ismeretszemléltetés a szemantikus háló továbbfejlesztésének tekinthető.

Neve:	Bútor
Magasság:	< 4m
Magasság:	> 0.2m
.	
.	
Anyag:	fa
Súly:	IF-NEEDED: Térfogat* fajsúly

Neve:	Szék
Szín:	barna
Lábszám:	4
Funkció:	ülőhely

Neve:	Asztal
Funkció:	étkezés
Magasság: default:	< 1.4m 1m
Magasság:	> 0.4m
Lábszám:	4

Neve:	Bárszék
Magasság:	1.2m
Lábszám:	3
Anyag:	alumínium

Egy
keretrendszer
részlete

Alosztály

Alosztály

Alosztály

A szemantikus hálókkal **megegyező** tulajdonságok

- Hierarchikus egyed - alosztály - osztály szerkezet.
- Tulajdonság örökítés, mely kiterjed a procedurális tulajdonságokra is. Konfliktusok feloldása specifikusság, prioritás, vagy alapértelmezés figyelembe vételével.
- Hasonló számítógépes reprezentáció: keretek - memóriahelyek; kapcsolatok - mutatók. Gyors működés.
- Keretkezelő program a következtetés, problémamegoldás kivitelezésére, de jóval gazdagabb feladatkörrel.
- Grafikus ábrázolás használható, de a grafika inkább a keretleíró nyelvek támogatója.
- Rugalmas tudás bővítés, módosítás, törlés.

A szemantikus hálókön túlmutató tulajdonságok

- Egységbefoglalás: objektum, attribútumok, értékek, deklaratív és procedurális összetevők. Slot - filler, attribútum - érték párok, speciális attribútum a keret neve.
- Az attribútumok és attribútum-értékek megadása más keretekre való utalással, többszörös egymásba ágyazással is lehetséges.
- Default, alapértelmezett értékek szolgálják a kérdések megválaszolását.
- A procedurális ismeretszemléltetés részeként értékeket előállító függvények, az értékváltozásokra működésbe lépő mechanizmusok, eseményvezérelt démon rendszer működik.

IF_NEEDED

IF_ADDED

IF_MODIFIED

IF_DELETED

démonok működésének procedurálisaként definiálva áll

- Az attribútumok értékkészletére, értéktartományára, alap (default) értékére adhatunk meg előírásokat.
- A keret tudásábrázolás sokkal elterjedtebb, mint a szemantikus háló, mivel gyakorlatilag annak összes tulajdonságát magába foglalja. Speciális **keretkezelő nyelveket** hoztak létre a keretek használatának megkönnyítésére (FRL, KRL, OWL, NETL, KL-ONE, ART, stb). Ezenkívül több **hibrid**, azaz többféle tudásszemléltetési módszert egyesítő rendszerben is alkalmazásra került (KappaPC, Level5 Object, Nexpert Object/Smart Elements, Aion Development System, CBR Express, stb.).

Példa eseményt leíró keretre

Általános Előadás keret

Megnevezés: előadás

Terem:

Lehetőségek: római számos előadóterem, kb.20db,
arab számozású kistermek, kb. 200 db,
laborok, kb. 40 db.

Kezdési idő: 8:00, 9:00, ... , 18:00.

Időtartam: 40 perc - 180 perc.

Default: 50 perc.

Befejezés időpontja:

Ha szükséges: Kezdési idő + Időtartam.

Eszközök:

Lehetőségek: krétás tábla, filctollas tábla, számítógép, írásvetítő,
diavetítő, projektor, video, TV, film, modell, laboreszközök.

ME Alkalmazott Informatikai Tanszék MI előadás keret

Megnevezés: MI előadás

Terem:

Lehetőségek: I, II, XXX.

Default: I.

Kezdési idő: szerda, kb.14:00.

Időtartam: 160 perc - 175 perc.

Default: 170 perc.

Befejezés időpontja (öröklött függvény):

Ha szükséges: Kezdési idő + Időtartam.

Eszközök:

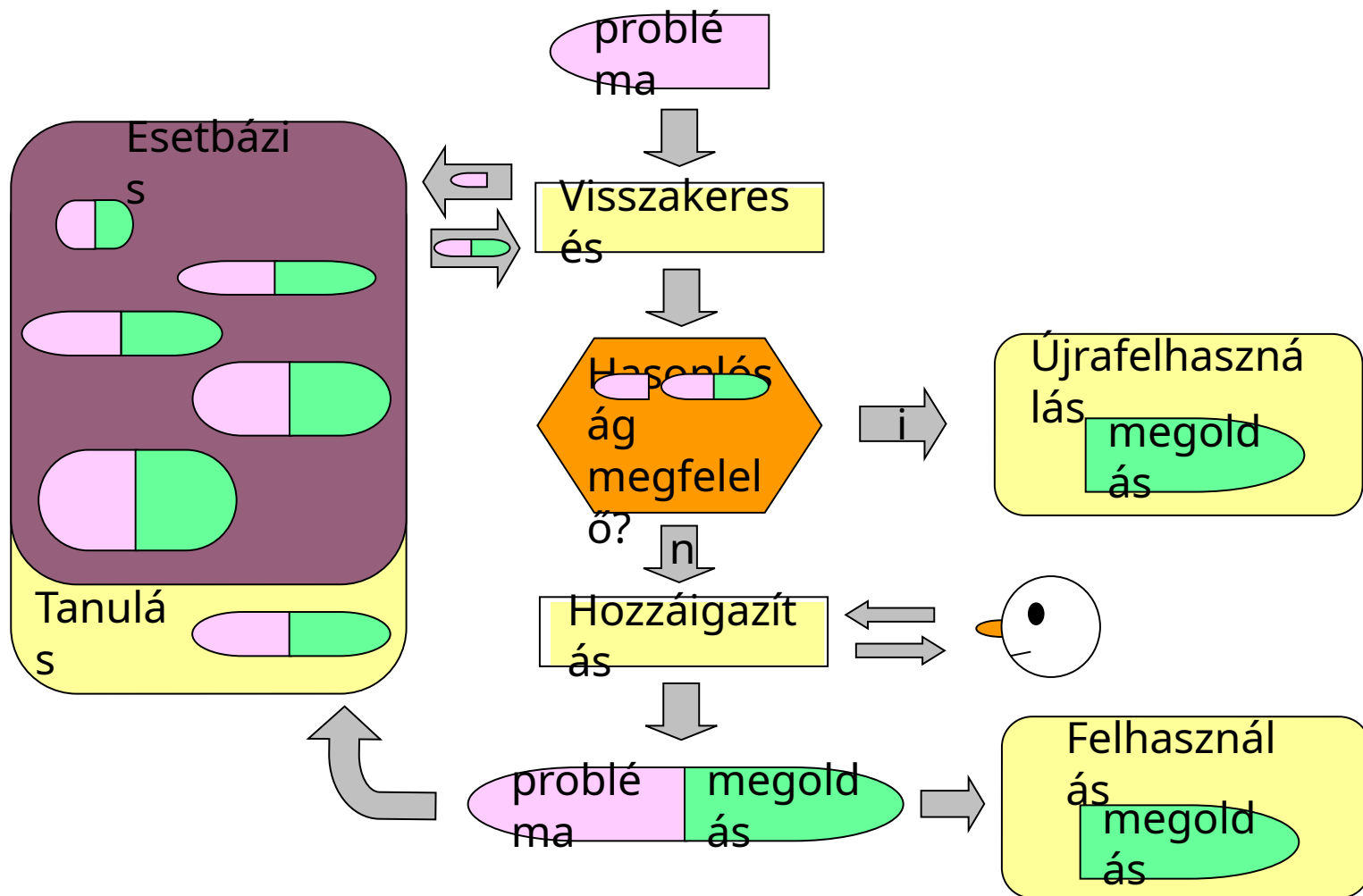
Lehetőségek: krétás tábla, számítógép, írásvetítő, projektor.

Esetalapú rendszerek

- Cél: Régebbi feladatok megoldásakor szerzett tapasztalatok hasznosítása **hasonló** aktuális feladatok megoldásához.
- Egy **eset** összetevői:
 - A **probléma** leírása
 - A probléma **megoldásának** leírása
 - A megoldás jóságának/rosszaságának minősítése.
- Az eset leírása történhet bármilyen ismeretreprezentációs módszerrel, leggyakoribb a keretalapú szemléltetés.
- A **probléma** leírásánál olyan formalizmust kell alkalmazni, amely olyan **metrikát** értelmez, amely révén az esetek problémaleírásai egymással számszerű eredménnyel összehasonlíthatók (Közelség). (10cm - 20cm; piros színű - narancs színű; szép - gyönyörű. Eltérő adattípusokra nem egyformán könnyű metrikát találni.)
- Az eseteket esetbázisban tároljuk.

Az esetalapú következtetés működése

1. **Visszakeresés:** Az esetbázisban megkeressük a megoldandó aktuális problémához legjobban hasonlító, az alkalmazott metrika szerint legközelebbi korábbi problémaleírást.
2. **Újrafelhasználás:** amennyiben a hasonlóság egy megadott nagy értéket elér, a korábbi eset megoldását használjuk fel az aktuális probléma megoldására.
3. **Hozzáigazítás:** Amennyiben a legközelebbi eset hasonlósága nem éri el a kívánt szintet, a rendszer **interaktív** módon hozzáigazítja az eset problémaleírását az aktuális problémához, eközben természetesen az eset megoldás oldalát is módosítva. Az ily módon előállt megoldást használjuk fel az aktuális probléma megoldására.
4. **Tanulás:** A 3. pontban előállt hozzáigazított esetet az esetbázishoz adjuk, a megoldás jóságának/rosszaságának minősítésével együtt.



Az esetalapú következtetés tulajdonságai

Előnyök:

- A probléma modelljének előzetes kidolgozása nélkül is alkalmazható
- Használat közben fejlődik, könnyen bővíthető
- Robusztus: hiányos, vagy rosszul definiált fogalmakkal is megadhatóak esetek
- Nem algoritmizálható problémák esetén is alkalmazható
- Képes támogatni a korábbi hibás megoldások elkerülését is.

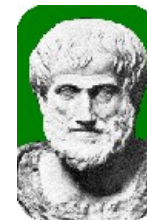
Hátrányok:

- Emberi interakciót igényel az esetek többségében
- Minősége romolhat az eltérő felhasználók eltérő igény szintje miatt a tanulás során.

Mesterséges intelligencia alapjai

Formális logika

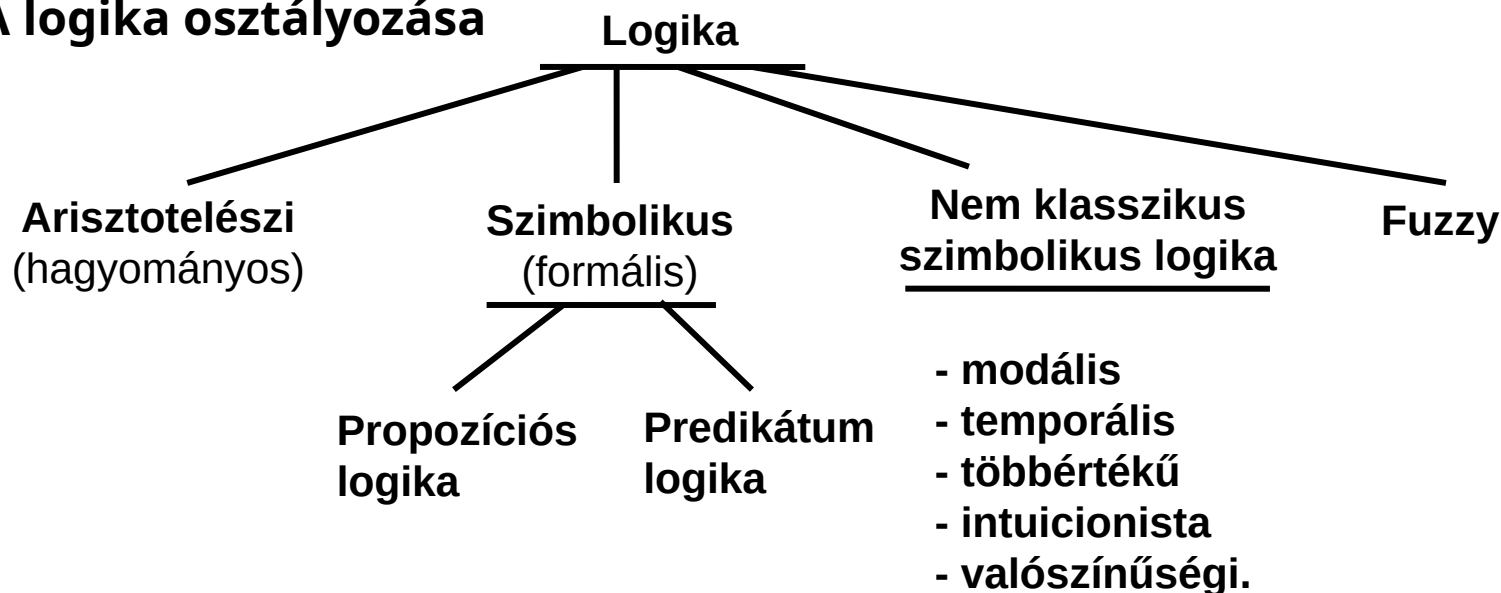
Tudásszemléltetés formális logikával



- **A logikáról**

A logika a bölcselés tudománya, a helyes gondolkodás művészetének tana.

- **A logika osztályozása**



A szimbolikus logika kifejlődése

- **Petrus Ramus (1515-1572)**
 - Bírálta az arisztotelészi logikát.
 - "A logika: a helyes érvelés eszköze."
- **Francis Bacon (1561-1626)**
 - "A logika az új ismeretek megszerzésének eszköze."
 - Kísérletek, mérések fontossága.
- **René Descartes (1596-1650)**
 - *"Értekezés a módszerről"* c. műve
 - A problémákat osszuk független, vagy kevésbé összefüggő részekre
 - Az egyszerűtől haladjunk a bonyolultabb felé
 - Csak olyan dolgokat fogadjunk el igaznak, melyek igazsága megkérdőjelezhetetlen
 - Törekedjünk a teljes felsorolásokra.

- **Gottfried Wilhelm Leibniz (1646-1716)**

- A szimbolikus logika előfutára
- "Logika $\cong$ matematika.,,
 - *a logika a matematikához hasonló rutinszerűen alkalmazható műveletekkel lenne kezelhető.*
- **Azonosság** definíciója:
"Azok a terminusok tekintendők azonosnak, vagy egybeesőnek, amelyek egymással tetszés szerint felcserélhetők anélkül, hogy ezáltal bármelyik állítás igazsága megváltozna."

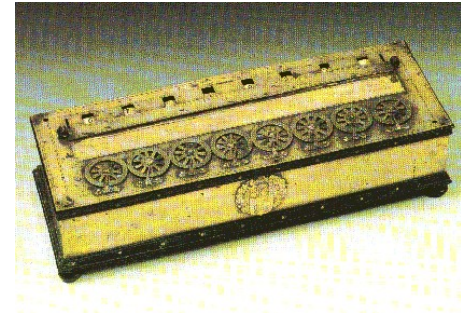
- **George Boole (1815-1864)**

- A szimbolikus logika megalapítója.
- *"A logika matematikai analízise"* című műve.
- Matematikai és más kijelentések formailag hasonlóan kezelhetők tartalmuktól függetlenül
- A **logikai szorzás** és **logikai összeadás** műveletének bevezetése.

- **John Venn (1834-1923)**

- *"Symbolic Logic"* című műve
- Diagramok: teljes esemény = téglalap; körök: mindenféle egymásbametszés

- **William S. Jevons** (1835-1882)
 - "megengedő vagy"
 - Első logikai gép (a logikai zongora)



- **Charles S. Peirce** (1839-1914)
 - Boole összes logikai művelete kiváltható *egyetlen művelettel*, a 'sem...., sem...', NOR művelettel.
- **Augustus de Morgan** (1806-1871)
 - Reláció-azonosságok.

- **Gottlob Frege (1848-1925)**

- Integrálta a korábbi eredményeket
- **Bevezette a logikai függvény fogalmát**
- **"A szimbolikus logika atyja"**
- *"Fogalomírás, a tiszta gondolkodás aritmetika mintájára megalkotott formanyelve" c. műve*

- **Bertrand Russell (1872-1970)**

- Frege-től függetlenül ugyanoda eljutott
- Hézagok az elődök munkáiban: logikai **paradoxonok**.
- *"Az osztályok osztálya eleme-e önmagának?"*
- **Típuselmélete:** magyarázat a paradoxonokra.

Új irányzatok a logikában

- A **modális** logikákkal olyan terminusok is kifejezhetők, mint a "lehetséges", "szükségszerű" és ezek változatai. A modális logika a klasszikus logika kiterjesztése.
- A **deontikus** logika olyan etikai és jogi kifejezések elemzésével foglalkozik, mint a "Megtiltott, hogy...", "Megengedett, hogy...", stb.
- **Jan Lukasiewicz** (1878-1956)
 - **Háromértékű logika: 0, 1/2, 1**, ahol az 1/2 érték a bizonytalanságot fejezi ki.
 - Eljutott a *végtelenértékű logikáig*.
- **E.L.Post** (1897-1954)
 - **Többértékű** logika, Lukasiewicz-csel egyidőben.
- **J.M.Keynes** (1883-1946)
 - Ch.S.Peirce-szel együtt a **valószínűségi** logika megalkotójának tartják.

Propozíciós logika

- Egyéb elnevezések: kijelentéskalkulus, ítéletkalkulus, nulladrendű predikátum-kalkulus.
- *Propozíció*: egy kijelentő mondat formában megadott állítás, mely az adott kontextusban egyértelmű *igaz*, vagy *hamis* logikai értékkel bír.
- A propozíciós logika olyan egyszerű nyelvtani kapcsolatokat alkalmaz a legegyszerűbb "atomi mondatok" között, mint az ***és, vagy, nem***.
- *Atomi mondatok* :
A= " A víz száz fokon forr."
B= " A macskák ugatnak."

Az ilyen atomi összetevők (objektumok) **egyértelműen megítélhetők** az igaz, vagy hamis értékek valamelyikével.
(Általános tapasztalat szerint A igaz, B hamis).

- ***A propozíciós logika tárgya, célja*** összetett szerkezetek kiértékelésére formális szabályt adni. A kiértékelésben az atomi összetevők (objektumok) igazságértékeit kell csak figyelembe venni, jelentésüktől eltekintünk.
- Atomi kijelentések helyett szimbólumok:

Esik az eső. $\Rightarrow A$

- Eltekintünk a természetes nyelvi jelentéstől
Nem természetes átalakítás, pl.:
A: Jóska megbetegedett.
B: Jóska elment az orvoshoz.

Hétköznapi értelemben **A és B** jelentése eltér **B és A** összetett kijelentésétől.

Csak az igaz, vagy hamis értéke fontos számunkra A-nak és B-nek.

A propozíciós logika szintaxisa

- **Jelkészlet**

- **Elválasztójelek:** $, () []$

- **Logikai operátorok** (műveleti jelek): {csökkenő precedenciasorrendben}

$\neg$	$\wedge$	$\vee$	$\rightarrow$	$\equiv$
negáció	konjunkció	diszjunkció	implikáció	
ekvivalencia				
'nem'	'és'	'vagy'	'ha ... akkor ...'	'akkor és csak akkor'
		'következik'	'azonos'	

- **Logikai változók** (ítéletváltozók): $p, q, r, \dots$

- **Logikai konstansok** (ítélet-, predikátumkonstansok) : T, F (*True, False*), (*igaz, hamis*).

- A propozíciós logika (kijelentéskalkulus, ítéletkalkulus) **formuláit** a jelkészlet elemeiből a **szintaxis szabályai** szerint építjük fel:

- *Atomi formula* (atom)

- Minden ítéletkonstans atom : **T, F**
- Minden ítéletváltozó atom : **p, q, r, ...**

- *Formula* (jól formált formula):

- Minden atomi formula egyben formula
- Ha **A** és **B** formulák, akkor a

$$\neg \mathbf{A}, \mathbf{A} \wedge \mathbf{B}, \mathbf{A} \vee \mathbf{B}, \mathbf{A} \rightarrow \mathbf{B}, \mathbf{A} \equiv \mathbf{B}$$

kifejezések is formulák.

A propozíciós logika szemantikája

- Egy logikai formulának az igazságértéke ad **jelentést**, mely igazságértéket a *szemantika szabályai* szerint kapja meg.

- **A jelentésadás lépései:**

1. Interpretálás

2. Kiértékelés

1. Interpretálás

A formula *interpretálása*: minden egyes ítéletváltozójához az *igaz*, vagy a *hamis* értéket rendeljük *minden lehetséges módon*. Egy hozzárendelés egy interpretációt ad.

Az interpretációk száma a propozíciós logikában véges: 2^n ahol n a változók száma.

pl.: $A := (p \wedge q) \rightarrow (r \equiv (\neg s))$

két lehetséges interpretációja:

$I_1: (p, q, r, s) \leftarrow (T, T, F, F)$

$I_2: (p, q, r, s) \leftarrow (F, T, T, F)$

2. Kiértékelés

Az interpretált formula kiértékelését a logikai **műveleti jelek szemantikája alapján** végezzük:

A	B	$\neg A$	$A \wedge B$	$A \vee B$	$A \rightarrow B$	$A \leftrightarrow B$
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

Pl.: Ha süt a nap, nem tanul Péter.

süt a nap : p

tanul Péter : q

$A := p \rightarrow (\neg q)$

A lehetséges 4 interpretáció egyike: $p = T$, $q = F$,

a kiértékelte formula: $T \rightarrow (\neg F)$; $T \rightarrow T$; T , az A állítás igaz.

Kielégíthetetlen és érvényes kifejezések

- A kielégíthetetlen logikai kifejezések és az érvényes kifejezések fontos szerepet játszanak a tételbizonyításban. A tétel azt jelenti, hogy állításokból logikailag következik a konklúzió.

• **Tautologikus törvény:** $A \vee B = A$, ha A tautológia,
 $A \wedge B = B$, ha A tautológia.

A logikai kifejezés tautológia, azaz **érvényes** kifejezés akkor, ha **minden lehetséges helyettesítése igaz**.

• **Kielégíthetlenségi törvény:** $A \vee B = B$, ha A kielégíthetetlen,
 $A \wedge B = A$, ha A kielégíthetetlen.

A logikai kifejezés **kontradikció**, azaz kielégíthetetlen kifejezés akkor, ha minden lehetséges helyettesítésre hamis értéket ad.

Természetesen a formulák többsége egyes interpretációkban hamis, más interpretációkban igaz, azaz kielégíthető.

Tételbizonyítás a propozíciós logikában

- Egy propozíciós logikai formula **érvényességének igazolására** alkalmas módszerek:

- Igazságtáblás
- Formális levezetés
- [Quine algoritmus](#)
- [Wang algoritmus](#)
- Rezolúció.

- A rezolúció szabálya:
$$\frac{\alpha \vee \beta, \neg \beta \vee \gamma}{\alpha \vee \gamma}$$

Mivel β nem lehet egyszerre igaz és hamis, ezért valamelyik premisszában, előfeltételben a másik tagnak (α , vagy γ) igaznak kell lennie, tehát az $\alpha \vee \gamma$ igaz.

- A rezolúció módszere kiemelkedik a többi közül, mivel ez a bizonyítási módszer alkalmazható a predikátum logikában is.

- A **tétel** igaz állításokból és egy, azokból állítólag következő konklúzióból áll.
 - Jelölése: $\frac{\alpha, \beta, \gamma}{\delta}$, ahol α, β, γ tények, és δ konklúzió
- A **tételbizonyítás** feladata formális szabályok gépies alkalmazásán keresztül igazolni, hogy a konklúzió a premisszák következménye.
- A rezolúciós módszer **lépései**:
 - A bizonyítandó tétel **tagadott alakját** vesszük alapul azáltal, hogy feltesszük, hogy a konklúzió ellentettje igaz a premisszák igaz értékei mellett. Ha az ily módon kapott összetett formula kielégíthetetlen voltát sikerül belátni, akkor az eredeti tétel igaz.
 - Ehhez konjunktív normál forma alakra (KNF) hozzuk az összetett formulát és a **rezolúció ismételt alkalmazásával**, fokozatos egyszerűsítéseken keresztül **jutunk el az ellentmondáshoz**.
- A rezolúció alkalmazása a propozíciós logikában (ítéletkalkulus) korlátozott, mivel bonyolultabb problémák leírására ez a logika nem ad eszközöket.

- A konjunktív normálforma

- Klózok konjunkciója, ÉS kapcsolata
- Az eredeti formulával ekvivalens
- Egyszerű felépítése miatt gépiesen kezelhető.

- A klóz literálok diszjunkciója (VAGY kapcsolata), vagy egyetlen literál.
- A **literál** egy ítéletváltozó (logikai változó), vagy annak negáltja.

Példa:

konjunktív normál forma

$$(p \vee q) \wedge (\neg p \vee r) \wedge (\neg r \vee s) \wedge q \wedge s$$

The diagram illustrates the Conjunctive Normal Form (CNF) of a logical formula. The formula is shown as a conjunction of five clauses, each enclosed in parentheses and separated by an AND symbol ($\wedge$). The clauses are: $(p \vee q)$, $(\neg p \vee r)$, $(\neg r \vee s)$, q , and s . Above the formula, a large curly brace spans the entire expression, labeled "konjunktív normál forma". Above each clause, a smaller curly brace indicates that each clause is a disjunction of literals. The literals are: p and q in the first clause; $\neg p$ and r in the second; $\neg r$ and s in the third; q in the fourth; and s in the fifth. The literals p , q , r , and s are highlighted in orange, while the negation symbols ($\neg$) are in purple.

- Formulák konjunktív normálformára hozásának lépései:

$$1. \text{ Azonosságok kiküszöbölése: } A \equiv B = (A \rightarrow B) \wedge (B \rightarrow A)$$

$$2. \text{ Implikációk kiküszöbölése: } A \rightarrow B = \neg A \vee B$$

3. A negálás hatáskörének redukálása:

$$\neg (A \vee B) = \neg A \wedge \neg B$$

$$\neg (A \wedge B) = \neg A \vee \neg B$$

4. Klózek konjunkciójának létrehozása:

$$A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C).$$

- A diszjunktív normál formából elhagyva a konjunkció operátorokat, a formula klózekra esik szét, melyek halmazára alkalmazzuk a rezolúció szabályát ciklusban. Ily módon a rezolúció egy olyan könnyen automatizálható eljárás, melynek segítségével egy klózhalmoz, illetve a neki megfelelő konjunktív normál forma kielégíthetlenségét belátjuk.
- A klózhalmoz egyszerűsítése rezolválható klózpárokon keresztül történik.
- Rezolválható klózpár: egyetlen ellentett literálpárt

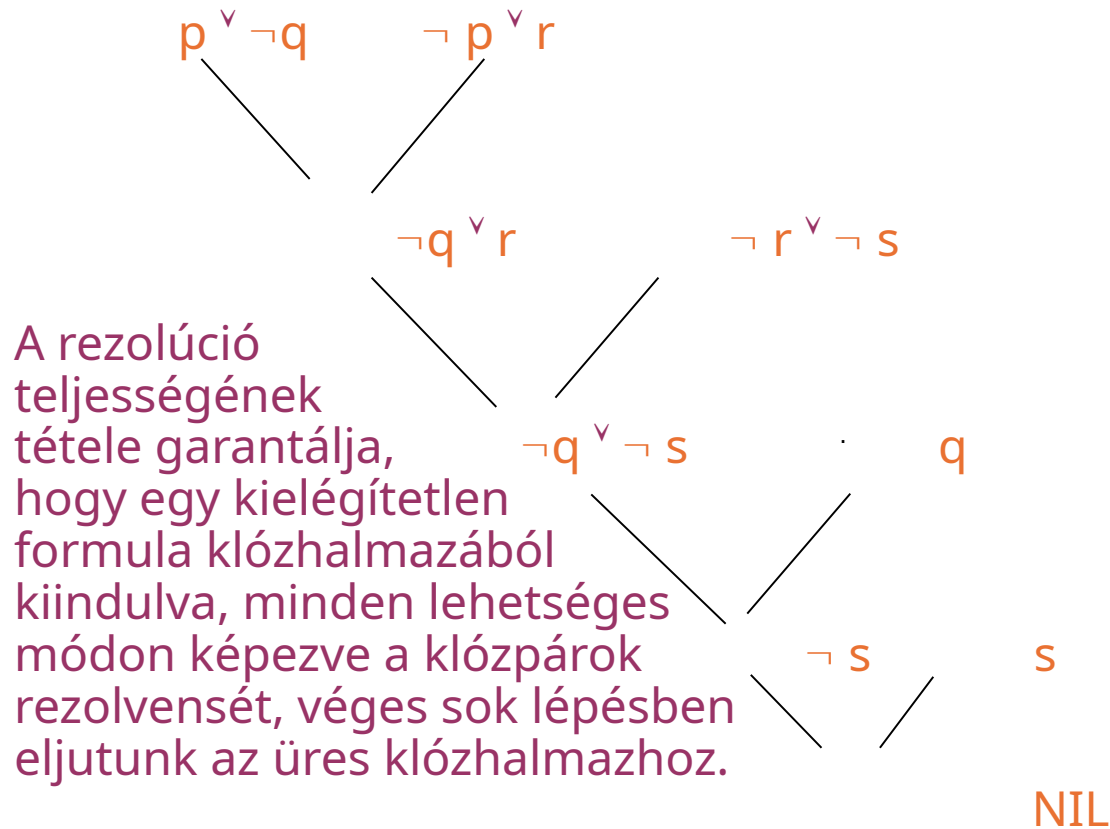
- A rezolúciós folyamat során a rezolválható párok rezolválása eredményeként adódó rezolvens klózt a halmazhoz adjuk.
- A klózhalmaz kielégíthetetlenségét az jelzi, hogy végül üres klózt kapunk.
- A rezolúció algoritmus:

Procedure Rezolúció

1. KLÓZHALMAZ feltöltése
 2. do
 3. Rezolválható klózpár C_i, C_j kiválasztása a KLÓZHALMAZ-ból
 4. Rezolválás: $C_{ij} \leftarrow R(C_i, C_j)$
 5. KLÓZHALMAZ bővítése C_{ij} rezolvenssel
 6. while $C_{ij} \neq \text{NIL}$
- end

Példa:

KNF: $(p \vee \neg q) \wedge (\neg p \vee r) \wedge (\neg r \vee \neg s) \wedge q \wedge s$



Predikátum logika

További elnevezések: elsőrendű predikátumkalkulus, vagy egyszerűen predikátumkalkulus.

- **Predikátum:** objektumok sajátságainak és az objektumok közötti kapcsolatok, viszonyok, relációk szimbolikus formában való megadására alkalmas állítás, mely egy adott interpretációban egyértelmű igaz, vagy hamis minősítéssel bír.
- **A predikátum logika jellemzői:**
 - MI feladatok reprezentálására alkalmas (ld.: PROLOG).
 - *Kijelentéseink **tartalmát** is leírja*, mivel az állítások az individuumkonstansok, individuumváltozók és individuumfüggvények révén egy tartomány (, alaphalmaz, domain) elemeire vonatkozhatnak.
 - Az ítélet-, vagy logikai függvények, azaz a **predikátumok** használatával individuum paraméterektől függő igazságú állítások fogalmazhatók meg.
 - Új logikai operátorok, a **kvantorok** segítségével kifejezhetjük a "minden" és a "létezik" fordulatokat is.
 - Az *interpretáció módja eltér* a propozíciós logikáétól, és ebből eredően a predikátum logikában egy formulának **végtelen sok**

Bonyolultabb tételek bizonyíthatók, mint a propozíciós logikában:

Pl.:

Premisszák, ismert tények:

- (1) Van olyan hallgató, aki minden tárgyat szeret.
 - (2) A szócséplést egyik hallgató sem szereti.
-

Igaz-e a fenti állítások ismeretében a következő állítás?

Bizonyítandó állítás:

- (4) Egyik tárgy sem szócséplés.



A predikátum logika szintaxisa

- **Jelkészlet:**

- **Elválasztójelek:** $, () []$

- **Logikai operátorok** (műveleti jelek): $\neg \wedge \vee \rightarrow \equiv$

- **Kvantorok:** univerzális kvantor ("minden"): $\forall$

egzisztenciális kvantor ("létezik" vagy "van olyan"): $\exists$.

(precedencia: $\forall \exists \neg \wedge \vee \rightarrow \equiv$)

- **Elemkonstansok** (individuumkonstansok): $a, b, c, \dots$, vagy nevek (például: *János, asztal*).

- **Elemváltozók** (individuum változók): $x, y, z, \dots$

- **Függvényszimbólumok** (individuumfüggvények): $f, g, h, \dots$, vagy például: $testvére(Józsi) \Rightarrow Laci$

- **Itéletkonstansok** (logikai konstansok): *True, False*.

- **Itéletváltozók** (logikai változók): $p, q, r, \dots$

- **Predikátumszimbólumok** (logikai függvények): $P, Q, R, \dots$, vagy például: $testvérek(x,y), P(x,y)$.



- **A jelkészlet elemeiből a *kifejezések* 3 osztálya alkotható meg:**

Term-ek, Atomi formulák (Atomok) és Jólformált formulák (Formulák).

- **Term** (individuum konstans, individuum változó, individuum függvény):

- Minden elemkonstans (individuum konstans) term. Pl.: $a, b, \dots, asztal, szék$, stb.

- Minden elemváltozó (individuum változó) term. Pl.: x, y .

- Ha f egy n -argumentumú függvénytípus és $t_1, \dots, t_n$ term-ek, akkor $f(t_1, \dots, t_n)$ is term.

Pl.:

$f(a, x)$,

$gyereke(Pál, x)$, ($x=Bori$ esetén az eredmény Józsi, $x=Julcsi$ esetén

az eredmény Béla)

$színe(haj, időskorban) \Rightarrow \text{ősz}$,

$színe(haj, fiatal korban) \Rightarrow \text{szőke}$

- **Atomi formula** (kiértékelve *True*, vagy *False*):

- Minden ítéletkonstans atomi formula. (*True*, *False*)
- Minden ítéletváltozó atomi formula. (*p*, *q*, *r*,...)
- Ha *P* egy *n*-argumentumú predikátumszimbólum (logikai függvény) és $t_1, \dots, t_n$ term-ek, akkor $P(t_1, \dots, t_n)$ atomi formula.

Pl.:

$\text{házaspár}(\text{Pál}, \text{Irén}) \Rightarrow \text{True}$

$\text{gyereke}(\text{Pál}, x)$ ($x=\text{Józsi}$ esetén az eredmény igaz, $x=\text{Bori}$ esetén az eredmény hamis)

$\text{színe}(\text{szőke}, \text{haj}, \text{időskorban}) \Rightarrow \text{True}$

- **Formula** (jólformált formula):
 - Minden atomi formula egyben formula. ($True$, p , $P(a,b)$)
 - Ha A és B formulák, akkor a
 $(\neg A)$, $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$, $(A \equiv B)$ kifejezések is formulák.
 - Ha az A egy formula és az x egy elem (individuumváltozó),
 akkor a

$(\forall x) A$, $(\exists x) A$

kifejezések is formulák. Pl.: $(\forall x)$ testvére $(x, Pál)$

Precedenciasorrend: $\forall \exists \neg \wedge \vee \rightarrow \equiv$, pl.: $((\exists x)A) \vee (B)$ helyett: $(\exists x)A \vee B$

Példa: Vagy van egy részeg, vagy nem volt már ital.

$A := \text{részeg}(x) = R(x)$,

$C = \text{volt ital}$

$(\exists x)R(x) \vee \neg C$.

- **Kötött változó:**

Minden előfordulása a formulában kvantor hatása alatt áll.

$(\forall x) P(x,y)$ x kötött változó, y nem kötött változó.

$(\forall x) [P(x,y) \wedge (\exists y) Q(x,y)]$

mindkét x kötött $[]$ miatt,

y első előfordulása nem kötött,
 y második előfordulása kötött.

- **Mondat:** olyan formula, melyben minden változó összes előfordulása kötött. Csak mondatokkal foglalkozunk.

Példa mondatra: $(\forall x) [P(x) \vee (\exists y) Q(x,y)]$

Példa a formulázásra

Feladat:

- Állítások: (1) Van olyan hallgató, aki minden tárgyat szeret.
(2) A szócséplést egyik hallgató sem szereti.
-

(3) Egyik tárgy sem szócséplés

Formalizálás:

Alkalmazott predikátumok:

$H(x)$: az x egy hallgató

$T(y)$: az y egy tárgy

$C(y)$: az y egy szócséplés

$S(x,y)$: az x szereti y -t.

A formalizált állítások:

F1: $(\exists x)[H(x) \wedge (\forall y)(T(y) \rightarrow S(x,y))]$

F2: $(\forall x)[H(x) \rightarrow (\forall y)(C(y) \rightarrow \neg S(x,y))]$

F3: $(\forall x)[T(x) \rightarrow \neg C(x)]$

A predikátumkalkulus szemantikája

- Az elsőrendű predikátumkalkulus formuláinak is hasonlóan adunk **jelentést**, mint az ítélet-kalkulusban:
 - először **interpretáljuk**, majd
 - **kiértékeljük** a formulát. (Eredmény: True, vagy False).
- **Interpretáció:**
 - **Individuumtartomány** (az értelmezés alaphalmazának) **megválasztása**: jele D , nem üres.
 - **Hozzárendelések**:
 - Minden *individuumkonstans-szimbólumnak* feleltessük meg D egy elemét. ($a := \text{asztal}$)
 - Minden n -argumentumú *individuumfüggvényhez* (pl.: $f(a,b)$) rendeljünk hozzá egy $D^n \Rightarrow D$ leképezést. (Pl.: $\text{gyermeke}(a,b) \Rightarrow c$, $\text{gyermeke}(\text{Pál}, \text{Irén}) \Rightarrow \text{Jutka}$).
 - Minden n -argumentumú *predikátumfüggvénynek* feleltessünk meg egy $D^n \Rightarrow \{T, F\}$ leképezést, ahol T a logikai igaz, F pedig a logikai hamis értéket jelöli. (Pl.: $\text{házaspár}(\text{Pál}, \text{Irén}) \Rightarrow \text{True}$).

Predikátum-logikai formulák kielégíthetősége

- **Kielégíthető** az a formula, amelyik valamely interpretációban igaz.
- **Érvényesnek** nevezünk egy formulát, ha minden interpretációban igaz.

Például a $(\forall x)P(x) \vee (\exists y) \neg P(y)$.

- **Kielégíthetetlen** az a formula, amely minden interpretációban hamis értékű.

Például a $(\forall x)P(x) \wedge (\exists y) \neg P(y)$.

- **Az érvényes, illetve a kielégíthetetlen formulák a fontosak** számunkra, ugyanis a tételbizonyítást egy formula érvényességének, illetve kielégíthetetlenségének a belátására lehet visszavezetni.

- **Probléma:** az elsőrendű predikátum kalkulus egy formuláját *nem tudjuk az összes lehetséges interpretációban kiértékelni*, mivel végtelen sok alaphalmazt lehet megadni.

- **Megoldás:** a rezolúciós eljárás mellyel egy formula

Rezolúció a predikátum logikában

Bonyolultabb tételek bizonyíthatók, mint a propozíciós logikában, ugyanakkor a bizonyítás lépései is jóval összetettebbek.

A következőkben egy egyszerű példát mutatunk, amely rámutat a predikátum logikán belüli rezolúció egyik erősségére: A bizonyítás folyamán individuumváltozó - individuumkonstans hozzárendeléseket is végez, amely révén arra is választ kapunk, hogy milyen feltételek mellett igaz a tétel.

Igazi erejét megtapasztalhatjuk a Prolog programozási nyelvben.

Pl.: Feladat: Értik-e a hallgatók a rezolúciót?

Premissák, ismert tények:

- (1) A hallgatók értelmesek.
- (2) Az értelmesek mindent értenek, ami egyszerű.
- (3) Az értelmeseknek a rezolúció egyszerű.

Igaz-e a fenti állítások ismeretében a következő állítás?

Bizonyítandó állítás:

- (4) A hallgatók értik a rezolúciót.

• Formulázás:

- Alkalmazott predikátumok:

hallgató(x) - az x az egy hallgató

értelmes(x) - az x értelmes

érti(x,y) - az x érti az y-t

egyszerű(y) - az y egyszerű

- A premissák és a negált következmény formalizált alakban:

(1) $\neg \text{hallgató}(x) \vee \text{értelmes}(x)$

(2) $\neg \text{értelmes}(x) \vee \text{érti}(x,y) \vee \neg \text{egyszerű}(y)$

(3) $\neg \text{értelmes}(x) \vee \text{egyszerű}(\text{Rezolúció})$

(4) $\text{hallgató}(x) \wedge \neg \text{érti}(x, \text{Rezolúció}) \Leftarrow \text{tagadása} \Leftarrow$

$\neg \text{hallgató}(x) \vee \text{érti}(x, \text{Rezolúció})$

A klózalmaz:

$\neg \text{hallgató}(x) \vee \text{értelmes}(x)$ $\neg \text{értelmes}(x) \vee \text{érti}(x,y) \vee \neg \text{egyszerű}(y)$
 $\neg \text{értelmes}(x) \vee \text{egyszerű}(\text{Rezolúció})$ $\text{hallgató}(x) \quad \neg \text{érti}(x,\text{Rezolúció})$

A rezolúciós folyamat lépései:

$\neg \text{hallgató}(x) \vee \text{értelmes}(x)$ $\text{hallgató}(x)$
 $\text{értelmes}(x)$ $\neg \text{értelmes}(x) \vee \text{érti}(x,y) \vee \neg \text{egyszerű}(y)$
 $\neg \text{érti}(x,\text{Rezolúció})$ $\text{érti}(x,y) \vee \neg \text{egyszerű}(y)$
 $y \Leftarrow \text{Rezolúció}$
 $\neg \text{egyszerű}(\text{Rezolúció})$ $\neg \text{értelmes}(x) \vee \text{egyszerű}(\text{Rezolúció})$
 $\neg \text{hallgató}(x) \vee \text{értelmes}(x)$ $\neg \text{értelmes}(x)$
 $\neg \text{hallgató}(x)$ $\text{hallgató}(x)$
 NIL

A PROLOG

- **PROLOG** (Programming in Logic, Logika alapú programozás)
Alain Colmerauer, Bob Kowalski, 1970.
- **Predikátum konstanson alapuló** általános célú programnyelv, amely azonban legelőnyösebben logikai problémák megoldására alkalmazható.
- A PROLOG létrehozására kihatott az a tapasztalat, amelyet a korábbi MI programozási nyelvekkel szereztek: minél több a rendszerbe épített lehetőség, a rendszer annál inkább lelassul.
- **Deklaratív** programnyelv: elegendő megadnunk csak a problématerületre vonatkozó tényeket, szabályokat és a megválaszolandó kérdést ill. minősítendő állítást és a

Bevezetés a PROLOG programozási nyelvbe

- Jellemzői:

- Az eljárások és a tények a **mintaillesztés** során kerülnek elő
- Betartja a bemenőállítások **sorrendjét**.
- A keresést automatikus **visszalépéssel** (back track) hajtja végre, amelyet azonban a programozó a ! (cut, **vágás**) operátorral módosíthat. Ez a lehetőség **ellentmond a logikai programozás ideális koncepciójának**: a programozó feladata csak a logikai probléma megadása, a problémaleírás algoritmikus kiértékelését pedig a gépnek kell elvégeznie.

Kowalski képlete:

algoritmus = logika + vezérlés

Jelentése a következő: egy algoritmus mindig két komponenst tartalmaz implicit formában:

- a logikai összetevőt, amely megadja a kérdéses problémára vonatkozó tudást,
- a vezérlési összetevőt, mely megtestesíti a megoldási stratégiát az adott probléma esetére.

- Bevezetésre került a **symbol** típus (domain), hogy a predikátumlogika individuumkonstansai, -változói és -függvényei a valóság egy részletét leképezhessék.
- Fontos adatszerkezet a **lista**. A listafeldolgozás a **mintaillesztésen** és a **rekurzió**n alapul.
- A függvények a predikátumfüggvényeken alapulnak, és **nincs különbség a bemenő és a kimenő változók között**.
- A bemeneti állítások (**szabályok**, clauses) formátuma korlátozott:

$$\frac{}{fej} \quad \frac{P \text{ if } Q \text{ and } R \text{ and } S.}{törzs}$$

A fej egyetlen predikátum, a törzs pedig atomi formulák (többnyire predikátumok, azaz logikai függvények) negációt nem tartalmazó konjunkciója (csak *and* lehet).

- Mivel a törzs atomi formulák konjunkciója, azaz Horn-halmaz, így azt mondhatjuk, hogy a PROLOG **a Horn halmazokra a rezolúció elvét alkalmazza.**
- **A felhasználóra hagyja a rezolúciós stratégia megvalósítását:** néhány megszorítást tett a bemenőállításokra, és az illesztés a bemenőállítások (clauses, tények és szabályok) megadott sorrendjében hajtódik végre. A rendszer így tömörebb lett és gyorsabb végrehajtást tesz lehetővé.
- A predikátum logikára épül, de vannak eltérések:
 - Csak az elemváltozók kezdődnek **nagybetűvel.**
 - Ítéletkonstansok közül a False (fail) **ritkán**, a True és ítéletváltozók még ritkábban fordulnak elő.
 - **Rendszerpredikátumok** léteznek, melyek valamit elvégeznek és közben igaz értéket vesznek fel. Pl.: read(). write(), circle(), stb.
 - **Infix** (közbeírt) alakú relációoperátorok (<,>, <=,>=,<>=,><) és aritmetikai operátorok (+,-,*,/) a predikátumos alak helyett. Pl.: kisebb(X,Y) helyett $X < Y$ írható.

- Egyenlőséges reláció egyetlen hiányzó értékű változójának képes **kiszámítani** az értékét. Pl.: $5*7 = X \Rightarrow X=35$
- Aritmetikai kifejezéseket, műveleti operátorokat végrehajtja.
- A szabályokban szereplő **if** (:-) logikai szimbólumot úgy kezeli, mint egy infix alakban használt speciális rendszer-predikátumot, amely a PROLOG-tól egy speciális kiértékelő algoritmust kíván.
- A szabályok és az atomok (főként predikátumok) ugyanolyan módon kezelendő objektumoknak minősülnek.
- Megengedi **ítéletváltozók** alkalmazását a szabályokban és megengedi **helyettesítésüket szabályokkal**.
- Képes manipulálni saját "adatbázisát" (az **assert** és a **retract** rendszerpredikátumok útján.)
- Kényelmes [**Fej** | **Törzs**] listaalak. [] az üres lista jele.
- A megoldási folyamat programozó általi vezérlésére alkalmazza a **!jelet**, mely formailag egy atomi formula. Pl.: $a \text{ if } b \text{ and } c \text{ and } ! \text{ and } d$. A **!** (**cut**) megváltoztatja, lerövidíti a PROLOG Depth-First (mélységi bejárás) algoritmusát a felesleges ágak bejárásának letiltásával, az ágak

Példa

A PROLOG programok felépítésének és eszköz-rendszerének részletezése előtt tekintsünk egy egyszerű feladatot és PROLOG programját!

Ismertek az alábbi tények:

Cleopátra szebb Ginánál.

Gina szebb Ursulánál.

Kérdés: *Ki sokkal szebb Ursulánál ?*

- A feladat megválaszolására alkalmas **PROLOG program:**

domains hölgy = **symbol**

predicates szebb(hölgy, hölgy)
 sokkal_szebb(hölgy,hölgy)

goal sokkal_szebb(Valaki, ursula)

clauses szebb(cleopátra,gina).

 szebb(gina, ursula).

 sokkal_szebb(A,C) if szebb(A,B) and

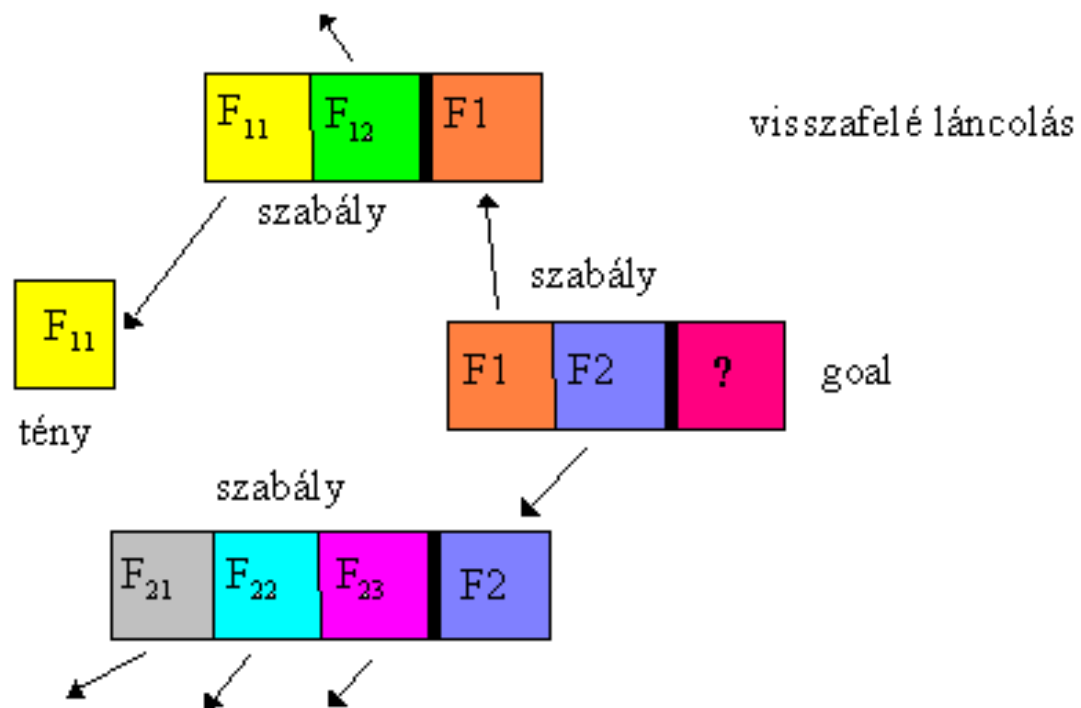
szebb(B,C).

A PROLOG működése

A működés két alapvető összetevője az

- **illesztés** (matching) és a
- **visszalépés** (backtracking).

- Lényege: a goal-ban feltett kérdést megpróbálja **tényállításokra visszavezetni**. Amennyiben nem létezik illeszkedő - nevében és a megadott paramétereiben egyező - tény, akkor illeszkedő szabályt keres. Ha van illeszkedő szabály, akkor az előbbi lépések ismétlésével megpróbálja a szabály Törzsében lévő feltételek kiértékelésével meghatározni a szabály Fejének igazságértékét (Hátraláncolás).
- Az **összes** lehetőség végigpróbálásával megállapítja, hogy a kérdés teljesülhet-e, és ha igen, hányféle módon.
- Ha próbálkozás közben zsákutcába jut - nincs illeszkedő tény, vagy szabály -, **visszalép** a legutolsó elágazási pontig, ahol a több illesztési lehetőség közül a következőt választja.



Példa fagráf

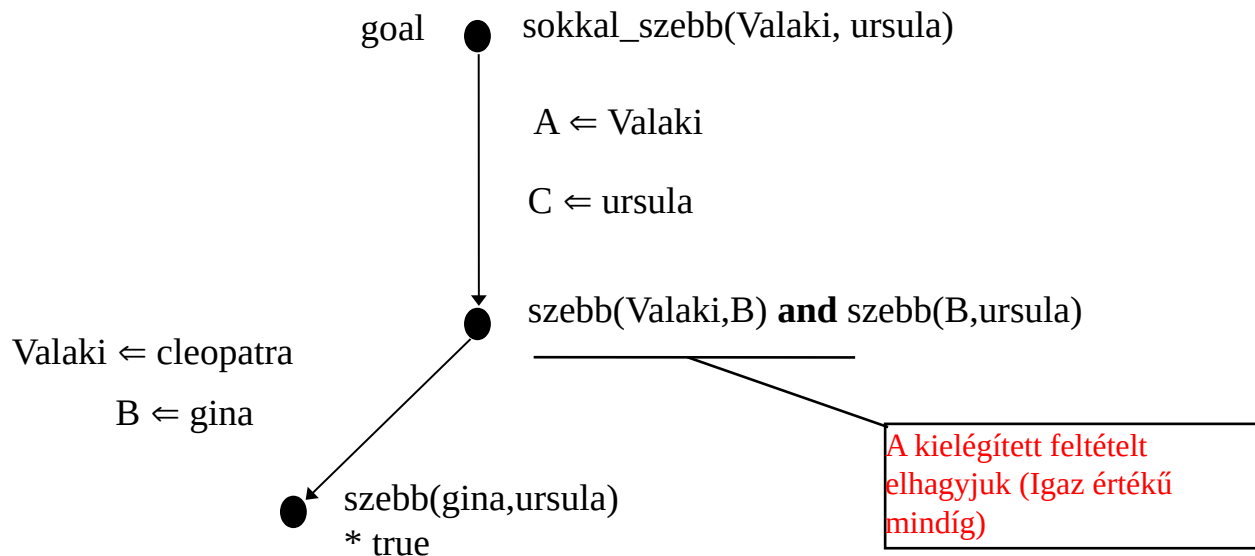
domains hölgy = **symbol**

predicates szebb(*hölgy*, *hölgy*)
 sokkal_szebb(*hölgy*, *hölgy*)

goal sokkal_szebb(*Valaki*, *ursula*)

clauses szebb(*cleopátra*, *gina*).
 szebb(*gina*, *ursula*).

sokkal_szebb(*A*, *C*) **if** szebb(*A*, *B*) **and** szebb(*B*, *C*).



Példa fagráf módosítva

domains hölgy = **symbol**

predicates szebb(hölgy, hölgy)

sokkal_szebb(hölgy, hölgy)

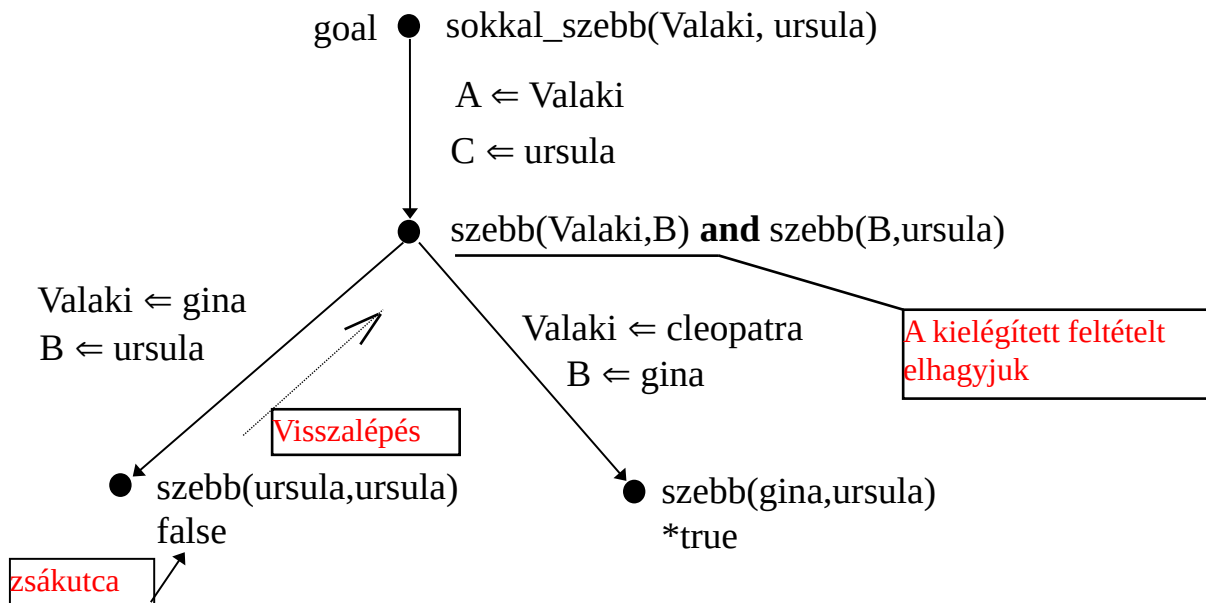
goal sokkal_szebb(Valaki, ursula)

clauses szebb(gina, ursula).

szebb(cleopátra, gina).

sokkal_szebb(A,C) **if** szebb(A,B) **and** szebb(B,C).

A tényeket felcseréltük



A Fuzzy logika alapjai

A Fuzzy logika Lofti Zadeh nevéhez fűződik. (1965)

- A hagyományos kétértékű logika helyett végtelenértékű, **folytonos változókat** és a hamis és igaz közötti folytonos átmenetet használ a fogalmak, jellemzők igazságértékének megadására.
- Hiányos adatok elfogadása, zajtűrő képessége, kollektív döntést utánzó működése a mesterséges neurális hálózatokhoz hasonlóan **robusztus**, megbízható alkalmazások készítését teszi lehetővé bizonytalan adatok esetére is.
- Szimbolikus szinten működik, **nyelvi változókat** használ, ezáltal közel áll az emberi gondolkodáshoz.
- Alkalmazása különösen a szabályozástechnikában igen elterjedt.

Fuzzy halmazok

- Egy elemnek a fuzzy halmazba való tartozásának foka, 0-1 közötti mértéke van, nem merev igen/nem.

- Értelmezés:

Ha **X** az x jellemzők, értékek egy csoportja, akkor az **X**-en értelmezett **A** fuzzy halmaz a következő rendezett párok halmaza:

$$\mathbf{A} = \{ (x, \mu_A(x)) : x \in \mathbf{X} \}$$

A $\mu_A(x)$ neve *tagsági függvény*, értéke az x tagsági aránya, beletartozásának mértéke az **A** fuzzy halmazba, ill. az **A** tagsági függvényhez társított nyelvi érték igazságának foka.

- A $\mu_A(x)$ tagsági függvény az **X** értékalmazt leképezi az **M** tagsági térbe ($\mu_A(x) \in [0;1]$).

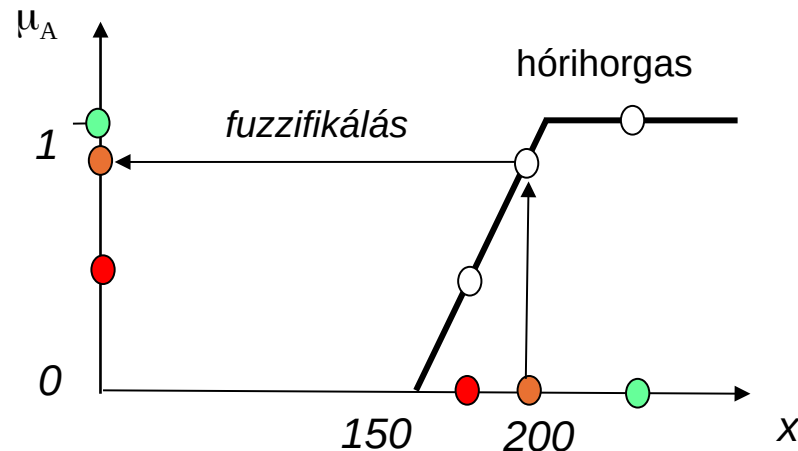
•

Példa: **X** testmagasságértékek [cm].

A halmaz: a „hórihorgas” nyelvi értéknek megfelelő x , $\mu_A(x)$ számpárok halmaza, ábrája:

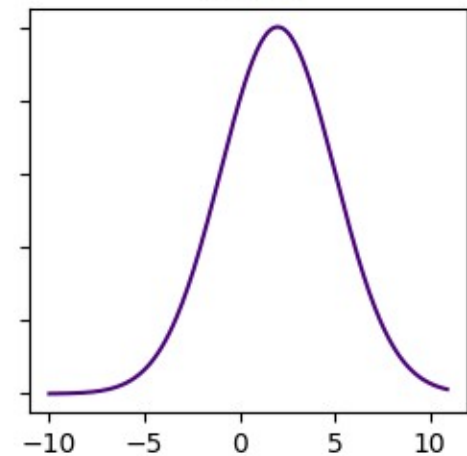
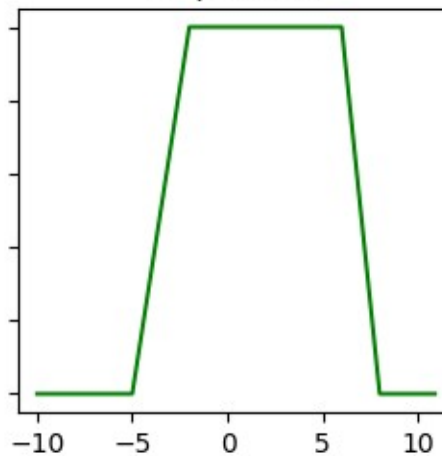
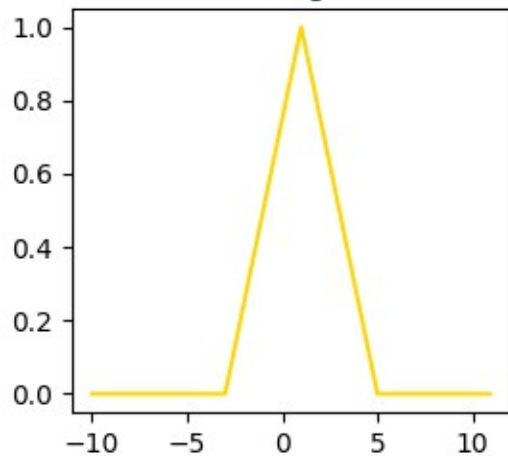
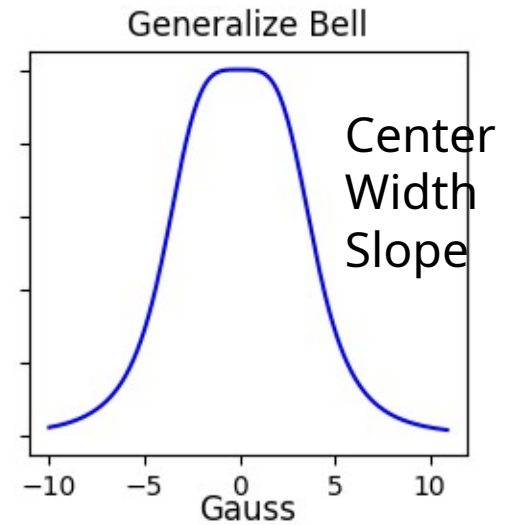
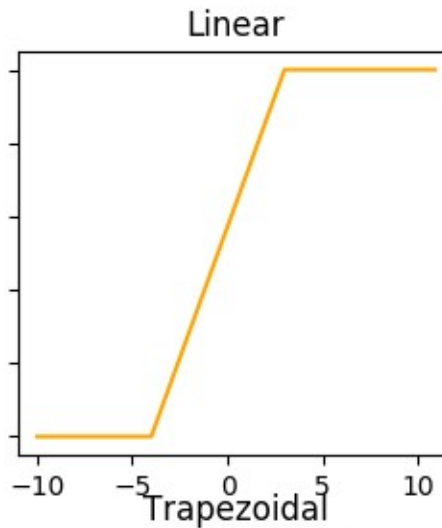
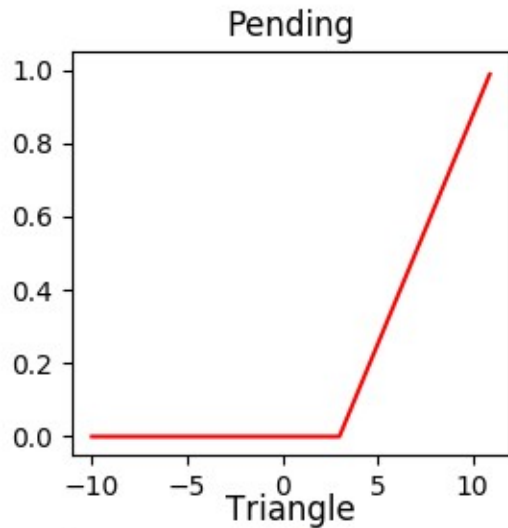
fizikai változó: $x = 200$;

fuzzy változó: testmagasság = hórihorgas
igazságértéke = 0.9

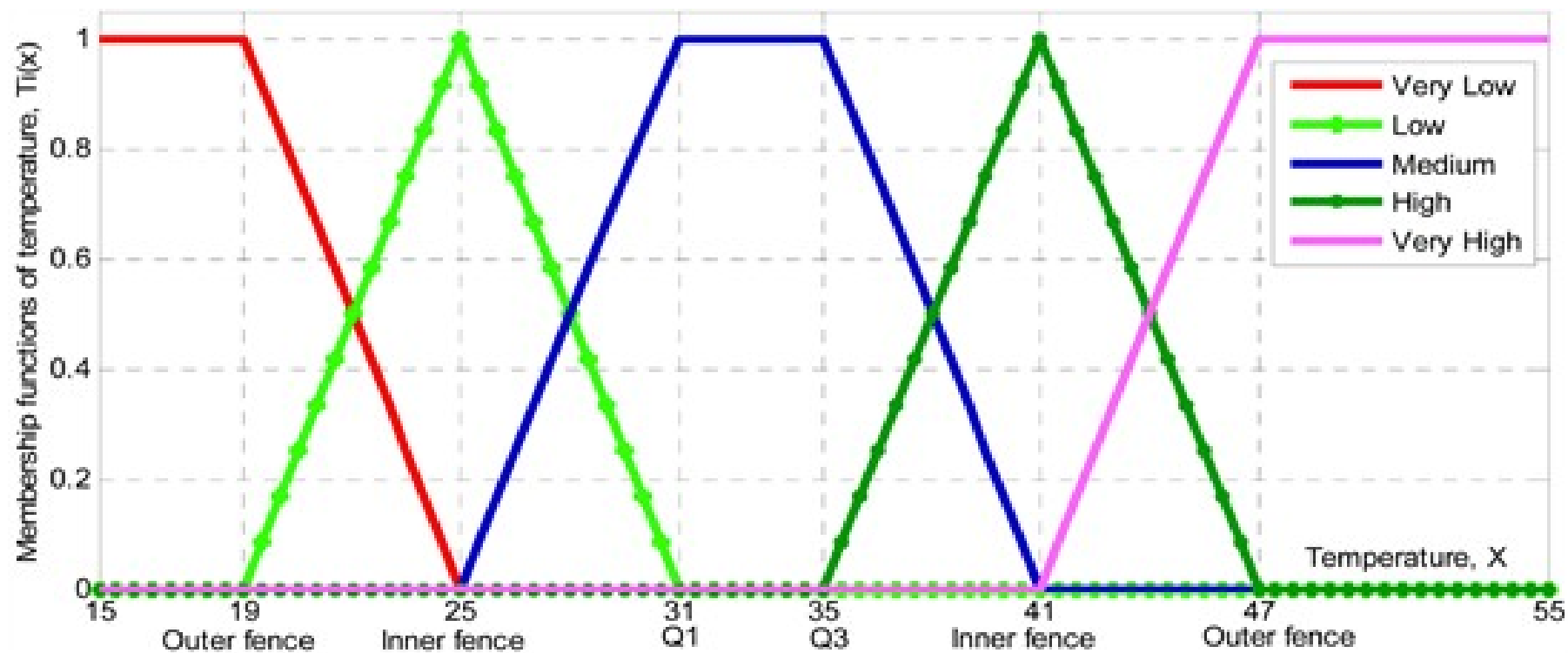


Tagsági függvények

Membership functions



Mean
Standard
deviation



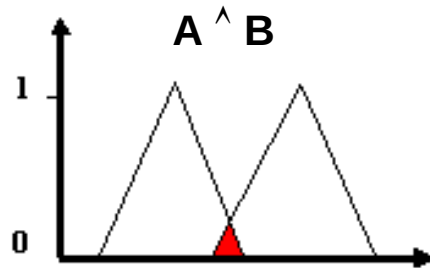
Fuzzy logikán alapuló következtető rendszer

- Produkciós, szabályalapú működés ($R1$: Ha $x=A$ akkor $y=B$)
 - A az $R1$ szabály antecedense (előzmény, feltétel), B pedig az $R1$ szabály konzekvense (következmény)
 - Tehét a szabályok feltételoldalán fuzzy ÉS, ill. fuzzy VAGY logikai műveletekkel összekötött
 - fuzzy változó = nyelvi érték
- beletartozási relációk vannak, melyek értékét a fuzzy változóhoz tartozó fizikai változó (x) aktuális értéke és a nyelvi értékhez tartozó tagfüggvény együttesen határozzák meg.
- A feltételoldali logikai kifejezés eredő érvényességi, igazsági értékét a fuzzy ÉS, ill, VAGY logikai műveletek értelmezésének felhasználásával számítjuk és ez lesz a következmény oldal érvényessége is egyben.
- A következmény oldalon egyetlen
 - fuzzy változó = nyelvi érték
- reláció áll, melynek érvényessége, igazságértéke megegyezik a feltételoldalra meghatározott értékkel.

Fuzzy műveletek

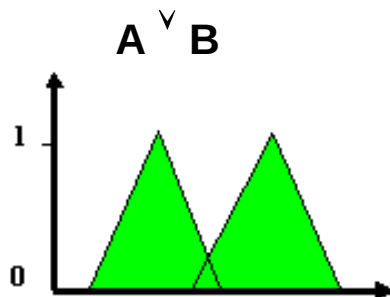
- Fuzzy ÉS művelet értelmezése ($C = A \wedge B$):

$$\mu_C(x) = \min\{\mu_A(x), \mu_B(x)\}, x \in X$$



- Fuzzy VAGY művelet értelmezése ($C = A \vee B$):

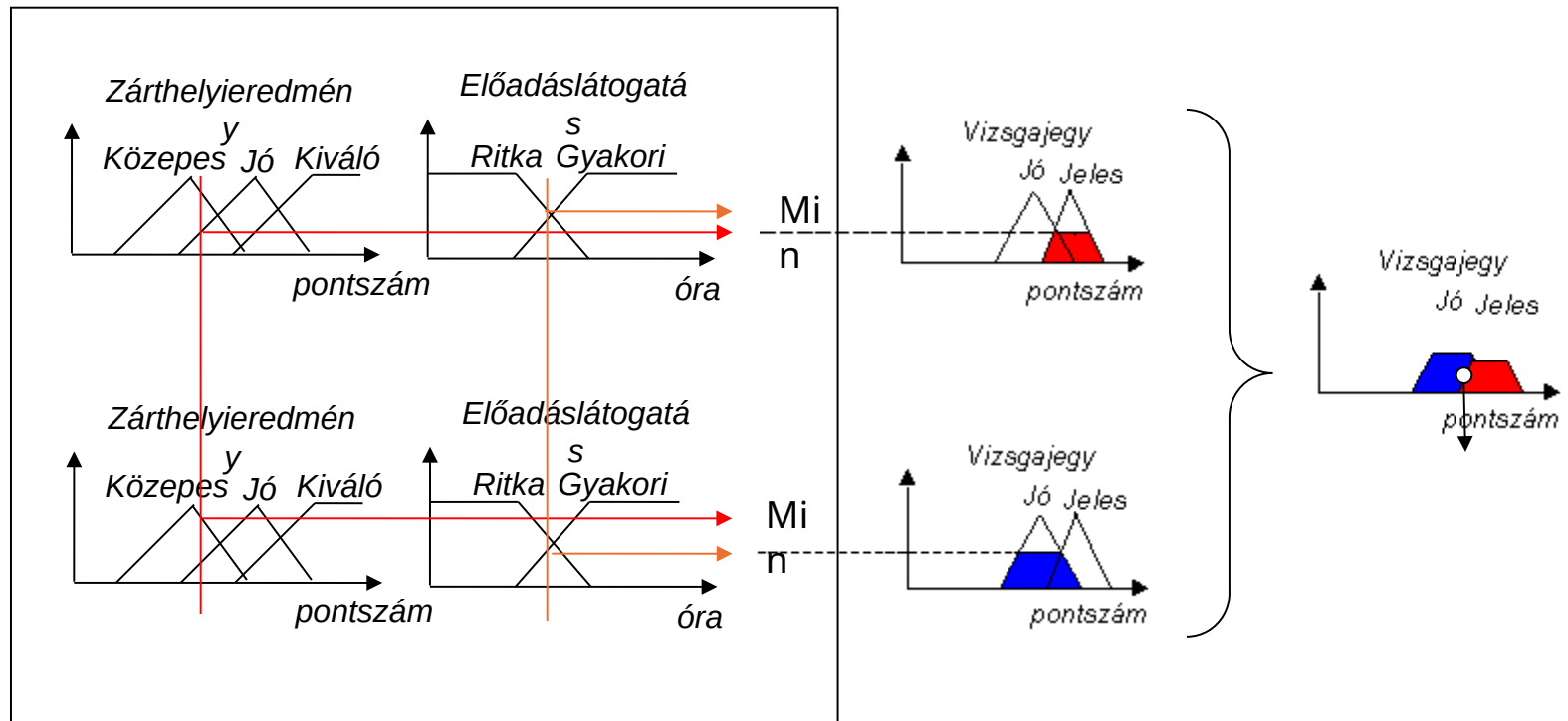
$$\mu_C(x) = \max\{\mu_A(x), \mu_B(x)\}, x \in X$$

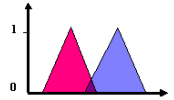


- Az összes szabály egyszerre működik a fizikai változók, mint bemenő tények felhasználásával és kollektív döntést hoz. A kollektív döntés eredményét a fuzzy nyelvi értékek tagfüggvényeinek felhasználásával, a fuzzy változó terében az érvényességi értékekről a fizikai változóra való áttéréssel, ún. defuzzifikálással nyerjük.
- A defuzzifikálás módszerei:
 - Területközep pont meghatározással (legelterjedtebb eljárás)
 - Tagsági függvény maximumok súlyozott átlagának meghatározásával
 - A legnagyobb érvényességű következmény-tagfüggvény maximumával.

If Zárthelyieredmény = Jó ÉS Előadáslátogatás = Ritka then Vizsgajegy = Jeles

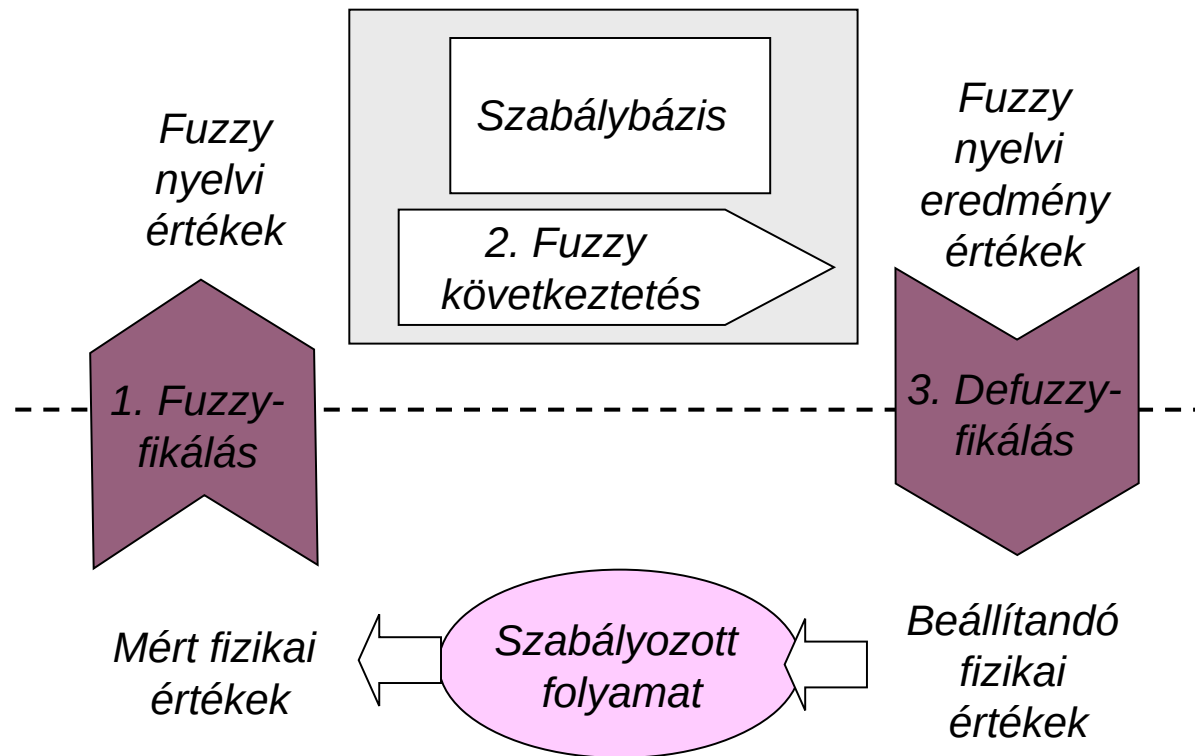
If Zárthelyieredmény = Közepes ÉS Előadáslátogatás = Gyakori then Vizsgajegy = Jó





- A Fuzzy logika alapjai ..

Fuzzy szabályozórendszer vázlata



Mesterséges neurális hálózatok

2024/25/1

Motiváció

- A mesterséges neurális hálózatok kutatásának, kifejlesztésének célja az emberi információfeldolgozás mechanizmusainak megértése és felhasználása az emberi mentális folyamatok gépi modellezésében
- Ezen tevékenység összetevői:
 - A kognitív folyamatok alapvető figyelembevétele
 - Párhuzamos számítási egységeket tartalmazó új formájú számítási modell, mely figyelembe veszi az idegrendszerből, pszichológiából eredő követelményeket, valamint a számítási lehetőségek mélyen átgondolt adottságait
 - A megismerésre vonatkozó új koncepciók melyek áthelyezik a hangsúlyt a szimbolikus feldolgozásról azon állapotokra, melyek tükrözik az ábrázolandó ismeretek és a megalkotott működési struktúrák lehetséges konfigurációi közötti egyezés minőségét
 - A tanulás hangsúlyozása (folyamatos alkalmazkodás és tanulás köré épített mechanizmusok)

Mesterséges neurális hálók

- A mesterséges neurális hálók (Artificial Neural Networks, ANN) a biológiai neurális hálók modellezésével létrehozott, erőteljes elosztott párhuzamos feldolgozást, kollektív számítást megvalósító rendszerek.
- Mivel a tudást olyan módon hordozzák és dolgozzák fel, hogy az az ember számára közvetlenül követhetetlen, a szimbolikus tudásszemléltetést alkalmazó modellektől való megkülönböztetést a szubszimbolikus elnevezéssel is hangsúlyozzák.
- A működésben domináló kapcsolódásokat kiemelő „konnekcionista rendszerek” elnevezés, valamint a tanulóképessegre utaló „adaptív hálók” elnevezés is használatos.

- Mai neurális hálók még messze állnak a teljes céltól, az emberi aggyal összemérhető szintű működéstől
- Jelenlegi szinten iteratív numerikus algoritmusok osztályába sorolhatóak, de vannak eltérések
 - A párhuzamos feldolgozásra való alkalmasság
 - Zajos és hibás input kezelésének képessége
 - Nemlineáris numerikus számítási elemek alkalmazása
 - Elmélet analízisük hiányosságai ellenére sikeresek a gyakorlati alkalmazások

Mesterséges neurális hálók által megoldható feladatok

- Statisztikai feladatok
 - Mintázatfelismerés
 - Regresszió- és sűrűségfüggvény becslés
 - Kompresszióelemzés
 - Független komponensekre való szétválasztás
- Operációkutatás
 - Kombinatorikus optimalizálás
- Analízis
 - Függvény-approximáció

A mesterséges neurális hálók fő működési jellemzői

- Tanulás
- Általánosítóképeség
- Zajos és hiányos adatok elfogadása
- Gyors működés
- Szövevényes viszonylatok kezelése
- Önszervezés
- Hatalmas adatmennyiség elemzése
- Rugalmasság

Hol használnak neurális hálót?

Példák

- A New York-i Gyógyászati Központban, művégtagok mozgatásához szükséges számítások elvégzésére használtak ANN-t.
- A NASA neurális hálót alkalmazott robotoknál a véletlenszerűen elhelyezkedő tárgyak megfogásához szükséges mozgások vezérlésére.
- A General Dynamics egy vízalatti figyelőrendszerben alkalmazott ANN-t. Ez képes volt a vízi járművek beazonosítására a vízben terjedő hangjuk alapján. Még a vízfelszínen hullámokat kavaró helikopter felismerése sem okozott gondot.
- A pennsylvaniai egyetemen kifejlesztett katonai repülőgép-felismerő rendszer képes volt megkülönböztetni 18 hüvelykes részleteket 50 mérföld távolságról, és repülőgépeket beazonosítani a teljes minta 10%-ának birtokában.
- Az amerikai légierő egy ANN alapú repülésszimulátort készített pilóták kiképzéséhez.
- A Ford autógyár ANN-t alkalmaz a motor szenzorjeleinek figyelésére és a fellépő problémák beazonosítására.
- New York repülőterén egy ANN-en alapuló bombadetektort használnak.
- Amazon, AliExpress ANN segítségével mutat ajánlott és kapcsolódó termékeket.
- Tesla automata pilóta funkciója ANN segítségével ismeri fel az útburkolati jelöléseket és az akadályokat

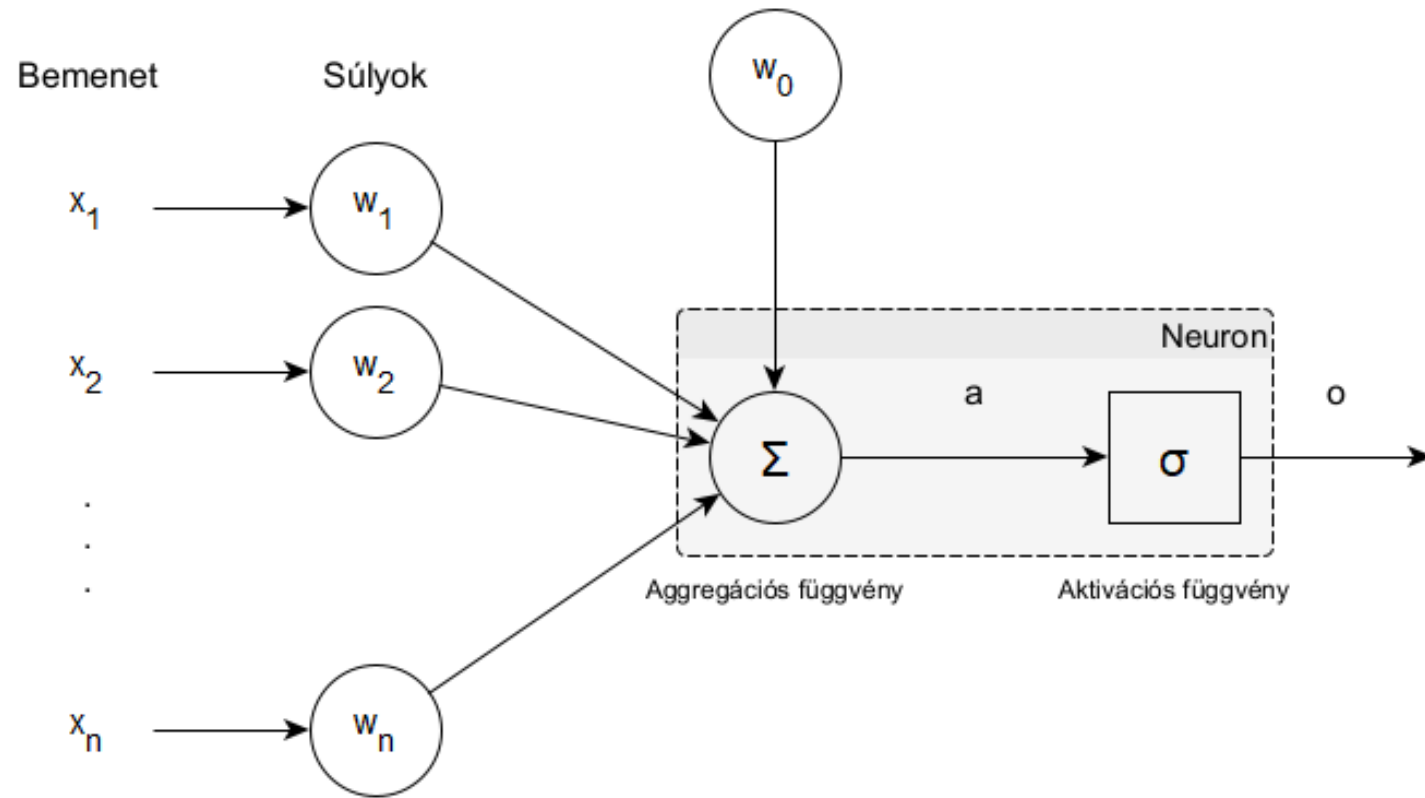
Mesterséges neuron

- Rosenblatt találta fel 1957-ben
- A modell biológiai neuronokhoz való hasonlósága mellett érvelt
 - Sok input- dendritek
 - Egy output- axon
 - Ha az ingerek (súlyozott) összege elér egy küszöböt, a neuron tüzel, különben nem
 - A neuron kimenete bináris

Mesterséges neuron

- Az inputok az $x_1, x_2, \dots, x_n$ jellemzőknek felelnek meg
- Az egyes kapcsolatok erősségét a $w_1, w_2, \dots, w_n$ súlyok modellezzik
- Ezek mellett van egy w_0 „bias” paraméter
- A neuront érintő ingerek súlyozott összességét aktivációnak hívjuk
 - $a = \sum_1^n w_i x_i + w_0$
- Az a aktivációból az $o(a)$ outputot az aktivációs függvény állítja elő
 - Az eredeti modellben ez egy egyszerű küszöbölést végez
- A modell tanulandó paraméterei a $w_0, w_1, w_2, \dots, w_n$ súlyok

Mesterséges neuron ábra



Mesterséges neuronok kapcsolata

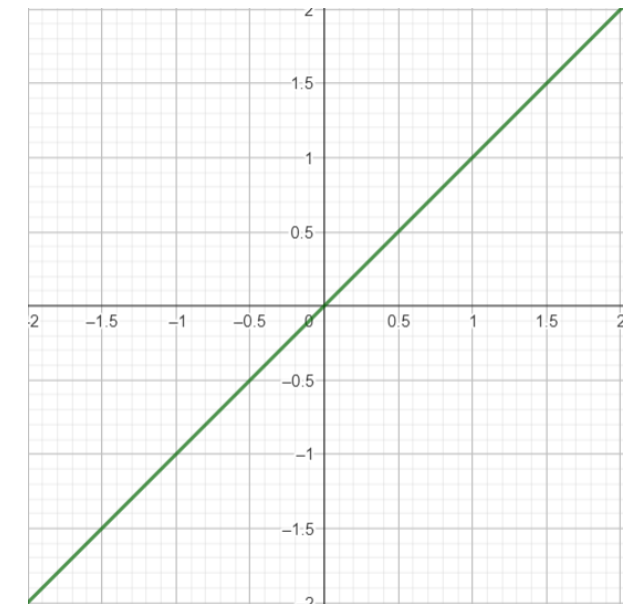
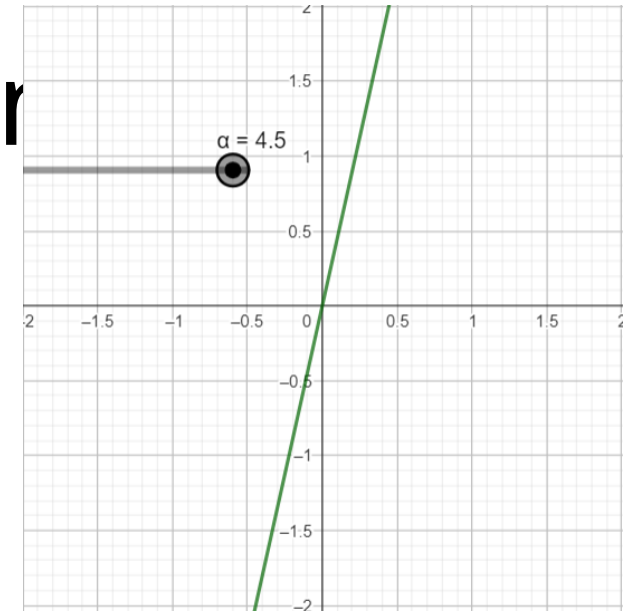
- Egy mesterséges neurális háló egyszerű számítási egységekből, mesterséges neuronokból áll, melyek egymásnak küldött jelekkel kommunikálnak. A jelek nagyszámú súlyozott kapcsolaton áramlanak.
- u_i a háló i . mesterséges neuronja
- o_j a j . neurontól érkező jel az u_i neuron bemenetére
- w_{ij} a j . neurontól az i . neuronba érkező jel szorzója, a u_j - u_i kapcsolat súlya
- a_i az u_i neuron aktivációs potenciálja, melyet a bemenetek súlyozott összegéből az aggregációs függvény állít elő. Ez a függvény egyszerű esetben elmarad. Az a_i aktivációs potenciál gerjesztő input híján időben csökkenhet, a gerjesztett, izgatott állapotból visszatérhet egy inaktív szintre
- o_i az aktivációs függvény által generált aktivációs potenciálból számított u_i neuron kimenő jele.

Aktivációs függvények

- Leggyakrabban olyan aktivációs függvényeket használunk, amelyek
 - Monoton növekvőek
 - Jobbról folytonosak
 - Határértékük a $-\infty$ -ben 0, a $+\infty$ -ben pedig 1
- Fajtái
 - Lineáris
 - Nem-lineáris

Lineáris aktivációs függvény

- Képlete:
 - $o_i = f(a_i) = \alpha * a_i$, ahol α egy valós szám
- Ha $\alpha=1$, akkor az identitás függvény
- Monoton
- Vannak vele problémák, mivel a deriváltja konstans, így nem annyira szokták használni

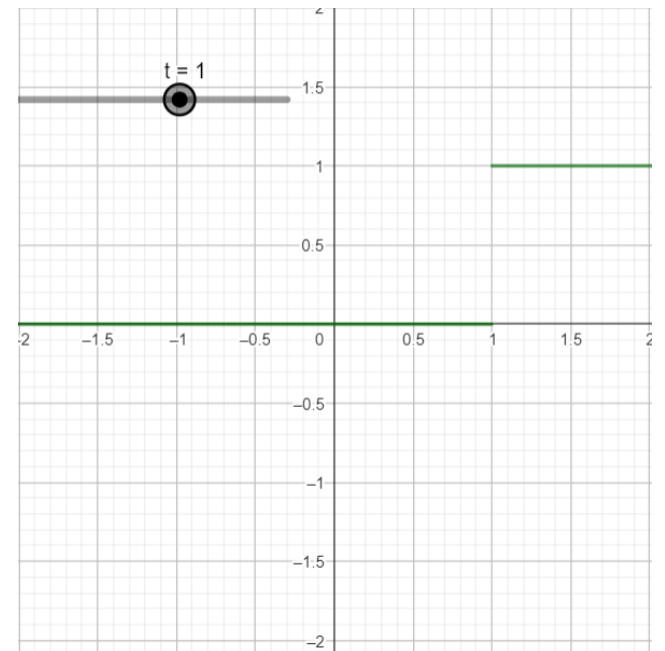
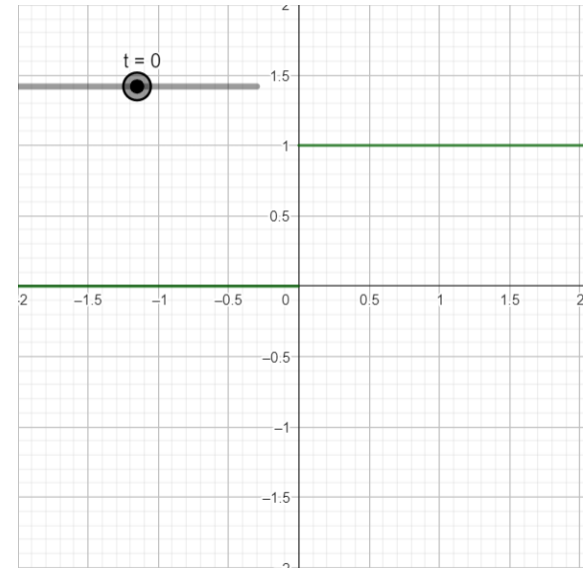


Küszöb aktivációs függvény

• Képlete:

$$o_i = f(a_i) = \begin{cases} 0 & \text{if } t > a_i \\ 1 & \text{if } a_i \geq t \end{cases}$$

- A változtathatóság egyszerűsítésére a küszöbértéket nem a neuronban adják meg, hanem egy plusz bemenet megadásával realizálják.
- A bemenet inputértékeként -1 értéket megadva, a bemenet súlya meg fog egyezni a szükséges küszöbértékkel. Ily módon a betanítás során végzett automatikus súlymódosítások a küszöböt is állítani fogják.
- Egy n bemenetes neuron tüzel, ha
 - $w_{i1} * x_1 + w_{i2} * x_2 + \dots + w_{in} * x_n > \theta$
- Ugyanígy, az $n+1$ bemenetes neuron tüzel, ha
 - $w_{i1} * x_1 + w_{i2} * x_2 + \dots + w_{in} * x_n + \theta * (-1) > 0$
- Monoton
- Deriváltja nem monoton

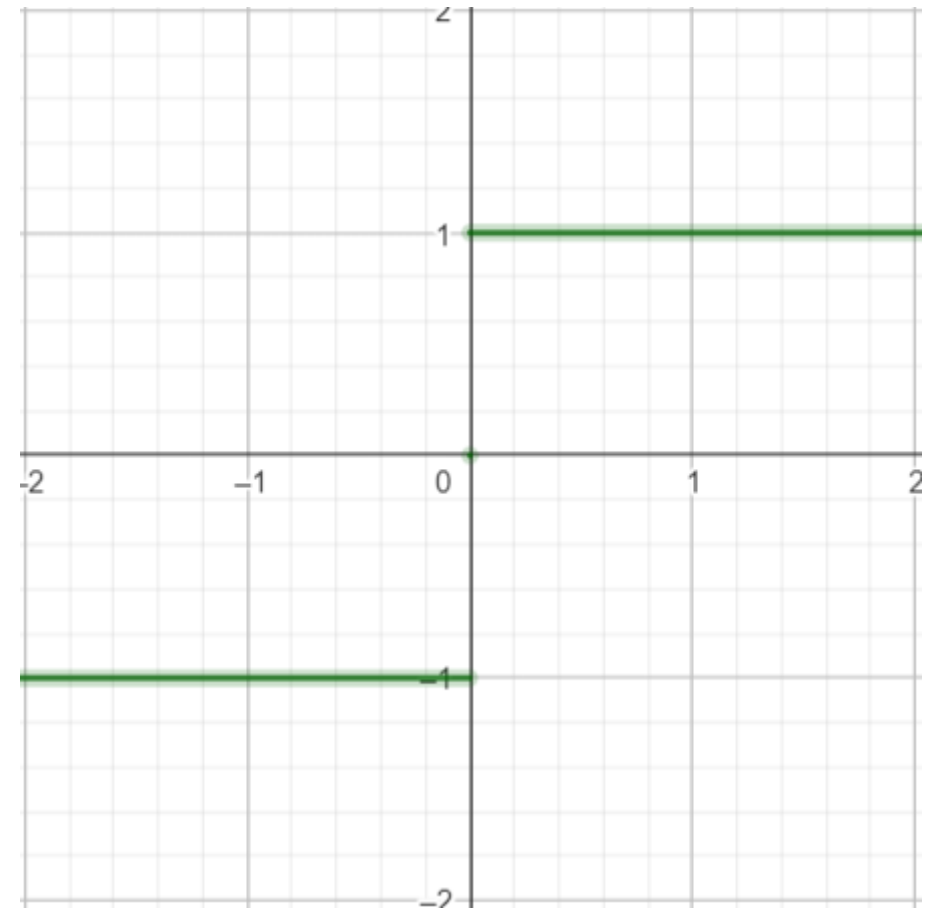


Szignum aktivációs függvény

◦ Speciális küszöb aktivációs függvény

• Képlete:

$$o_i = f(a_i) = \begin{cases} -1 & \text{if } 0 > a_i \\ 0 & \text{if } a_i = 0 \\ 1 & \text{if } a_i > 0 \end{cases}$$

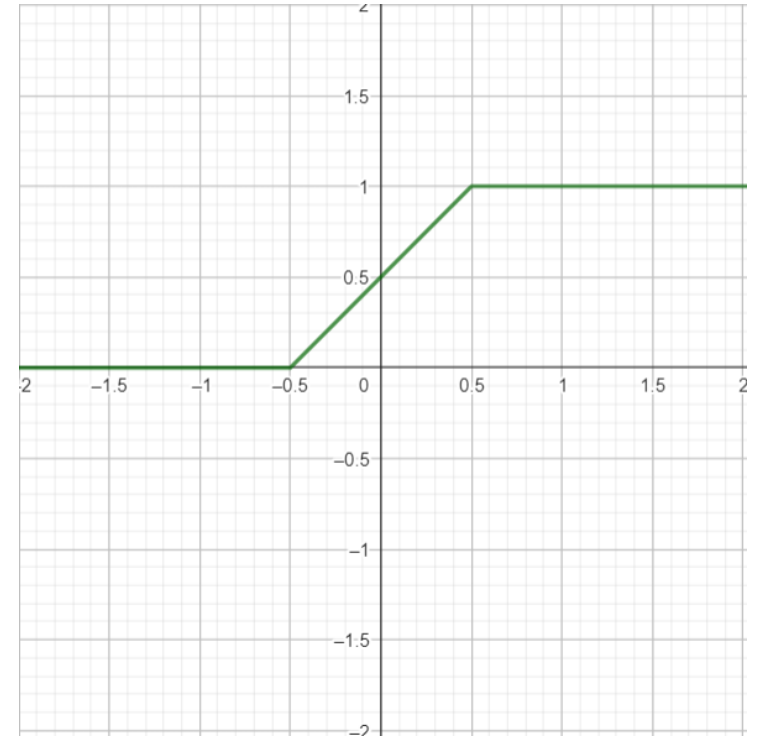


Darabonként-lineáris aktivációs függvény

- Képlete:

$$o_i = f(a_i) = \begin{cases} 1 & \text{if } a_i \geq 0.5 \\ a_i + 0.5 & \text{if } -0.5 < a_i < 0.5 \\ 0 & \text{if } a_i \leq -0.5 \end{cases}$$

- Akár a határokat, és a hozzáadott értéke(ke)t is fel lehet venni paraméterként
- Paraméterektől függ a monotonitás

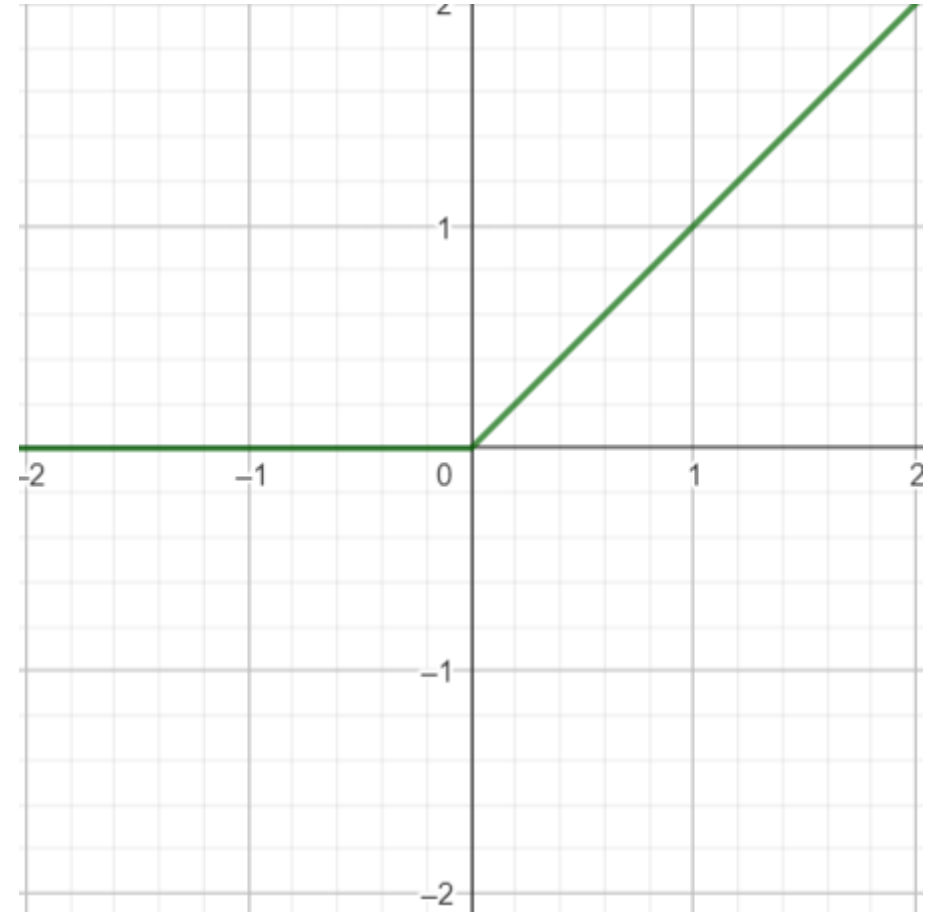


ReLU aktivációs függvény

Rectified linear unit

• Képlete:

$$\bullet o_i = f(a_i) = \begin{cases} 0 & \text{if } 0 \geq a_i \\ a_i & \text{if } a_i > 0 \end{cases} = \max(0, a_i)$$



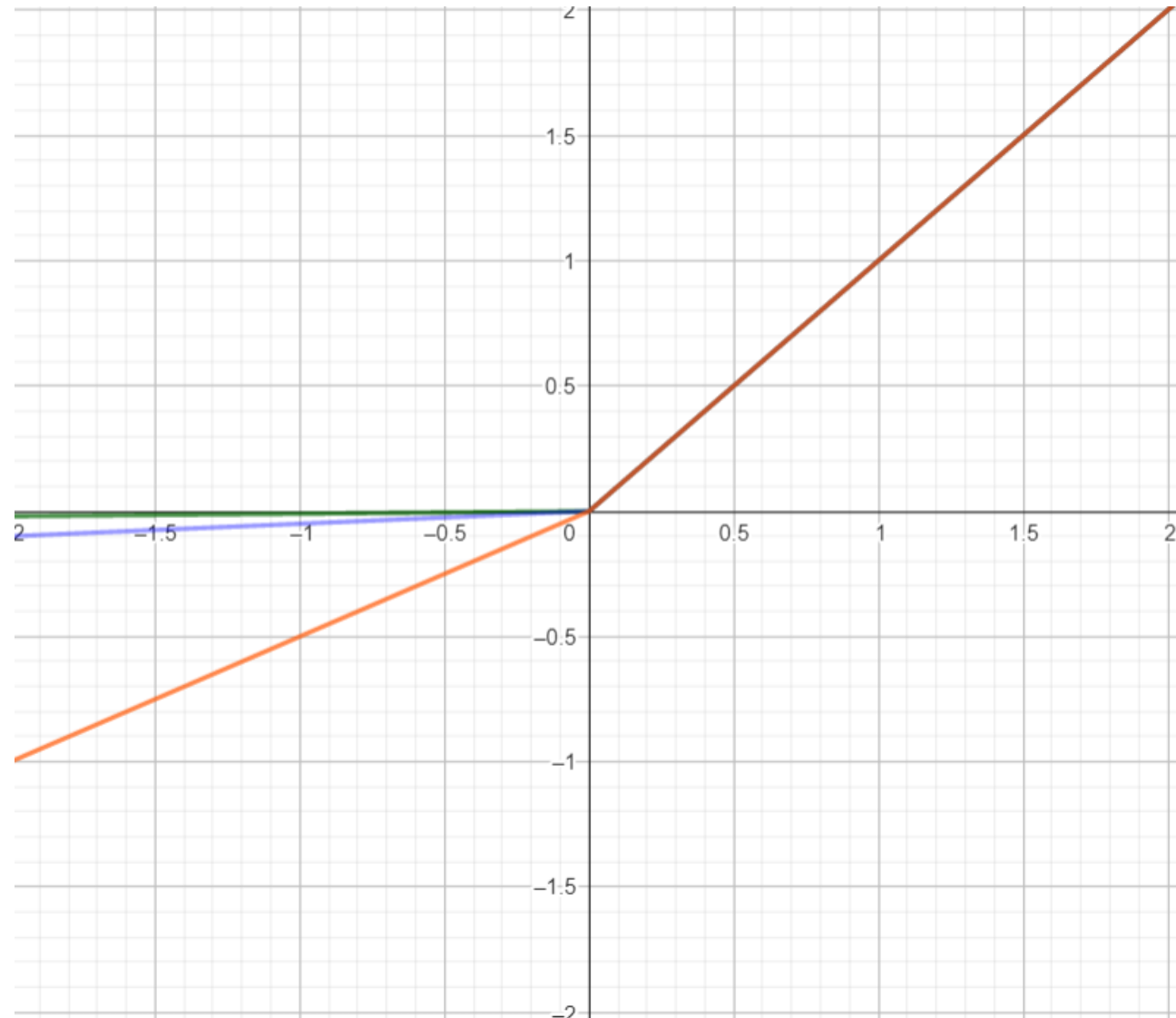
Leaky ReLu aktivációs függvény

Szivárgó ReLU

• Képlete:

$$o_i = f(a_i) = \begin{cases} 0.01a_i & \text{if } 0 > a_i \\ a_i & \text{if } a_i \geq 0 \end{cases}$$

- A szivárgás mértéke lehet akár paraméter is, akkor a neve Parametric rectified linear unit (PReLU)



Nem-lineáris aktivációs függvények

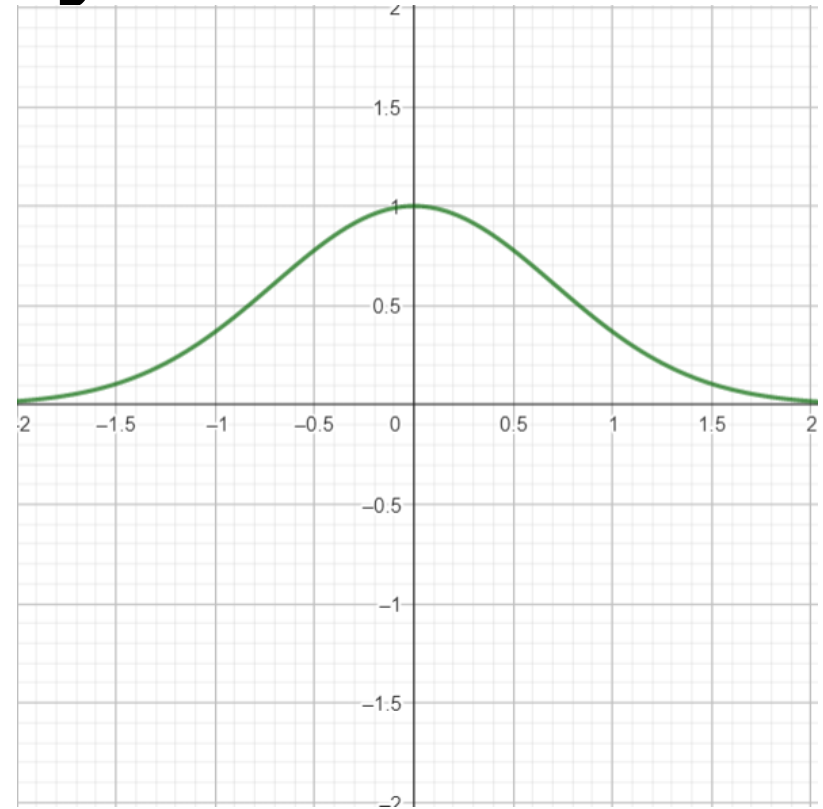
Gauss aktivációs függvény

• Képlete:

- $o_i = f(a_i) = e^{-ai^2}$

- Nem monoton

- Deriváltja nem monoton

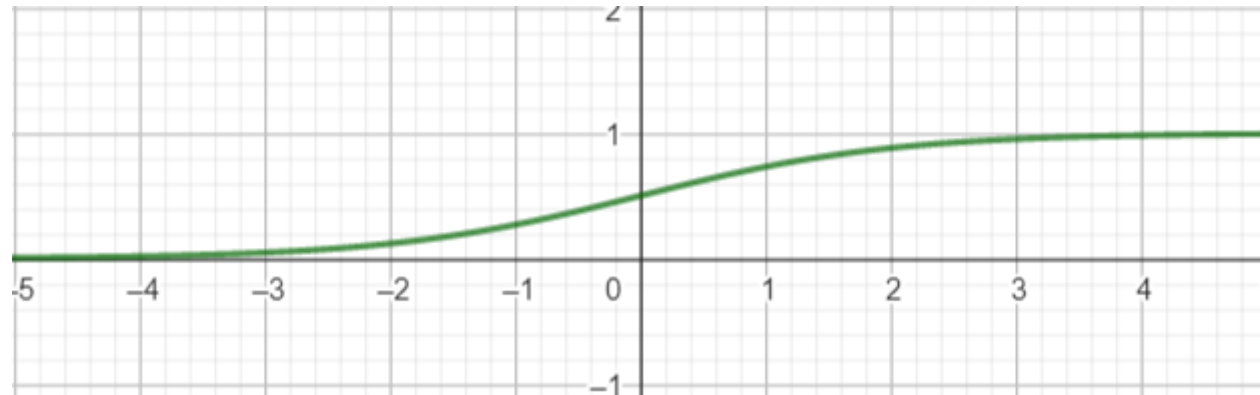


Sigmoid/logisztikus aktivációs függvény

• Képlete:

$$• o_i = f(a_i) = \frac{1}{1+e^{-a_i}}$$

• A leggyakrabban használt aktivációs függvény



Swish

◦ A szigmoid módosítása

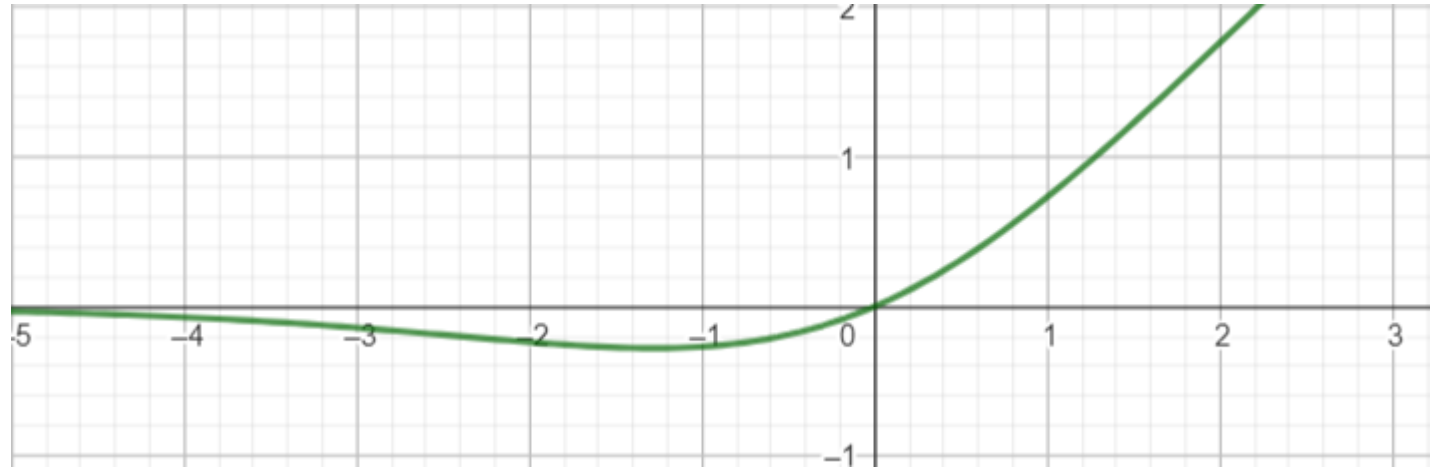
- Nem monoton

- Képlete:

- $o_i = f(a_i) = \frac{a_i}{1+e^{-a_i}}$

- Létezik tanítható, parametrizált változata is:

- $o_i = f(a_i) = \frac{a_i}{1+e^{-\beta a_i}}$





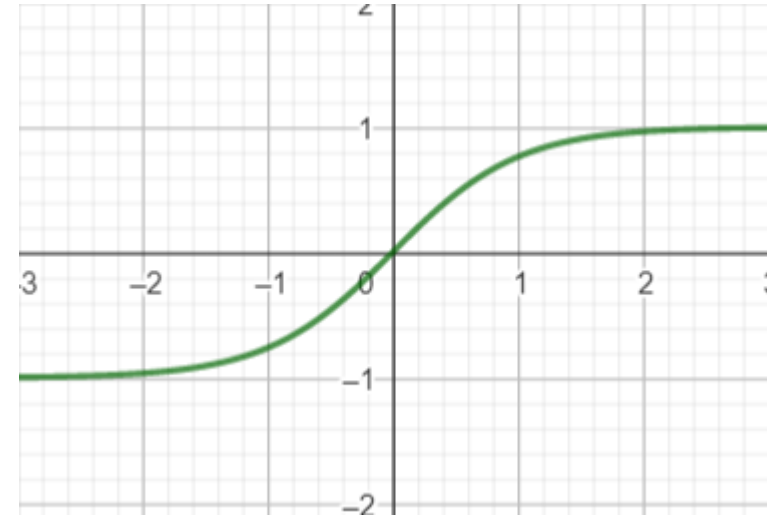
+ Parancssor...



Tangens-hiperbolikus aktivációs függvény (tanh)

• Képlete:

$$\bullet o_i = f(a_i) = \frac{e^{a_i} - e^{-a_i}}{e^{a_i} + e^{-a_i}}$$

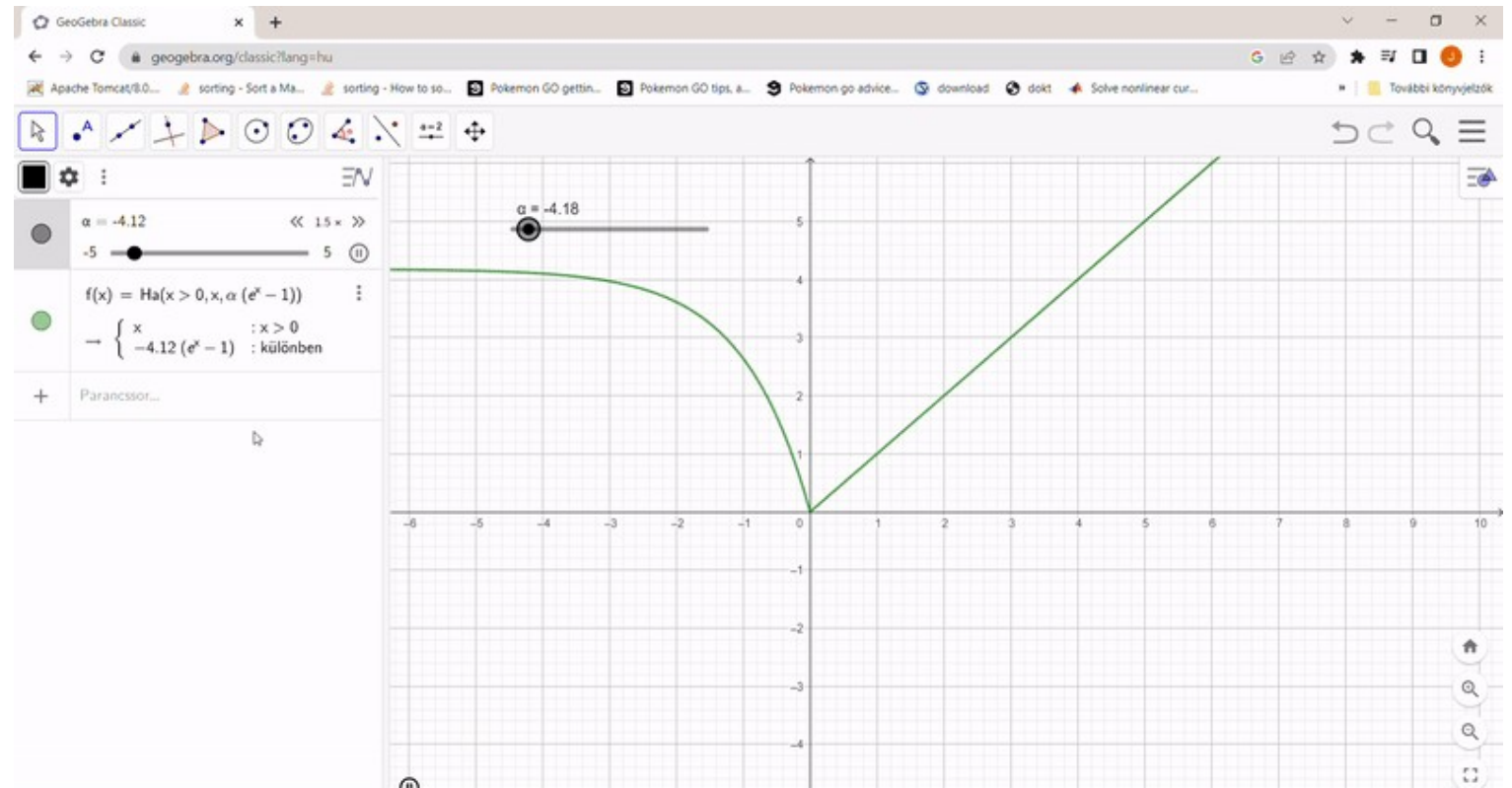


Exponential linear unit aktivációs függvény

ELU

• Képlete:

$$o_i = f(a_i) = \begin{cases} \alpha(e^{a_i} - 1) & \text{if } a_i \leq 0 \\ a_i & \text{if } a_i > 0 \end{cases}$$



Hálózati topológiák

- A hálózat topológiáján a neuronok összeköttetési rendszerét és a hálózat bemeneteinek és kimeneteinek helyét értjük.
- A neuronokat egymáshoz kapcsolják, és a rendszer általános viselkedését ezen kapcsolatok struktúrája és erőssége (w_{ij}) adja meg.
- A neuronokat csoportokba, vagy rétegekbe rendezik el. Az egyetlen réteget alkotó, egymással kapcsolódó neuronok hálózatát gyakran tartalom által címezhető memóriának (CAM, Content Addressable Memory) nevezik.
- A neurális hálók eltérő működéséért elsősorban az eltérő topológia a felelős

- A hálózat topológiáját általában irányított gráffal reprezentáljuk.
- A gráf csomópontjai a neuronoknak felelnek meg, míg a kapcsolatokat a kimenetek és bemenetek között a gráf élei reprezentálják a neuron-bemenettől a kimenet felé irányítva. (Az élekhez értelemszerűen hozzárendelhetjük a megfelelő w_{ij} bemeneti súlytényezőket s

- A gráf egyes csomópontjai rendszerint nincsenek az összes többi csomóponttal kapcsolatban, csupán a csomópontok egy részhalmazával.
- Ez legtöbbször lehetőséget nyújt arra, hogy a gráf csomópontjainak – a neuronoknak – halmazát diszjunkt részhalmazokra bontsuk.
- Ennek megfelelően háromféle neuront különböztethetünk meg:
 - Bemeneti neuron
 - Kimeneti neuron
 - Rejtett neuron

Neuronok fajtái

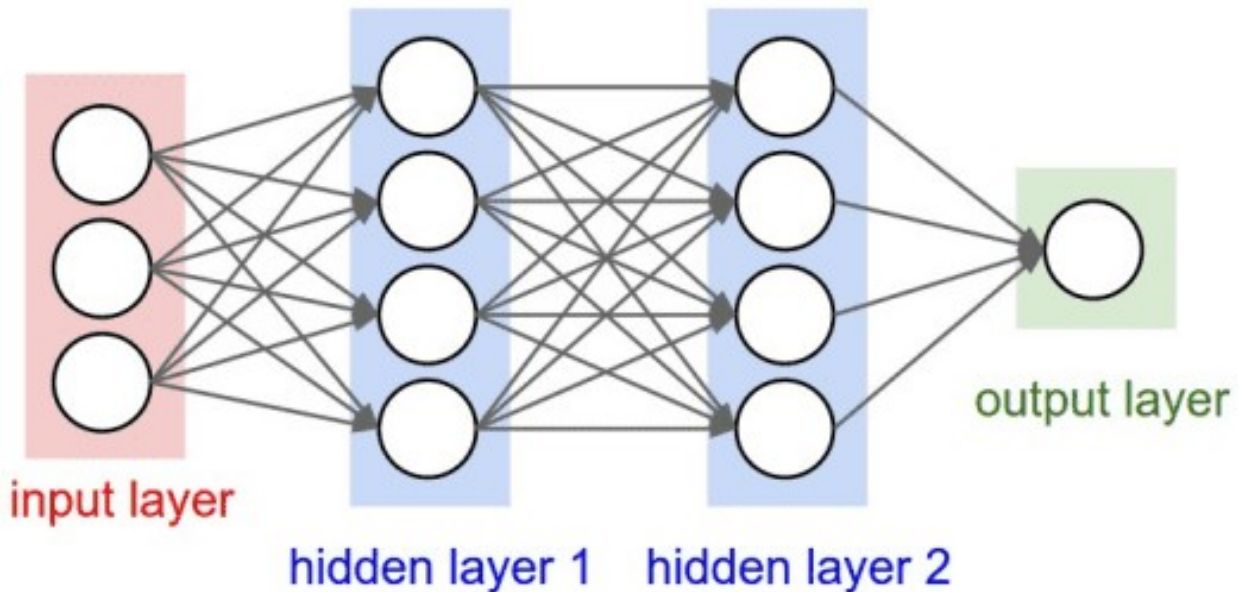
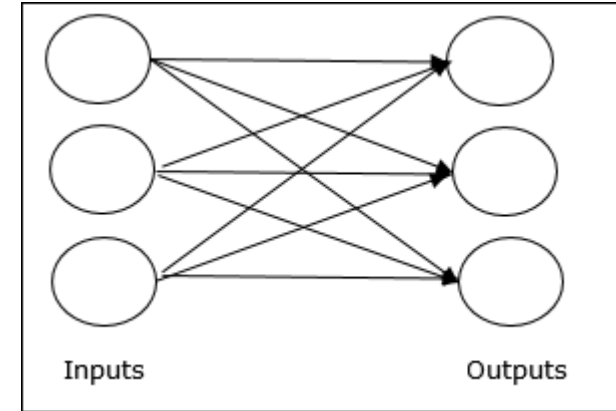
- Bemeneti neuron:
 - Típusukban is különböznek a többi neurontól
 - Egybemenetű
 - Egykimenetű
 - buffer jellegű
 - Jelfeldolgozó, processzáló feladatuk nincs
 - Bemenetük a hálózat bemenete
 - Kimenetük más neuronok meghajtására szolgál
- Kimeneti neuron:
 - Kimenete a környezet felé továbbítja a kívánt információt
 - Típusukra nézve nem feltétlenül különböznek a többi neurontól
- Rejtett (hidden) neuron:
 - Bemeneteikkel és kimenetükkel kizárólag más neuronokhoz kapcsolódnak

- Neuronokat sok esetben rétegekbe (layers) szervezzük
 - egy rétegbe hasonló típusú neuronok tartoznak
 - Ezen neuronoknak a kapcsolataik is hasonlóak
- Az azonos rétegbe tartozó neuronok mindegyikének
 - bemenetei a teljes hálózat bemenetei
 - Egy másik réteg neuronjainak kimeneteihez kapcsolódnak, kimenetei pedig ugyancsak egy másik réteg neuronjainak bemeneteit képezik
 - a teljes hálózat kimeneteit alkotják.
- Ennek megfelelően beszélhetünk
 - bemeneti rétegről (input layer)
 - rejtett réteg(ek)ről (hidden layer(s))
 - kimeneti rétegről (output layer).

- A bemeneti réteg
 - buffer jellegű neuronokból épül fel
 - információfeldolgozást nem végez
 - feladata csupán a háló bemeneteinek a következő réteg bemeneteihez való eljuttatása
- Ennek megfelelően egy rétegekbe szervezett háló legalább két réteggel, egy bemeneti és egy kimeneti réteggel kell rendelkezzen.
- E két réteg között elvben tetszőleges számú rejtett réteg helyezkedhet el.
- A neuronhálókat az egyes neuronok közötti összeköttetési rendszer alapján két fő csoportba sorolhatjuk:
 - előrecsatolt hálózatok (feedforward networks)
 - visszacsatolt hálózatok (recurrent networks).

Előrecsatolt hálózatok

- Működése nem időfüggő
- A jelek áthaladnak a hálón a bemeneti réteg irányából a kimeneti réteg felé
- A rejtett rétegek száma alapján van:
 - Egyrétegű (single layer)- száma 0
 - Többrétegű (multilayer)- száma 1+
- A kimenet függ:
 - A bemenettől
 - A neuronok aktivációs jellemzőitől
 - Súlymátrixtól



Hátracsatolt/Visszacsatolt hálózatok

- Visszacsatoltnak nevezünk egy neurális hálót, ha a topológiáját reprezentáló irányított gráf tartalmaz hurkot, egyébként a neurális háló előrecsatolt
- Visszacsatolás fajtái:
 - Globális: a hálózat kimenetét csatoljuk vissza a bemenetére
 - Lokális
 - Elemei visszacsatolás: ha egy réteg neuronjának kimenete közvetlenül egyik saját bemenetére van visszacsatolva
 - Laterális: valamely réteg(ek) neuronjainak kimenetei ugyanazon réteg neuronjainak bemeneteire kapcsolódnak, de nem értjük ide a neuron önmagára való visszacsatolását
 - Rétegek közötti visszacsatolás: több réteget tartalmazó hurkot hoznak létre a gráfon

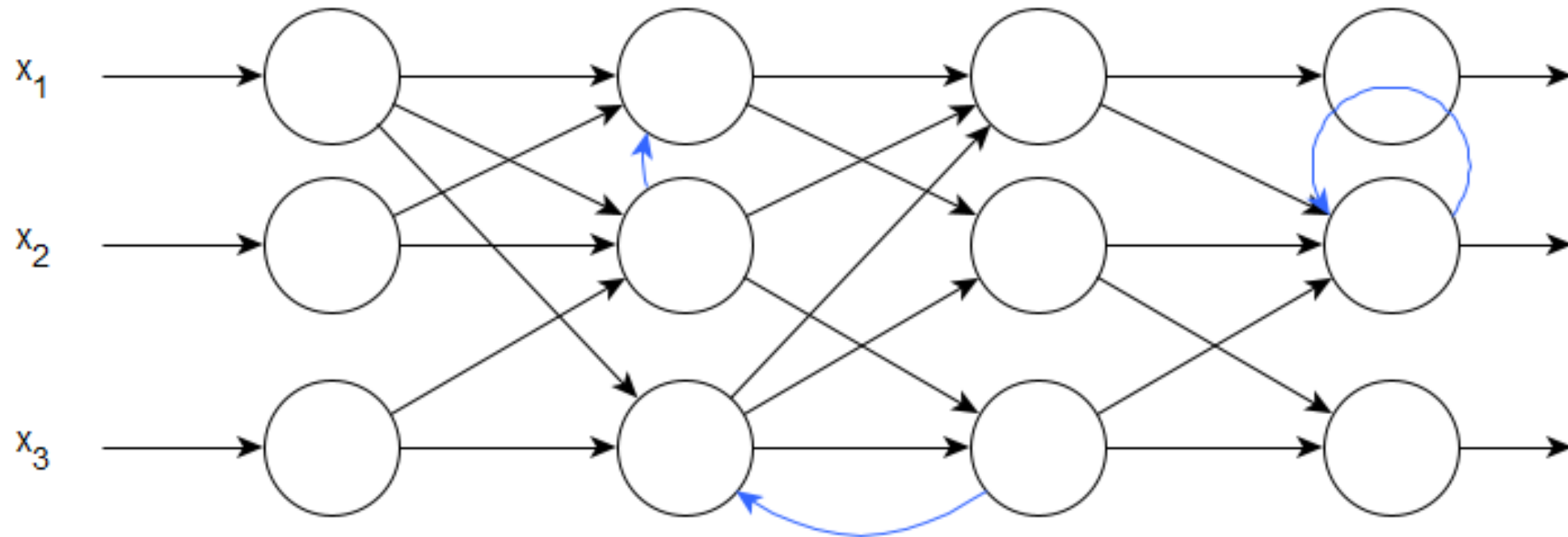
Bemenet

bemeneti réteg

1. rejtett réteg

2. rejtett réteg

Kimeneti réteg



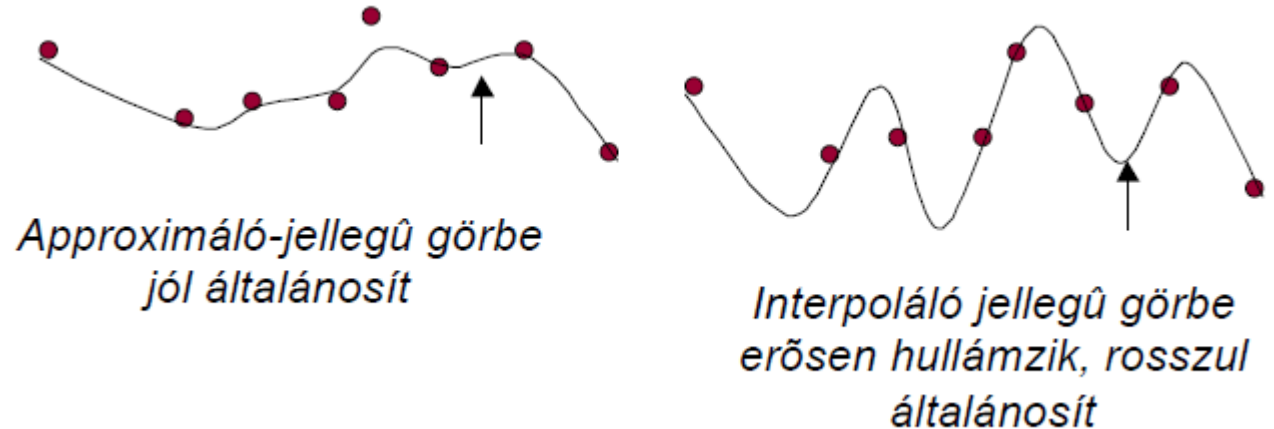
- A bemenet ráadása egy, általában konvergens folyamatot indít el, amelyben a jelek előre felé és visszafelé is haladnak
- A t . időpillanatban számított jelek megjelenhetnek a korábbi rétegek bemenetén a $t+1$. időpillanatban
- A konvergencia végén a jelek értéke állandósul, ekkor leolvashatjuk a kimenő rétegen a kimenet értéket
- A hátracsatolt hálóknak néhány jellemzője:
 - Lehetnek konvergenssek egy stabil állapot felé, vagy instabilak, oszcillálóak
 - A konvergencia, a stabil állapot elérése hosszabb időt igényelhet
 - Nehezebben taníthatóak és bonyolultabb matematikai apparátussal írhatóak le
 - Rendelkezhetnek belső állapottal

Megfeleltetések a biológiai rendszer, a neurális háló matematikai modellje és a gondolkodás tevékenységei között

Biológiai rendszer	Matematikai modell	Gondolkodás
Idegsejtek	Mesterséges neuron	Hipotézis
Kisülés frekvenciája	Aktiválás	Bizonyosság szintje
Depolarizáció szétterjedése	Aktiváltság szétterjedése	A bizonyosság továbbterjesztése: következtetés
Szinaptikus érintkezés	Kapcsolat	Gondolati-következtetési viszonylatok
Gerjesztés/tiltás	Pozitív/negatív súly	Pozitív/negatív következtetési reláció
A depolarizáció közelítő összegződése	Inputok összegzése	A bizonyosság közelítő összegződése
Kisülési küszöb	Aktiválás továbbítási küszöbje	Függetlenség az irreleváns információtól
Korlátozott dinamika tartomány	Szigmoid aktivációs függvény	A feldolgozás erősségének korlátozott tartománya

Hálózati topológia választása

- A hálózati struktúra változtatható paraméterei kihatnak a működésre
- A hálók bemeneti és kimeneti neuronjainak a számát a neuronhálóval megoldani kívánt feladat egyértelműen meghatározza.
- A belső rétegek száma és a bennük lévő neuronok száma azonban szabadabban választható meg.
- Túl kevés neuron kevés minta tárolását teszi lehetővé és megghiúsíthatja a betanulást.
- Túl sok neuron esetén a háló betanul, de a korábban nem látott inputokra a háló rosszul válaszol, adatbázisszerűen működik.



- Egy rejtett réteggel rendelkező hálók bármely folytonos függvény megtanulására képesek
- Két rejtett réteggel rendelkező hálók minden függvényt képesek approximálni, bár ezt az elvi lehetőséget a bemenetek számától exponenciálisan függő belső rétegbeli elemszám rontja.

Hálózat neuronjainak száma

- A hálózat neuronjai számának betanulás során való megállapítására két alapvető módszer létezik:
 1. Optimális agykárosítás modell
 - Teljes kapcsolati hálóval indul
 - A betanítás egyes lépéseiben a közel nulla értékű súlyokat elhagyjuk
 - Ha az így lecsökkent kapcsolatrendszerű háló továbbra is jól szerepel, akkor az elhagyás véglegesítődik, és megismétlődik a művelet
 - A sok kapcsolat kiesése mellett neuronok is kiesnek

2. Hálónövesztő algoritmusok

- A betanítási folyamat közben adnak hozzá újabb neuronokat és kapcsolataikat a hálóhoz
- A hozzáadás függhet:
 - A hibaminimumra törekvő háló esetén a hibafüggvény csökkenésének meredekségétől
 - A csempéző algoritmus esetében a beválasztott új neuronok teljesítményétől
- Az algoritmus azzal a neuronnal indít, amelyre a legtöbb minta a legtöbb kimenetet adja
- A következő hozzáadandó neuront hasonlóan választja
- Addig folytatja, míg az összes mintára kielégítő nem lesz a válasz

McCulloch – Pitts-neuron (MP neuron)

- Mesterséges neuron első számítási modellje (1943)
- Az akkori neurobiológiai eredményeken alapul, melynek előfeltevései:
 - a neuron "minden - vagy semmi" alapon működik
 - bizonyos adott számú szinapszishoz kell lennie gerjesztve az összegzési időben azért, hogy a neuron egyáltalán kisüljön és ez a szám független a neuron megelőző aktivitásától és állapotától
 - az egyetlen jelentős késleltetés az idegrendszerben a szinaptikus késleltetés
 - bármely tiltó szinapszis aktivitása egyedül elegendő a neuron gerjesztésének megakadályozásához
 - a hálózat struktúrája időben nem változik
- A cél az ideghálózat olyan matematikai modelljének létrehozása volt, amely biológiailag pontos és magasszintű kognitív képességeket mutat.

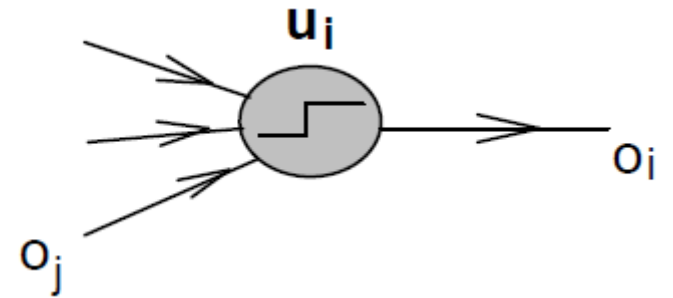
- › A neuron formális matematikai modellje:

$$a_i(t) = \sum_{j=1}^n o_j(t)$$
$$o_i(t+1) = \sigma(a_i(t))$$

- Ahol $o_j(t)$ az u_i neuron j . bemenete, bináris érték
- az $o_i(t)$ az u_i neuron kimenete a t . időciklusban, bináris érték
- az $a_i(t)$ az aktivációs potenciál az u_i neuronban a t . időciklusban
- $\sigma(a)$ pedig a Θ küszöbvel jellemezhető egységugrás függvény

$$H(a) = \begin{cases} 1, & \text{ha } a \geq \Theta \\ 0 & \text{egyébként} \end{cases}$$

- (Mintha minden bemenetnek 1 lenne a súlya)



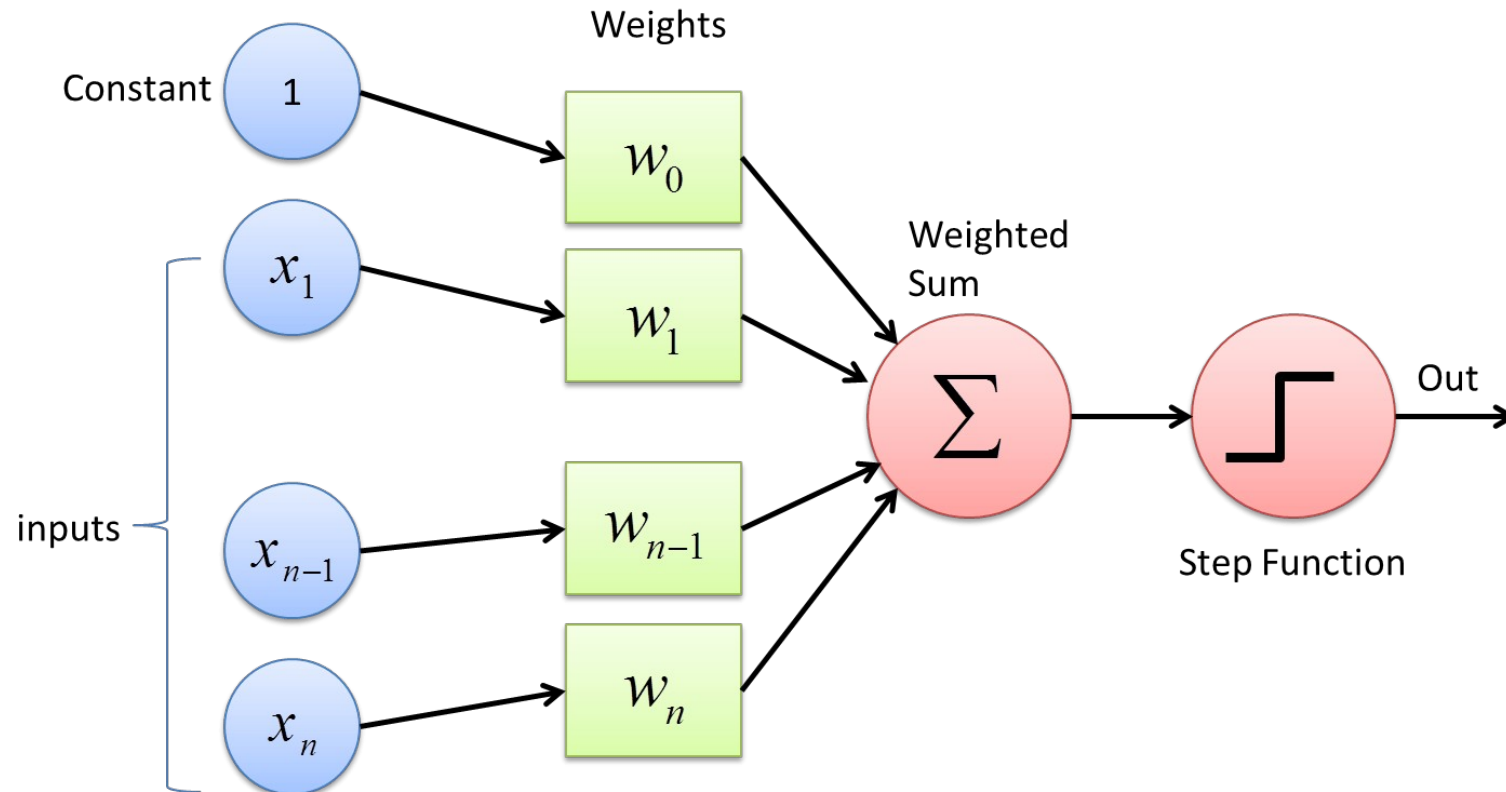
Rosenblatt perceptron

- Az első, és legegyszerűbb mesterséges neurális hálózat
- Képes arra megfelelő beállítás/betanulás után, hogy két lineárisan szeparálható bemeneti mintahalmazt szétválasszon
 - Kétszélyos esetekben lineáris osztályozási feladatok ellátására alkalmas
- A későbbiekben továbbfejlesztették több-elemű, illetve többrétegű hálózatokká, amelyek képességei az egyszerű perceptront messze felülmúlják
- Lineáris kombinációt megvalósító hálózat, amelynek a kiementén egy küszöbfüggvény-nemlinearitás szerepel, vagyis valójában egyetlen memória nélküli processzáló elem

$$a_i(t) = \sum_{j=1}^n w_j(t) x_j(t) - \Theta$$

$$o_i(t+1) = \sigma(a_i(t))$$

- Több helyen a $-\Theta$ helyett $+$ bias érték szerepel, tehát a j kezdőindexe 0



Perceptron jellemzők

- A látást, a retinát modellezte
- A retina-modell mátrix alakban elrendezett fényszenzorai képezték az inputot
- A szenzorkimeneteket mesterséges neuronok szoportjába vezették, mely csoport elemi különböző mintákat ismertek fel
- Az output réteg neuronja addig tüzel, amíg egy adott gerjesztési szint, azaz egy adott típusú inputmintázat nem jelentkezik
- Azon a megfigyelésen alapszik, hogy a hasonló összetettségű alakzatok közül egyesek könnyebben megtanulhatók és felidézhetők, mint mások
- Működik folyamatos értékű és bináris inputtal egyaránt

Perceptron tanulása

- A tanítás során a megfelelő elválasztó felület meghatározása a cél, vagyis azon w súlyvektor meghatározása, kialakítása, amely mellett az egyik osztálybeli bemeneti adatokra a lineáris kombináció eredménye pozitív, míg a másik osztálybeli pontokra negatív.
- Az elválasztó felület a bemeneti mintatérben egy, az origón átmenő, a súlyvektorra (w) merőleges sík (hipersík)
 - Ha nem választhatóak szét lineáris felülettel, akkor semmilyen tanító eljárással sem kaphatunk olyan hálózatot, amely az osztályozási feladatot tökéletesen megoldja

Tanulás lépései

1. Inicializáljuk a w_{ij} súlyokat és a Θ küszöböt kicsi véletlen értékekre
2. Adjuk meg t . inputot és a hozzátartozó elvárt outputot:
 $x_1(t), x_2(t), x_3(t), \dots, x_n(t)$, valamint $d(t)$
3. Számítsuk ki az inputhoz tartozó kimenetet:

$$o(t) = \sigma\left(\sum_{j=1}^n w_j(t) * x_j(t) - \Theta\right)$$

4. Módosítsuk a súlyokat:

- Ha $d(t) = o(t)$, akkor $w_j(t+1) = w_j(t)$, tehát változatlan
- Ha $o(t) = 0$, de $d(t)=1$, akkor $w_j(t+1) = w_j(t)+x_j(t)$
- Ha $o(t) = 1$, de $d(t)=0$, akkor $w_j(t+1) = w_j(t)-x_j(t)$

Ha módosultak a súlyok a 4. lépésben, akkor ugrás a 2. pontra, különben kész a betanítás

- Más lehetséges leállási feltétel:

- $\frac{1}{s} \sum_{j=1}^s |d_j(t) - o_j(t)| < \gamma$, ahol γ a hiba küszöbértéke, s pedig a tanítóminta mérete
- Adott számú iteráció elvégzésre került

Widrow-Hoff delta szabály a perceptron vonatkozásában

- A perceptron tanító algoritmus módosítását javasolták
- Errorként, hibaként értelmezték az elvárt kimenet és a számított kimenet eltérését és ennek nagyságával arányos súlymódosítást javasoltak

- Módosítsuk úgy a súlyokat:

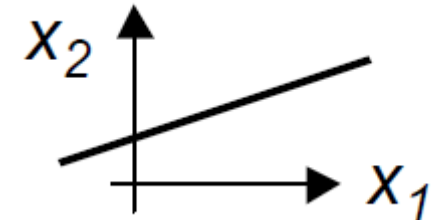
$$w_j(t + 1) = w_j(t) + r * (d(t) - o(t)) * x_j(t)$$

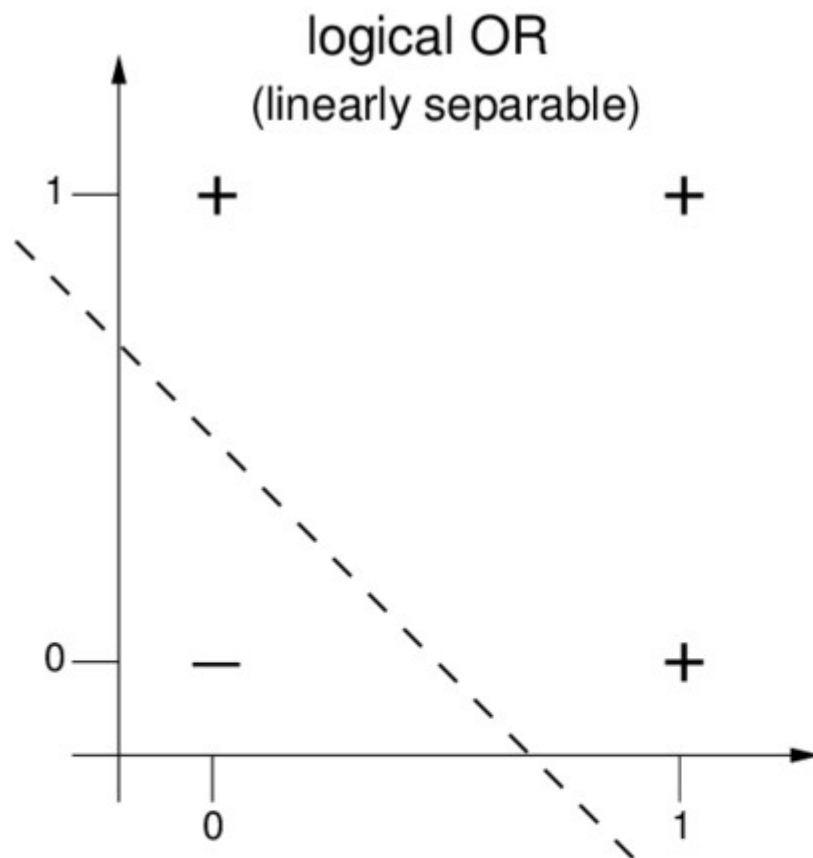
Ahol r a tanulási együttható ($0 < r < 1$)

- Az r tanulási együttható szabályozza a konvergencia fokát
 - Célszerű menet közben értéket változtatni, hogy eleget tudjon tenni az input eloszlásában meglévő nagy változásokhoz való gyors alkalmazkodásnak és az algoritmus végén a stabil súlyállapot eléréséhez szükséges input-átlagoásnak

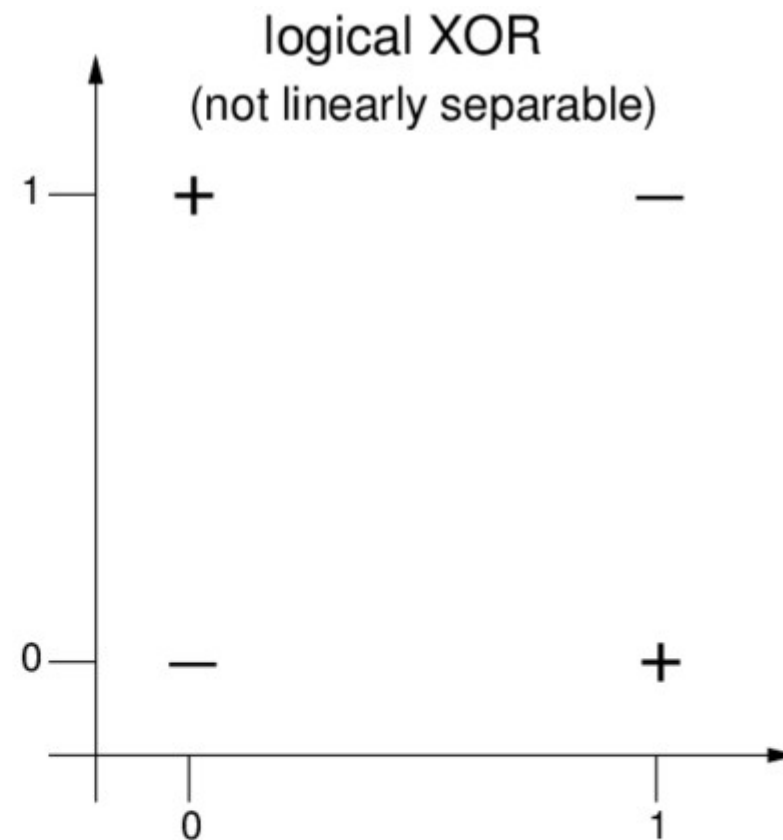
Perceptron hibája

- Csak lineárisan szétválasztható minták osztályozására képes
- Marvin Minsky és Seymour Papert 1969-ben jelentette meg a Perceptrons című könyvet, melyben elemzik a Perceptron adatrepresentációs képességeit.
- Lineáris függetlenség
 - A vizsgálathoz vesszük a perceptron kétbemenetes esetét
 - $w_1(t) * x_1(t) + w_2(t) * x_2(t) - \Theta = 0$, melyet átrendezve egy egyenes adódik
 - $x_2(t) = \frac{-w_1(t)}{w_2(t)} x_1(t) + \frac{\Theta}{w_2(t)}$





Input ₁	Input ₂	Output
1	1	1
1	0	1
0	1	1
0	0	0



Input ₁	Input ₂	Output
1	1	0
1	0	1
0	1	1
0	0	0

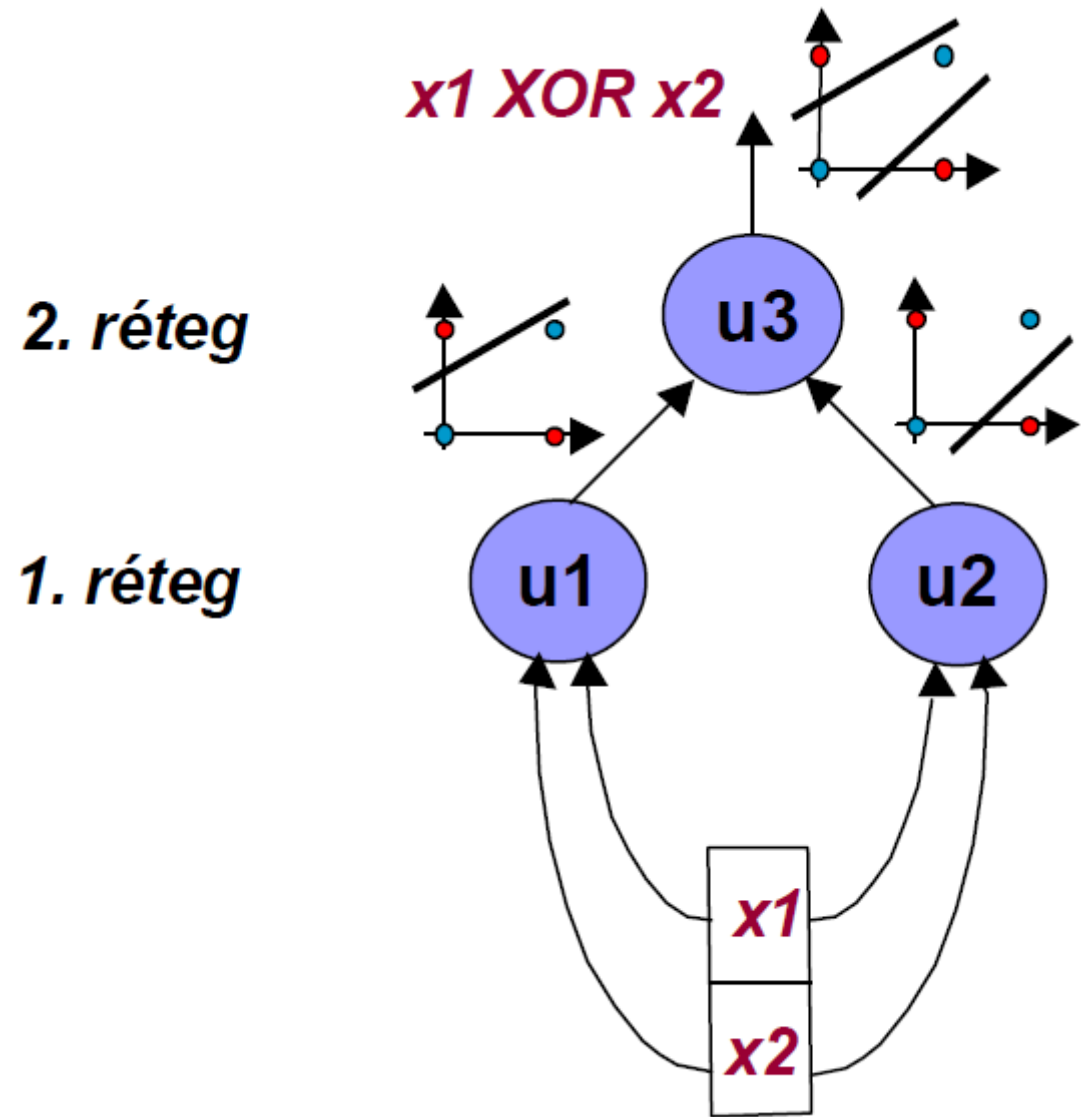
Adaline (Adaptive Linear Neuron)

- Lényegében ugyanaz a felépítése, mint a perceptronnak
 - Alapvető különbség a hiba értelmezésében és a háló tanításában van
 - Míg a perceptronnál a hálózat tényleges kimenetét hasonlítjuk össze a kívánt válasszal, addig az adaline-nál a lineáris kombináció eredményét, tehát a lineáris hálózatrész kimenetét vetjük össze egy kívánt jellel
 - A hálózat tényleges kimenete azonban itt is egy küszöbfüggvény nemlinearitás után kapható
- Mivel a hiba szempontjából a lineáris kimenetet tekintjük az adaline kimenetének, a kimenet a paraméterek lineáris függvénye és négyzetes hiba esetén a kritériumfüggvény kvadratikusan jelenti a felületet

Többrétegű perceptron (Multilayer Perceptron MLP)

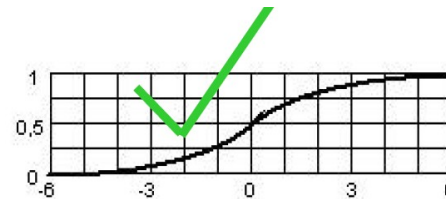
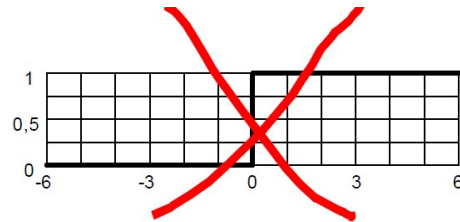
- Teljesen összekötött (fully connected) neurális háló, amely bővebb értelemben bármely előrecsatolt neurális hálót jelent, szűkebb értelemben több rétegbe szervezett perceptronokat (küszöb aktivációs függvényel)
- Célja a lineárisan szeparálhatatlan inputok megkülönböztethetőségének elérése
- Megoldása:
 - Perceptron kettő, vagy több rétegben
- Az első réteg neuronjai alkalmasak az inputminta kisebb, egyenessel elszeparálható részeinek megkülönböztetésére, a második réteg neuronjai pedig alkalmasak az első réteg neuronjaitól jövő kétféle osztályba tartozó jelek megkülönböztetésére

- Az u_1 neuron észleli a $(0,1)$ bemenetet, az u_2 pedig az $(1,0)$ bemenet esetén ad 1-et.
- Az u_3 neuronnak csak egy vagy függvényt kell realizálnia
- Jelen formában a háló nem képes tanulni, a súlyok csak megadással állíthatóak be



Többrétegű perceptron tanulási hibája

- Oka: a lépcsős átviteli függvény
- Emiatt a második rétegre nem jut el információ a bemenetek tényleges értékéről
- Mivel a tanulás megfelel az aktív inputok és az aktív neuronok közötti kapcsolat erősítésének, lehetetlen a háló megfelelő részét erősíteni, mivel az aktuális inputok el vannak vágva az outputtól a közbenső réteg által
- A kétállapotú neuron, lévén tüzel vagy nem tüzel állapotban, nem ad semmi jelzést a súlyok módosításának mértékéről
- A küszöbhez közeli, a neuront éppen csak tüzelésre készítő inputok súlyait nem kellene ugyanolyan mértékben módosítani, mint azokat, amelyek hatására a neuron aktiváltsága messze esik a küszöbtől
- Megoldása: A küszöb közelében átmenettel bíró átviteli függvényt alkalmazzunk
- Tanítására új tanulási szabályok: általánosított delta szabály vagy hibavisszaterjesztés (backpropagation)



Neurális hálók betanítása

- A neurális hálók betanítása szokás szerint nagymennyiségű adat formájában adott betanító minták alapján történik. A neurális hálók ereje a modell-mentes közelítő képességükben rejlik:
- Megadva egy véges méretű, időfüggetlen determinisztikus rendszert, melyből az input-output mintapárok származtathatók, mindig létezik egy mesterséges neurális háló, mely be tudja állítani úgy a saját paramétereit, hogy utánózni képes a rendszert input-output szempontból.

(Rumelhart és McClelland, 1986)

Neurális hálók tanulási módszerei

- A tanulást úgy definiáljuk, mint a hálózat memóriájának, azaz a súlymátrixnak a változását
- Fajtái:
 - Felügyelt tanulás (supervised learning)
 - Felügyelet nélküli tanulás (unsupervised learning)
- Felügyelt tanulás: külső tanárt, ill. globális információt igényel, fő fajtái:
 - Hiba-javító tanulás (error correction)
 - Megerősítő tanulás (reinforcement learning)
 - Sztochasztikus tanulás (stochastic learning)

Felügyelt tanulás

- Adott egy rendszer, mely a bemenetei és a kimenetei között valamilyen $g()$ leképezést valósít meg
- A leképezés nem ismert, viszont rendelkezésünkre áll egy $\{z_i\} = \{x_i, d_i\}$ ($1 \leq i \leq l$) mintapont-készlet, melynél az összetartozó bemeneti és kimeneti értékek közötti kapcsolatot az ismeretlen leképezés adja meg
- A mintakészlet elemei, a mintapontok általában többdimenziós vektorok, melyek a mintatérben helyezkednek el
- A tanulási feladat az, hogy egy $f(\mathbf{x}, \mathbf{w})$ paraméteres függvényhalmazból válasszunk egy olyan függvényt olyan módon, hogy az x_i bemenetre a kiválasztott függvény $y_i = f(x_i, \mathbf{w})$ válasza valamilyen értelemben a lehető legjobban közelítse az ismeretlen leképezés azonos bemenetre adott d_i válaszát (desired response), ahol $\mathbf{w} \in W$ a függvényhalmaz paramétere és W a paramétertér
- A függvényhalmaz szabad paramétereit tartalmazó $\mathbf{w}$ vektort a W paramétertér megfelelő pontjaként a rendelkezésre álló mintapontok alapján kell megválasztani vagy meghatározni

- A $\{x_i, d_i\}$ ($1 \leq i \leq l$) mintapont-készletet tanító készletnek vagy tanító mintakészletnek (training set) nevezzük
- Az eljárást, mely során a tanító mintakészlet felhasználásával az $f(\mathbf{x}, \mathbf{w})$ leképezést kialakítjuk, tanulásnak (learning) nevezzük
- A tanulási eljárás célja tehát egy valamilyen értelemben optimális $\mathbf{w}^*$ paraméter meghatározása a tanító mintakészlet alapján
- Tipikus feladatai az osztályozás és a regresszió
- Amennyiben az $f(\mathbf{x}, \mathbf{w})$ leképezést egy neuronháló valósítja meg, a függvénykészlet megválasztása a neuronháló típusának és architektúrájának megválasztásával történik, míg a tanítás valójában a $\mathbf{w}$ paramétervektor meghatározását, becslését szolgálja
 - Ebben az értelemben a tanulás egy paraméterbecslési eljárás

Minősítés, hibamérték

- A legjobb approximációt biztosító függvény meghatározásához az approximáció minőségét jellemző mértékre van szükség
- Ez a mérték a hibafüggvény (error function) felhasználásával határozható meg, amit költségfüggvénynek (cost function) vagy veszteségfüggvénynek (loss function) is szokás nevezni
- A veszteségfüggvény, továbbiakban $L(d, f(\mathbf{x}, \mathbf{w}))$, mind a háló válaszának, $f(\mathbf{x}, \mathbf{w})$ -nek, mind a rendelkezésre álló $d(\mathbf{x})$ kívánt válasznak a függvénye
- Leggyakrabban az approximáció $\varepsilon(\mathbf{x}, \mathbf{w})$ hibájának a függvénye, ahol a hibát a kívánt válasz és a tényleges válasz különbségeként definiálhatjuk
 - $\varepsilon(\mathbf{x}, \mathbf{w}) = d(\mathbf{x}) - f(\mathbf{x}, \mathbf{w})$

- A tanuló rendszer konstrukciója- akár osztályozási, akár regressziós probléma megoldására irányul- véges számú tanító mintapont alapján történik
- A tanuló eljárás célja azonban általában nem az, hogy a véges számú tanítópontban kapjunk megfelelő válaszokat, hanem egy olyan leképezés megtanulása, amely a tanítópontok által reprezentált bemenet-kimenet kapcsolatot is megadja
- A tanuló rendszer tehát általánosítóképességgel kell rendelkezzen:
 - Olyan esetekben is jó választ kell adjon, mely esetek nem szerepelnek a tanítópontok között
 - Ez adja a tanulás nehézségét
 - Véges számú tanítópont alapján kell egy problémáról „általános tudást” szerezni

Felügyelt tanulási módszerek

- hiba-javító tanulás (error correction)
 - Rendelkezésre állnak mintapárok, összetartozó be- és kimeneti értékek
 - A háló feladata megtanulni a leképezést
 - A hálózat válasza mindig összehasonlítható a kívánt válasszal
 - Célja, hogy hálózat tényleges viselkedése és a kívánt viselkedés közötti eltérés csökkenjen
 - Általában iteratív eljárással, a környezetből származó ismeret fokozatos felhasználásával, a tanítópontok egyenkénti vagy esetenként többszöri, ismételt felhasználásával történik
 - Az output réteg egyes neuronjainál vizsgált - az elvárt és a valós értékek különbözeteként adódó - értékek alapján a súlymátrix kapcsolatait módosítja
- megerősítő tanulás (reinforcement learning):
 - Nem áll rendelkezésre az elvárt kimenet
 - Csak arról van információ, hogy az adott bemenetre a hálózat értéke helyes vagy hibás
 - A módosítás mértékének meghatározására ez nem elegendő
 - Megfelelően végrehajtott akcióknál a súlyok megerősödnek, egyébként gyöngülnek, az akció jóságát az outputvektorból képezett skalár mutatja
 - Hasznos, ha egy bemenetre nem csak egy jó választ adhat+

- sztochasztikus tanulás (stochastic learning):
 - a súlymátrix random változtatása után meghatározza a hálózat ún. energiaértékét.
 - Ha a változás hatására az energia csökkent, akkor a változást elfogadja, egyébként pedig csak akkor, ha a változás megfelel egy előreválasztott valószínűségi eloszlásnak.
 - Ez a véletlen elfogadása a változásnak, mely időszakosan ronthatja a rendszer működését, jóságát, lehetővé teszi, hogy kikerüljön a lokális energiaminimum völgyekből, miközben az optimális állapotot keresi.

Felügyelet nélküli tanulás

- Nem igényel külső tanítót, lokális információ és belső kontrollstratégián alapszik
- Címkezetlen tanítópontjaink vannak, így egy előírt viselkedés megtanulása fel sem merül
- A hálózatnak a bemenetei és a kimenetei alapján, de kívánt válaszok ismerete nélkül kell valamilyen viselkedést kialakítani, a környezetből azonban nincs közvetlen visszajelzés, ami a hálózat viselkedésének a helyességére utalna
- Vizsgáljuk, hogy a mintapontok terében vannak-e olyan tartományok, ahol a mintáink sűrűsödnek, lehet-e a mintatérben csoportokat, klasztereket találni
- Azt próbálja felderíteni, hogy az adataink egyáltalán hordoznak-e valamilyen információt
 - Ha igen, akkor azt valamilyen módon meg akarjuk ragadni, ki akarjuk nyerni az adatokból
- Tipikus feladat a sűrűségfüggvénybecslés és a klaszterezés

- Feladatként megfogalmazható az is, hogy sokdimenziós térben vannak-e kitüntetett „irányok”, melyek az adatok reprezentálásánál elsődlegesen fontosak, míg más irányok szerepe kisebb, esetleg elhanyagolható
- Az ilyen típusú feladatok az adatok hatékony közelítő reprezentációjára is irányulhatnak
- A felügyelet nélküli tanulási eljárásokat sokszor adatelőkészítésre, az adatok előfeldolgozására használjuk, mielőtt egy felügyelt tanulással megoldható feladatot oldanánk meg
 - Ilyenkor a felügyelet nélküli tanulás bizonyos, a későbbi feldolgozást elősegítő lényeges információk kiemelésére, lényegkiemelésre, jellemzőkinyerésre (feature extraction) szolgál

Kritériumfüggvények

- A felügyelet nélküli tanulás során is megfogalmazhatóak kritériumfüggvények
- A felügyelet nélküli tanulás is interpretálható olyan feladatként (pl. szélsőérték-keresési feladatként), ahol valamilyen kritérium kielégítése a cél
- Leggyakoribbak:
 - az adatok eloszlásával kapcsolatos kritérium,
 - az adatok egymáshoz viszonyított elhelyezkedésével kapcsolatos kritérium,
 - az adatkomponensek közötti korrelációval kapcsolatos kritérium,
 - de akár négyzetes kritériumfüggvények
- Sokszor információelméleti fogalmakhoz kapcsolódnak:
 - Entrópia (olyan adatrepresentációt keresünk, ahol az entrópia valamilyen extrémum értéket vesz fel)
 - Kölcsönös információ (az adatok statisztikai függetlenségét szeretnénk biztosítani)
 - Kullback-Leibler divergencia (olyan tanuló eljárásnál, ahol két sűrűségfüggvény hasonlósága alapján történik a tanulás)

Tanulási alapelvek

- Hebb-féle szabály (1949):
 - Felügyelet nélküli tanulás
 - az egyidejűleg izgalomban lévő neuronok kapcsolata erősödik
 - x_i a neuron i -dik bemenete, amely a megelőző neuron kimenete, o a számított kimenet
 - $\Delta w_i = \eta * x_i * o$, ahol η a tanulási együttható, a tanulás sebességét befolyásolja
- Delta-szabály (Bernard Widrow és Ted Hoff, 1960):
 - Felügyelt tanulás
 - A tanítási folyamat során az output és a megkívánt output (t) közötti eltérés esetén a súlyokat olyan irányba kell módosítani, hogy csökkenjen az eltérés.
 - $\Delta w_i = \eta * x_i * (t - o)$, ahol η a tanulási együttható, a tanulás sebességét befolyásolja

Gradiens-alapú csökkentés módszer (gradient descent)

- Véletlenszerű súlyértékekből indul ki
- A súlyokat apró lépéseket ismételve változtatja
- A lépés mérete fontos paraméter lesz („learning rate”)
- Az optimális lépés irányát a hibafüggvény deriváltja alapján tippeli meg
- De ez csak tipp, nem garantált, hogy a hiba minden lépésben csökkenni fog, illetve az eljárás előbb-utóbb elakad

Derivált számítása egyetlen neuron esetén

- Egyetlen (sigmoid aktivációs) neuron MSE hibafüggvénye:

$$E(\mathbf{w}) = \frac{1}{N} \sum_{d \in D} (t_d - o_d)^2 \quad \text{- hibafüggvény}$$

$$o_d = 1 / (1 + e^{-a_d}) \quad \text{- aktivációs függvény}$$

$$a_d = \sum_i w_i x_i \quad \text{- lineáris aktiváció}$$

- Ezeket egymásba ágyazva kapjuk meg, hogyan függ $E(\mathbf{w})$ egy konkrét w_j -től
- Deriválás: ezek deriváltjai láncszabállyal összerakva

$$\frac{\partial E}{\partial w_i} = \frac{1}{N} \sum_{d \in D} 2(t_d - o_d) o_d (1 - o_d) x_{id}$$

- Súlyok frissítése minden lépésben:
 - Ahol η egy kis pozitív konstans („learn rate”)

$$w_i = w_i - \eta \frac{\partial E}{\partial w_i}$$

Back-propagation/visszafelé terjesztés

- A delta szabály variációja.
- A többrétegű előrecsatolt háló súlyait oly módon állítja be, hogy a számított kimenet és az elvárt kimenet eltérését felhasználva a kapcsolati súlyokat olyan irányba módosítja kismértékben, hogy az input következő bemutatásakor az eltérés csökkenjen.
- Több kimenő neuron, többrétegű hálózat esetén a MSE hiba

$$E(\mathbf{w}) = \frac{1}{2N} \sum_{k \in \text{test samples}} \sum_{d \in \text{targets}} (t_{kd} - o_{kd})^2$$

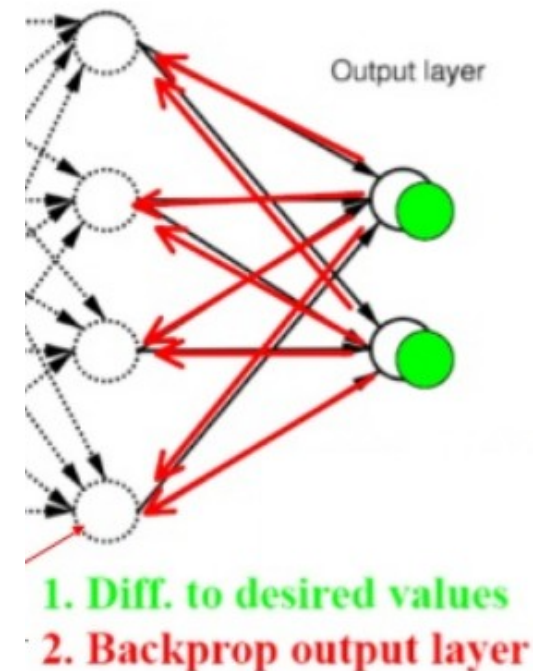
- A deriváláskor ugyanúgy a láncszabályt kell alkalmazni, de több lépésen keresztül, a deriváltat rétegről rétegre visszafelé „propagálva” (a felső neuronok bemenete nem x , hanem az alatta levő réteg kimenete))

- A deriváltat először levezzetjük a kimeneti rétegre
- Majd rétegről rétegre terjesztjük vissza az alsóbb rétegekre

- a k . kimenőegység hibája: $\delta_k = o_k(1 - o_k)(t_k - o_k)$
- a h . rejtett egység hibája (rétegenként visszafelé):

$$\delta_h = o_h(1 - o_h) \sum_{k \in \text{Uses}(h)} w_{kh} \delta_k$$

- Adott rejtett neuron hibája az ő kimenetét felhasználó neuronok hibájának súlyozott összegével arányos
 - Ahol a súlyok megegyeznek az adott kapcsolat súlyával



Back propagation

- Tehát fő lépései:
 - Az inputtól az output felé haladva kiszámoljuk az egyes neuronok kimeneteit a tanítópéldákon keresztül
 - A hibafüggvényhez kell tudnunk a kimeneteket
 - Az outputtól az input felé haladva kiszámoljuk az egyes neuronok hibáit
 - Frissítjük a súlyokat: $w_{ji} = w_{ji} + \eta \delta_j x_{ji}$
- A hiba-visszaterjesztés megengedi a ritka hálózatot, vagy a rétegek közötti ugrást tartalmazó hálózatot is.
- Az egyetlen lényeges megkötés a hálózati struktúrára, hogy ne legyen visszacsatolás (kör)
 - Ha nincs, akkor topológiai rendezés sorrendjében lehet frissíteni
 - Ha van, akkor visszacsatolt hálózatot kapunk, ezek tanítása jóval komplikáltabb

Neurális háló modellek jellemzői

- Adaptivitás: változó feltételeknek megfelelő válaszok adásának képessége. Részei:
 - Tanulás: alapvetően egyidőben jelentkező dolgok társítása.
 - Önszervezés: amikor a háló módosítja a neuronjait egy tanulási szabálynak megfelelően. Rendszerint az inputokra adott szinaptikus súlyváltozásokat jelenti.
 - Általánosítás: az a képesség, hogy korábban soha nem hallott kérdést is meg tud válaszolni a hasonló, vagy kapcsolódó információk alapján.
 - Gyakorlás: a tanultak elmélyítése.
- Rugalmasság: egy neuroncsoport azon képessége, hogy időben különféle igényekhez tudjon alkalmazkodni. Ha egyes neuronok sérülnek, mások átveszik a szerepüket.
- Dinamikus stabilitás: a hálózat azon képessége, hogy megmaradjon a működési keretei között és elérjen egy stabil állapotot, extrém input mellett is.
- Konvergencia: stabil állapotba való jutás képessége.
- Hibatűrés: az a képesség, hogy néhány neuron sérülése esetén is tovább tudjon működni, habár csökkent pontossággal, vagy sebességgel.
- Normalizálás: a súlyoknak egy előírt tartományon belül való tartása.

Neurális hálót definiáló összetevők

- hálózati topológia
- a háló mérete (a rétegek száma, a rétegenkénti neuronok száma)
- a működési jellemzők (a neuron input operátora, átviteli függvénye és a diszkrimináló függvénye)
- a tanulási eljárás (tanulási algoritmus, tanulási paraméterek, soros/batch aktualizálás, stb.)
- betanítás/érvényesítés (a betanítás mérete, minősége, adatformátuma, stb.)
- implementálás/realizálás (analóg, diszkrét, szoftveres, stb.)

Tervezési szempontok

- Biológiai rendszert akarunk emulálni ? Szükséges-e utánozni a neuron kisülési függvényét (aktivációs függvényét) ?
- Milyenek a sebességgel kapcsolatos elvárások ? Végbe tud-e menni valós időben a tanulás és a döntéshozatal ?
- Szükséges-e, hogy a neuron működési jellemzői invertálható, monoton, vagy folytonos tulajdonságúak legyenek ?
- Milyen természetű kapcsolati súlyokat alkalmazunk ? Binárisak, folytonos, vagy diszkrét értékűek lesznek-e ?
- Milyen típusú kimenetre van szükségünk ?

Előrecsatolt neurális hálókat minősítő jellemzők

- Kifejezőképesség (expressiveness): függvények reprezentálására ideálisak, de az optimális neurószám eltalálása művészet, nagy gyakorlatot kíván.
- Számítási hatékonyság (computational efficiency): valójában egy adott mintafájl betanulásának számításigénye. Kedvezőtlen esetben a bemenetek számának exponenciális függvénye lehet. Problémát jelentenek a hibafelület lokális minimumai, melyek ellen alkalmazható a szimulált hűtés módszere.
- Általánosító képesség (generalization): egyes függvénycsaládok megtanulására és interpolálására alkalmasabbak. Ezek azok, amelyeknél a bemenetek függetlenek egymástól, nincsenek összetett függőségek. Jó, ha a kimenet függvénye sima felület.
- Zajérzékenység (sensitivity to noise): zajtűrésük annak köszönhető, hogy nemlineáris regresszióval írják le az outputfüggvényt.
- Átláthatóság (transparency): nem képes magyarázatadásra, döntésindoklásra, ezért megnő a szerepe a tesztadatokkal való helyességellenőrzésnek. Gyakorlatilag azonban egy fekete doboz.
- Megelőző ismeret (prior knowledge): Kellő gyakorlat esetén a problémára vonatkozó előzetes ismeret segíthet a háló eredményes topológiájának kialakításában. Inkább heurisztikák ezek, mint matematikailag alátámasztott módszerek.

Neurális háló teljesítményének fejlesztői szempontjai

- **Képesség.** Milyen feladatot tud a háló elvégezni? Rendelkezésre áll-e elegendő adat a probléma megoldásának előállításához? Rendelkezik-e a háló a szükséges számítási teljesítménnyel a feladat megoldásához?
- **Kapacitás.** Mennyi ismeret tárolható a hálóban?
- **Tanulékonyság.** Milyen típusú problémákat képes megtanulni? Újra lehete a hálót tervezni, vagy az ismeretszemléltetést megváltoztatni abból a célból, hogy a feladat megtanulható legyen?
- **Ismeretszemléltetés.** Hogyan tárolja a háló az ismeretet? Hogyan hat ki az ismeret szemléltetési módja a szerkezetre, kapacitásra, képességre, tanulékonyásra?
- **Ismeretkinyerés.** Hogyan használja az ismeretet a világ modelljének előállítására? Megérthető módon van-e tárolva az ismeret? Hogyan hat a belső ismeretszemléltetés a tudáskinyerésre?
- **A háló megtervezése.** Hogyan kellene megtervezni a hálót, hogy specifikus műveleteket tudjon végrehajtani? Milyen topológia, neuronműködés szükséges?
- **Kredit hozzárendelés.** Adott a változó súlyok egy létező halmaza, valamint a betanító adatok, hogyan rendeljünk értéket minden egyes súlyhoz a betanítás időszaka alatt a korrekt válasz elérése érdekében?

A mesterséges neurális hálók mechanizmusai

- versenyző (kompetetiv) mechanizmus :
 - az azonos rétegben lévő neuronok versenyeznek egymással lecsökkentve egymás aktivációs szintjét, hogy csak egy nyerjen
- együttműködési (kooperativ) mechanizmus:
 - az azonos rétegben lévő neuronok erősítik egymás aktivációs szintjét, hogy együtt nyerjenek
- normalizációs mechanizmus, vagy skálázás:
 - a háló az azonos csoportba tartozó neuronok súlyait úgy próbálja beállítani, hogy azok aktivációs szintje soha ne telítődjön, ill. soha ne tüzeljenek.

Betanítésnél figyelembe veendő szempontok

- Hatékonyság. Mekkora betanító adattömeg szükséges a működési teljesítmény egy adott szintjének eléréséhez ?
- A tanulás hatékonysága. Mennyi időt vesz igénybe a betanítás a kívánt súlyok eléréséhez?
- Komplexitás. Hogyan változik a betanítás időszükséglete, ha a feladat szintje növekszik ?
- Általánosítás. Mikor végzi jól egy osztályozó háló az általánosítást ?
- Alakíthatóság. Lehet-e új elemeket, osztályokat, vagy viselkedést adni a hálózathoz anélkül, hogy lerontanánk az aktuális teljesítményt ?
- Érzékenység. Érzéketlen-e a háló a lényegeset nem érintő inputváltozásokra ? Hogyan érhető el ez az érzéketlenség ?
- Alkalmazkodóképesség. Hogyan befolyásolja a belső ismeretszemléltetési mód a betanítási időt ?

Neurális háló teljesítményének összetevői

- A különféle neurális háló elméletek kiértékelésére és összehasonlítására nem létezik tiszta mérőszám.
- A következő összetevőket kell vizsgálni egy teljesítményösszevetésnél:
 - Hullámszélesség. Mennyi időbe telik, amíg az output megnyugszik és eléri a kívánt megoldást?
 - Pontosság. Milyen pontos a megoldás ?
 - Méret. Hogyan függ a hálózat teljesítménye és kapacitása a háló méretétől ?
 - Robosztusság. Mennyire érzékeny a megoldás a komponensek pontatlanságaira ?

- Hibatűrés. Mennyire viseli el a háló az egyes komponensek meghibásodását ?
- Hordozhatóság. Mennyire könnyen telepíthető a háló különféle hardverekre ?
- Stabilitás. Stabil-e a háló működése ?
- Szeparálhatóság. Mennyire befolyásolja a teljesítményt a belső ismeretszemléltetés ?
- Memóriakapacitás (a tárolható minták száma)
- Zajos input tűrése
- Hamis memóriaállapotok elkerülése
- Térbeli, vagy időbeli minták alkalmazása.

Nevezetes problémák

Mesterséges intelligencia alapok

2025

Problémák csoportosítása

- Megoldási idő szerint
 - P (Polynomial)
 - Viszonylag egyszerűen megoldható problémák
 - P típusú egy probléma, ha létezik olyan problémamegoldó algoritmus, melynek időszükséglete legfeljebb N polinomjával függ össze
 - NP (Non-deterministic polynomial)
 - „Nehéz” problémák
 - Azok a problémák, melyek bármely feltételezett megoldása polinomiális idő alatt eldönthető, hogy helyes-e egy determinisztikus Turing gép segítségével
 - Vagy azok a problémák, amelyek megoldhatóak polinomiális idő alatt nemdeterminisztikus Turing gép segítségével
 - NP-teljes (NP complete)
 - A legnehezebben megoldható problémák
 - Nagy változós szám esetén megoldhatatlan
 - A megoldáshoz szükséges idő e^N (vagy $n!$) szerint változik
 - NP-nehéz (NP hard)
 - Olyan problémák, amelyre minden NP-beli probléma visszavezethető
 - De nem kell NP-nek lennie (Valóban nem is eldönthető)

P problémák

Áramkör érték probléma (Circuit Value Problem)

- Logikai áramkör kimenetének kiszámításának problémája egy adott bemeneten
- Lineáris időben, topológiai rendezéssel megoldható
- Speciális esete: Bool képletkiértékelési probléma
 - Az áramkör egy fa

3SUM

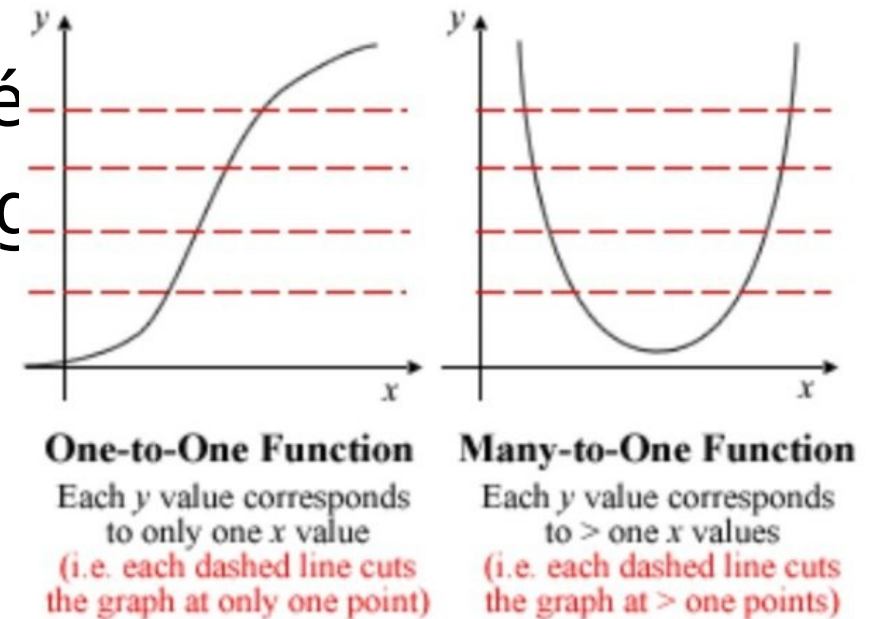
- Adott n valós számból álló halmaz
- Kérdés: tartalmaz-e három olyan elemet, amelyek összege nulla
- Általánosítása:
 - k -SUM: tartalmaz-e k számú elemet, amelynek összege nulla
- Időigénye: $O(n^2)$

Hozzárendelés probléma

- A probléma ügynököket és feladatokat tartalmaz
- Bármely ügynök hozzárendelhető bármilyen feladat végrehajtásához
 - Ügynök-feladat összerendeléstől függő költségekkel jár
- A lehető legtöbb feladatot úgy kell elvégezni, hogy
 - Minden feladathoz legfeljebb egy ügynököt rendelnek
 - Minden ügynökhöz legfeljebb egy feladatot rendelnek
 - Hozzárendelés összköltsége minimális legyen
- Ha az ügynökök és a feladatok száma megegyezik, akkor kiegyensúlyozott hozzárendelésnek nevezzük, különben kiegyensúlyozatlan hozzárendelésnek
- Ha a hozzárendelés összköltsége az összes feladatra megegyezik az egyes ügynökök költségeinek összegével, akkor lineáris hozzárendelésnek nevezzük

Ütközés probléma

- Leggyakrabban az elnevezés a 2 az 1-hez verziót jelenti
- Adott egy páros n és egy $f: \{1, \dots, n\} \rightarrow \{1, \dots, n\}$ függvény
 - A függvény 1 az 1-hez (injektív) vagy 2 az 1-hez hozzárendeléssel rendelkezik
- Csak az $f(i)$ értékeire tehetünk lekérdezés
- Kérdés: Hány lekérdezés szükséges, hogy megállapítsuk, hogy f 1 az 1-hez vagy 2 az 1-hez



Legkisebb élfedés probléma (Minimum edge cover problem)

- Élfedés: az élek részhalmaza, gráfélek gyűjteménye úgy, hogy az élvégpontok uniója megfelel a gráf teljes csúcskészletének
- Csak elszigetelt pont nélküli gráfoknak van élfedése
- Legkisebb lefedés: A legkevesebb élből álló halmaz, amely élfedést valósít meg

Elem megkülönböztetési probléma

- Annak az eldöntésére szolgál, hogy egy lista összes elemei különböző-e
- A probléma megoldható a lista rendezésével, majd annak ellenőrzésével, hogy vannak-e egymást követő egyenlő elemek
- Akár lineáris várható időben is megoldható
- Általánosítása: Ismétlődő elemek megtalálása

Üresség probléma

- Formális nyelvek témakörben
- Üres formális nyelv: Ha egyetlen valid mondata, eleme sincsen (Még az üres jelsorozat sem)
- Probléma: Eldönteni adott reprezentáció alapján, hogy üres-e a nyelv
 - Reprezentáció lehet például a véges állapotú automata
- n állapotú automata esetén a probléma $O(n^2)$ idő alatt elvégezhető
- Egyes variánsai már nem ilyen könnyen megoldhatóak

Leghosszabb közös részsorozat probléma

- Célja: Megtalálni a leghosszabb részsorozatot, amely az összes szekvenciára jellemző egy sorozathalmazban
 - Gyakran csak két szekvenciában
- Tetszőleges számú bemeneti sorozat esetén NP-nehéz
- Ha a szekvenciák száma konstans, a probléma megoldható polinomiális időn belül dinamikus programozással

NP-teljes problémák

Bool-féle kielégítési probléma (Boolean satisfiability problem (SAT))

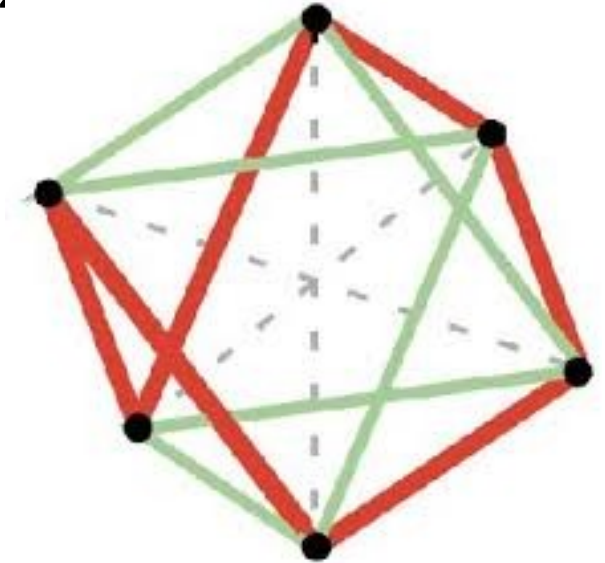
- Első ismert NP-teljes probléma (igazolta Stephen Cook által 1971-ben és Lenid Levin által 1973-ban)
- Kérdés:
 - Van-e olyan interpretációja egy adott Boole- formulának, amely azt kielégíti
- Más szavakkal, annak az eldöntése, hogyha az adott formula változóit konzisztensen lecseréljük IGAZ vagy HAMIS értékekkel, akkor a képlet IGAZ értéket ad-e
 - Ebben az esetben a formula kielégíthető
 - Ha minden lehetséges behelyettesítésre HAMIS, akkor kielégíthetetlen

3-SAT probléma (3-satisfiability problem)

- A SAT egy speciális esete
- A formula konjunktív normálformában van megadva, ahol minden tagmondat legfeljebb három literálra korlátozódik
- $(x \vee \neg x \vee y) \wedge (\neg x \vee \neg y \vee \neg y) \wedge (\neg x \vee y \vee y)$
- Minden tagmondatban van legalább egy IGAZ literálja
- Ez alapján bizonyítják más problémák NP-nehézségét
- Speciális esetei:
 - Konjunktív normálformában megadva, pontosan három literál egy tagmondatban
 - Diszjunktív normálformában
 - 1-in-3 SAT: Konjunktív normálforma, pontosan három literállal egy tagmondatban, de minden tagmondatban csak egy IGAZ literál lehet
 - Lineáris SAT: Ha minden tagmondatnak csak egy másik tagmondattal van metszete, és a metszetükben csak egy literál lehet közös

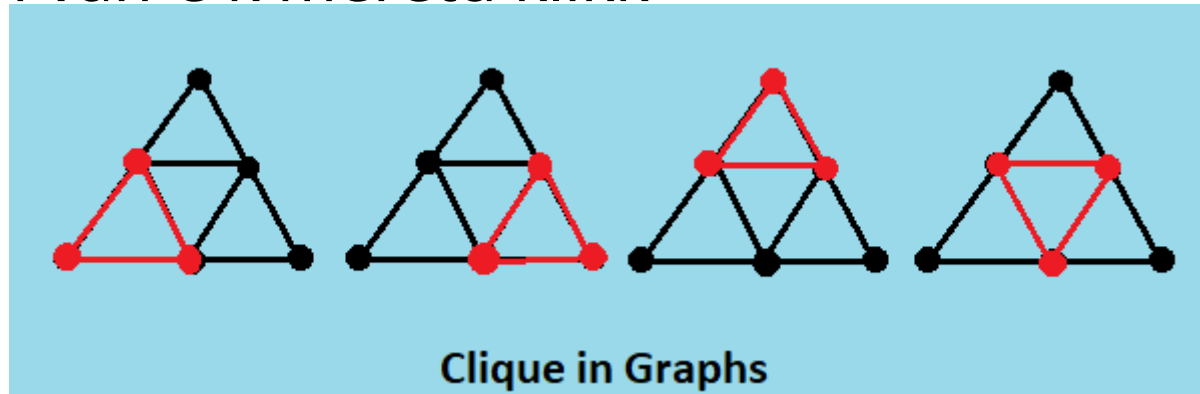
Gráf Hamilton köre/útja

- Hamilton kör: Olyan kör egy gráfban, amely a gráf összes csúcsán pontosan egyszer halad át
 - NP-teljes bizonyítása a csúcsfedés probléma segítségével
- Hamilton út: Olyan út egy gráfban, amely a gráf összes csúcsán pontosan egyszer halad át
- NP-teljes probléma:
 - Páros gráf
 - Irányítatlan síkba rajzolható gráfok maximum 3-as fokszámmal
 - Irányított síkba rajzolható gráfok maximum 2 bemeneti és kimeneti fokkal
 - Hídmentes irányítatlan síkba rajzolható, 3-regulális páros gráf
 - A négyzetrács gráf részgráfjai



Klikk eldöntési probléma (Clique problem)

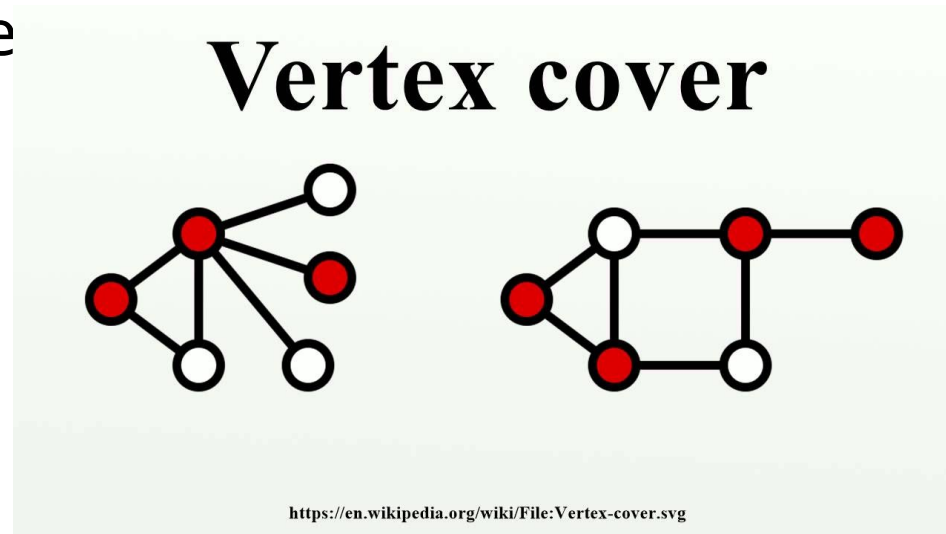
- Klikk: egy irányítatlan gráf csúcsainak olyan halmaza, melyek feszített részgráfja teljes
 - A klikk bármely két csúcsa között van él, bármely két csúcsa szomszédos
- Kérdés:
 - Bemenet: irányítatlan gráf, k paraméter (pozitív egész)
 - A gráfban van-e k méretű klikk



Minimum lefedő csúcshalmaz eldöntési probléma (Minimum vertex cover problem)

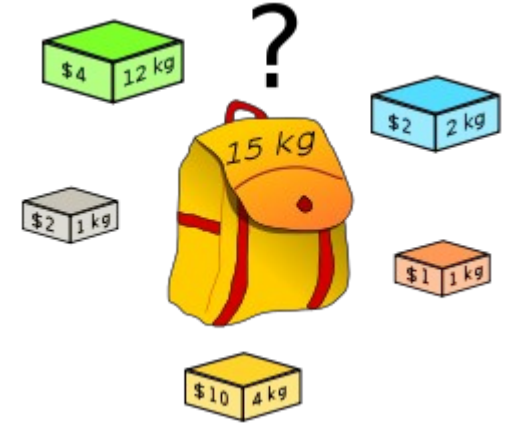
- Lefedő csúcshalmaz: A gráf csúcsainak egy olyan halmaza, amely a gráf minden élének legalább egy végpontját tartalmazza
- Adott gráf, k paraméter (pozitív egész)
- Kérdés:
 - A gráfnak van-e

halmaza



Hátizsák probléma

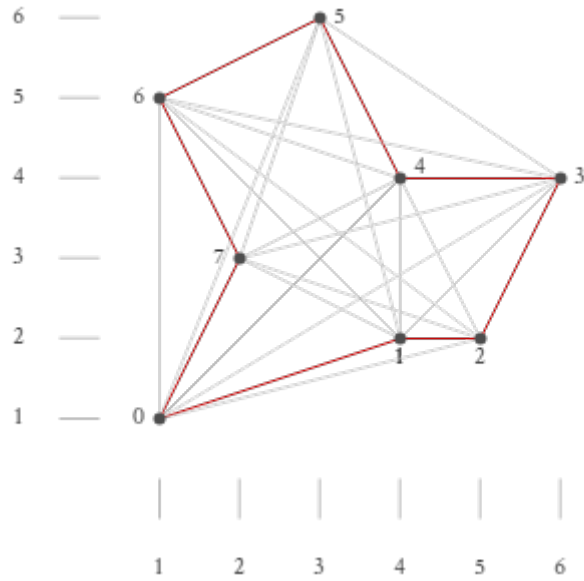
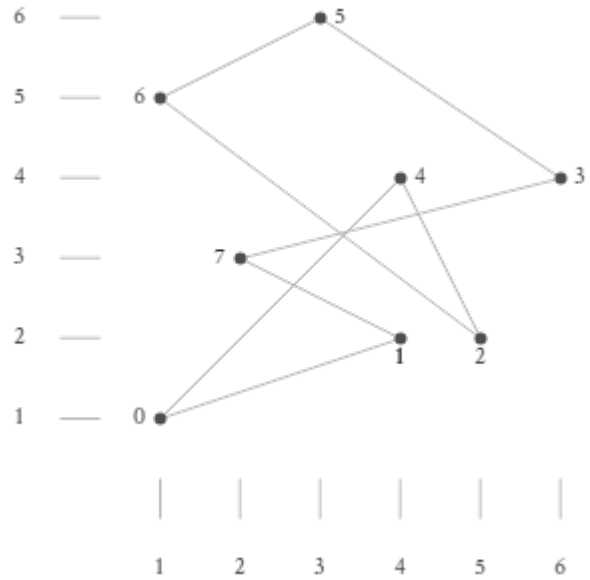
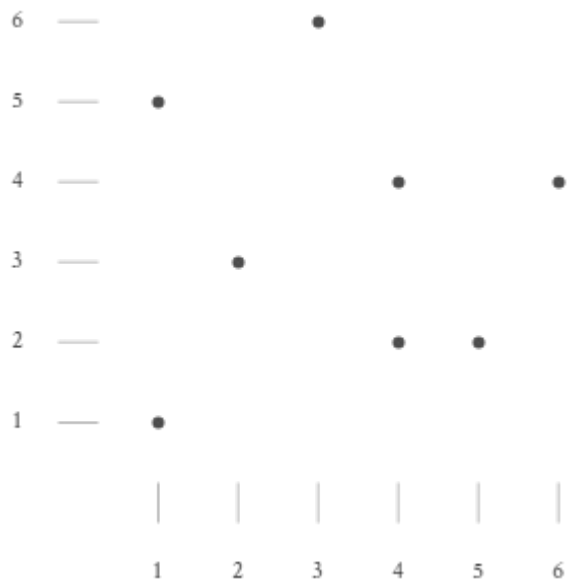
- Adott- elemek halmaza. Minden elemnek van:
 - Értéke
 - Súlya
- Paramétere: Súlykorlát
- Cél:
 - Meghatározni minden elem darabszámát, hogy az összsúlyuk ne haladja meg a súlykorlátot, de az összérték a lehető legnagyobb legyen
- A probléma először 1897-ben jelent meg



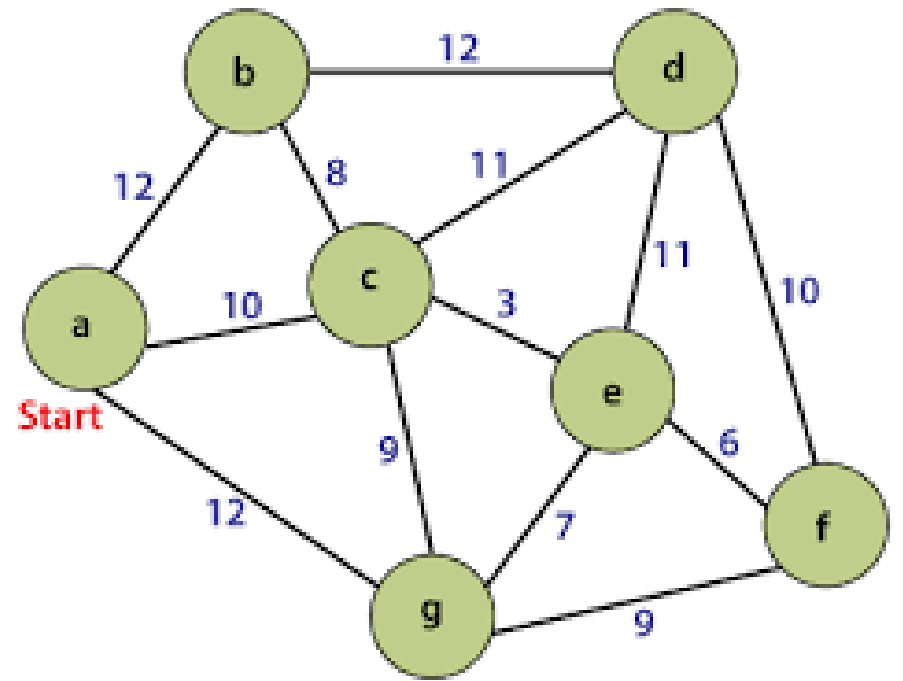
NP-nehéz

Utazóügynök probléma (Travelling salesperson problem- TSP)

- Adott városok listái, és köztük lévő távolság
- Kérdés:
 - Mi a legrövidebb út, amely az összes várost pontosan egyszer érinti, és visszatér a kezdő városba?
- Először 1930-ban fogalmazták meg
- Az optimalizáló megoldások számára viszonyítási alap
- Tervezésben, logisztikában, és mikrochip gyártásban jelentős gyakorlati felhasználása van
- Módosított változatai DNS szekvenálás és asztronómia témakörében is hasznosított
- Heurisztika és optimális utat eredményező algoritmus már ismert, milliós városoknál nagyon jól közelíthető



Travelling Salesman Problem

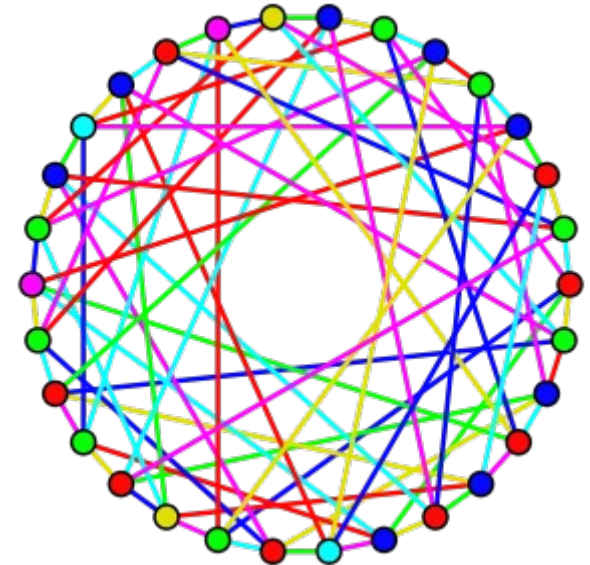


Minimum lefedő csúcshalmaz probléma

- Lefedő csúcshalmaz: A gráf csúcsainak egy olyan halmaza, amely a gráf minden élének legalább egy végpontját tartalmazza
- Kérdés:
 - A legkisebb lefedő csúcshalmaz megkeresése az adott gráfban
 - Visszaadja a k értéket

Teljes színezés

- A gráf csúcsaihoz és éleihez is szín van rendelve
- A szomszédos csúcsok, szomszédos élek, és az él és azok végpontjai sem lehetnek ugyanolyan színűek
- Cél a kromatikus szám meghatározása



K-minimum feszítőfa probléma

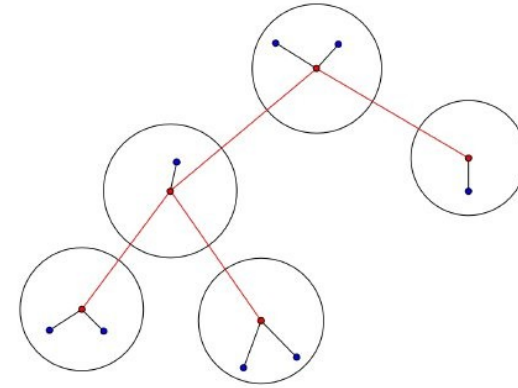
- 1993-ban jelent meg a probléma
- Bemenete:
 - Irányítatlan gráf, súlyozott élekkel
 - k paraméter
- Célja:
 - Minimális költségű fa, amelynek pontosan k csúcsa, $k-1$ éle van, és egy nagyobb gráf részgráfját képezi

Max-3SAT problem

- Általánosított SAT probléma
- Bemenete:
 - Konjuktív normálforma, maximum 3 literállal egy tagmondatban
- Célja:
 - Olyan hozzárendelés keresése, amely a legtöbb tagmondatot kielégíti

Metrikus k-középpont

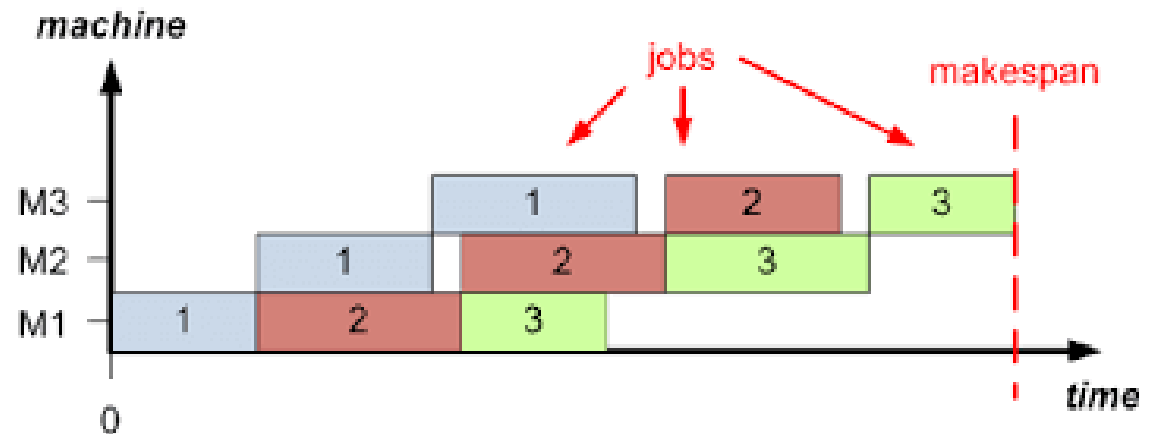
- Adott:
 - N város adott távolság függvényen
- Cél:
 - k db raktár építése a különböző városokban olyan módon, hogy minimalizálja a maximum távolságot a városoktól a raktárakig
 - Gráfelméleti megfogalmazás: meg kell találni egy k csúcsból álló halmazt, amelynek bármely pontjának legnagyobb távolsága a legközelebbi csúcstól a k -halmazban minimális



Flow Shop ütemezési probléma

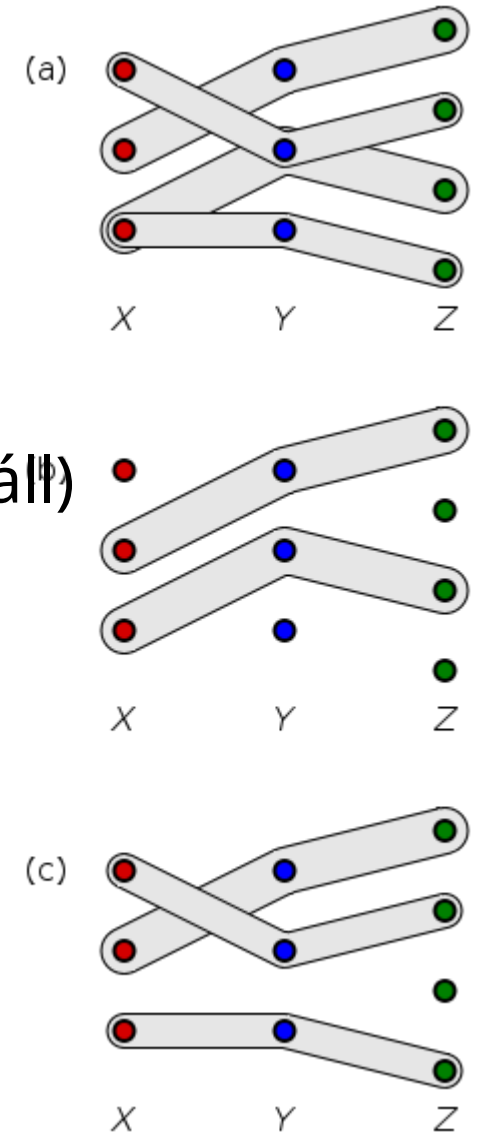
- Adott:
 - n darab munka $J_1, J_2, \dots, J_n$ különböző feldolgozási idővel
 - m különböző feldolgozási teljesítményű gép
- Cél:
 - A munkákat a gépekre kell ütemezni, miközben megpróbáljuk minimalizálni az átfutási időt (amikor az összes munkát befejeztük)
- Számos variánsa létezik

A termelésinformatika szakirány foglalkozik ezzel



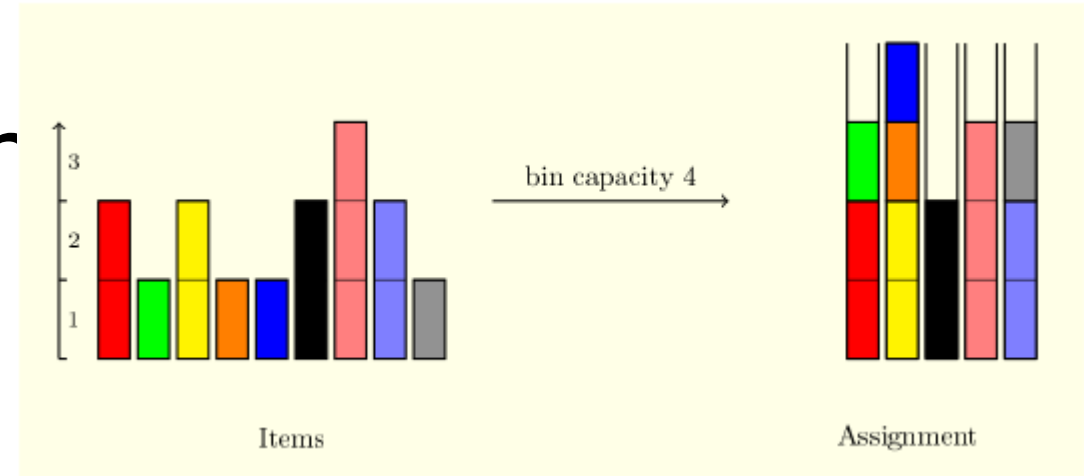
Háromdimenziós házaspárítás (3-dimensional matching)

- Adott:
 - X, Y és Z diszjunkt halmazok, mindegyik n méretű
 - T részhalmaza a $X \times Y \times Z$ -nek (T (x,y,z) hármasokból áll)
- $M \subseteq T$ egy háromdimenziós házaspárítás ha:
 - Bármely kettő különálló hármasra $(x_1, y_1, z_1) \in M$, és $(x_2, y_2, z_2) \in M$, akkor $x_1 \neq x_2$, $y_1 \neq y_2$, $z_1 \neq z_2$



Rekesz pakoló probléma

- Adott:
 - Különböző méretű tételek
 - Véges számú, meghatározott tárolóedény vagy konténer
- Cél:
 - A tételeket a konténerbe kell csomagolni oly módon, hogy minimalizálja a felhasznált tárolóedények számát
- Alkalmazási területei:
 - Konténerek felöltése
 - Teherautók betöltése súlykorlátozással



Leghosszabb út probléma

- Adott:
 - Gráf
- Cél:
 - A leghosszabb egyszerű útvonal keresése az adott gráfban
- Egyszerű útvonal: Nincs ismétlődő csomópontja
- A hossz lehet
 - Az élek száma
 - Súlyozott gráf esetén az élek összsúlya

NP problémák

Utazóügynök döntési verzió

- Bemenete egy n város közötti távolságok mátrixa
- Kérdés: Van-e olyan útvonal, amely
 - az összes várost meglátogatja
 - Teljes távolsága kisebb, mint k
- A feltételezett megoldás lehet egyszerűen a városok listája
 - Ekkor az ellenőrzés egyértelműen polinomiális időben végezhető el
 - Összeadja a két város közötti útnak megfelelő mátrixértékeket

Részgráf izomorfizmus probléma

- Adott G és H gráfok
- Kérdés: G gráf tartalmaz-e olyan részgráfot, amely izomorf a H gráfhoz
- A maximum klikk probléma és a Hamilton kör tesztelésének általánosítása
 - Alapvetően NP-teljes probléma
 - A részgráf izomorfizmus bizonyos más esetei megoldhatóak polinomiális időben

Egész faktorizációs probléma döntési verzió

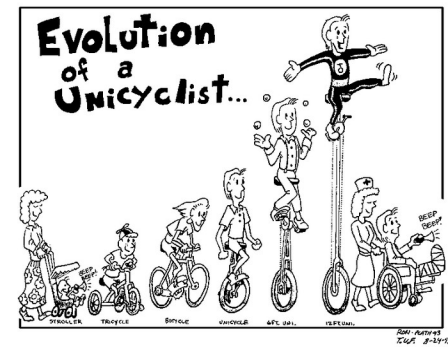
- Adott n és k egész szám
- Van-e olyan f tényező, amelyre igaz, hogy $1 < f < k$ és f osztja n -t?
- Jelenleg nincs olyan algoritmus, amely az összes egész számot polinomiális időben faktorizálja
- Kvantum számítógépre írt algoritmus meg tudja oldani polinomiális időben

Genetikus algoritmus

Mesterséges intelligencia alapok
2025

Dr. Dudás László diái alapján

Evolúciós algoritmusok



- Az evolúciós algoritmusok a biológiai evolúciót modellezik. Az ember számtalan probléma megoldásához merítette az alapötletet a természetből. Pl. madarak szárnya → repülőgépszárny, bambuszrúd → rúdugrók üvegszálás rúdja, halak léghólyagja → tengeralattjárók merülési berendezése, emberi szemlencse → fényképezőgép optikája, stb.
- Amit az ember gondolkodással próbál megoldani, azt a természet gyakran hatékonyabban oldotta meg a változatosság és a szelekció eszközével. A többnyire kétszülős biológiai szaporodás és a többnyire külső hatásoknak köszönhető véletlenszerű megváltozás, a mutáció **az egyedek sokszínű-ségét** eredményezi, míg a túlélésért folyó verseny a jobb, életképesebb egyedek **kiválogatódását**, tulajdonságaik továbbörökítésének lehetőségét hordozza magában. E két működés a populáció környezeti elvárásoknak való jobb megfelelését, állandó alkalmazkodását jelenti.

Evolúciós algoritmusok és az optimumkeresés

- Az evolúciós algoritmusokkal végzett problémamegoldást **biológiai számításnak** is nevezik.
- Az evolúciós algoritmusok **populációjának** egyedei megfeleltethetők a keresés állapotterében található **állapotok** egy részhalmazának, az egyedek életképességét, jóságát mérő ún. **fitness függvény** megfeleltethető a keresés **kritériumfüggvényének**.
- Az evolúciós mechanizmus mintájára létrehozott programokkal akkor is megoldhatók a feladatok, ha a problémamegoldásra nem tudunk részletes algoritmust adni. Az irányított „próba-szerencse” módszer feladatérzéketlenségéért, **robosztusságáért számítási idővel kell fizetnünk**. Ebben is a biológiai modellre hasonlít: az evolúció nem gyorsan zajló folyamat.
- A biológiai evolúció és az evolúciós algoritmusok **eltérése**
A biológiai evolúció célja a környezet elvárásaihoz való minél hatékonyabb **alkalmazkodás**, míg az evolúciós algoritmusokkal globális **optimumok keresését** végezzük. Ezen optimumkeresési feladatok általában nem biológiai beágyazó környezetben zajlanak.

Evolúciós algoritmusok osztályozása

- **Evolúciós stratégiák:** a kezdet, (Rechenberg, 1973, repülőgépszárny optimalizálás.)
- **Evolúciós programozás:** programkód kifejlesztése a kódrészletek mutálódása és szelektálása által. Véges automaták automatikus kifejlesztésére (Fogel, Owens, Walsh, 1966.).
- **Genetikus algoritmus,** GA: keresztezés, mutáció és szelekció matematikai modellezése (Holland, 1975).
 - Osztályozó rendszerek
 - **Genetikus programozás,** GP: programok kitenyésztése adott feladatra. (Koza, 1992).

Az Evolúciós algoritmusok általános lépései

1. Legyen P_0 a kezdeti populáció, $k=0$ a ciklusváltozó kezdőértéke.
2. Ha a megállási kritérium teljesül, add vissza a P_k populációt.
3. Egyébként bővítsd a P_k populációt új egyedekkel.
4. Szelekcióval állítsd elő a P_k populációból az új P_{k+1} populációt.
5. Inkrementáld a k ciklusváltozót és ismételd a 2. ponttól.

A 3. lépésben a bővítés alkalmazhat szülőválasztást, keresztezést és mutációt.

A 4. lépés feladata a populáció létszámának eredeti értéken tartása a vesztes egyedek törlésével.

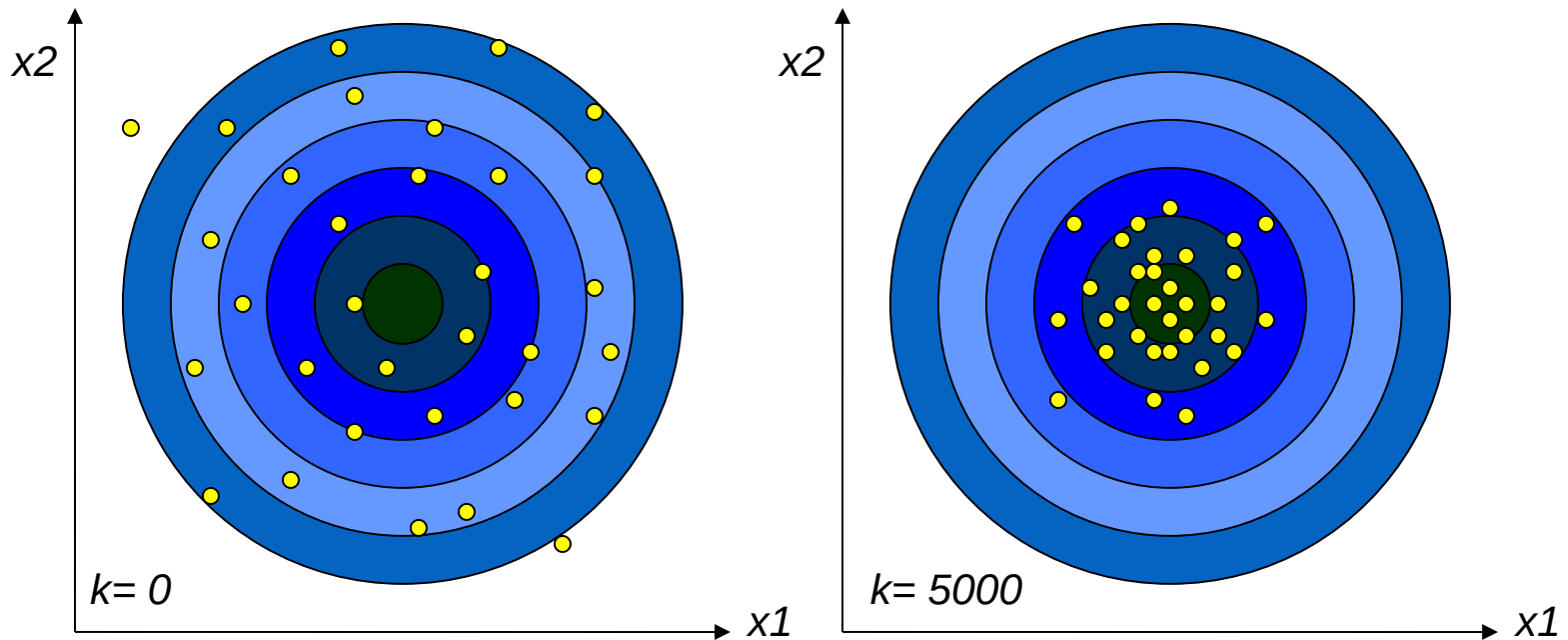
Amennyiben a populáció **egyelemű**, csak mutáció van, az algoritmus **helybeni**, és neve **sztochasztikus hegymászó**.

Genetikus algoritmus

- A genetikus algoritmus egy **globális kvázioptimum** megtalálására kifejlesztett kereső algoritmus, mely alapvetően **problémafüggetlen**. A lokális informáltságú, de globális célra törő algoritmusok, a többszörös indítású hegymászó, a szimulált hűtés és a tabu keresés rokona.
- A problémára vonatkozóan rendelkezésre álló **információ bevihető** az algoritmusba a valós egyedeknek megfelelő **fenotípusról** a számításban alkalmazott modelljére,
a **genotípusra** való áttéréskor, valamint
a keresztezés és a mutáció módjának megválasztásakor.
Az ily módon kialakított, problémára szabott genetikus algoritmusok **hatékonysága megnő**.
- A globális optimum megtalálásában az egzakt analitikus módszerekhez (differenciálás, gradiens módszer, simplex módszer) viszonyítva lassú, pontatlan, de diszkrét állapottereken is működik, igénytelen a feladattal szemben.
A hasonlóan robusztus és igénytelen kimerítő kereséstől hatékonyabb.

- A genetikus algoritmus **nevét** az élőlények utódaiban megjelenő kombinálódott szülői génekről, mint az utód tulajdonságainak hordozójáról kapta. A genetikus algoritmus alkalmazásának kulcskérdése a fenotípusról genotípusra való leképezés, azaz az egyed egyes tulajdonságait reprezentáló „gének”, azaz kromoszómarészletek kialakítása.
- Mivel egyszerre egy populációt, azaz egyedek egy csoportját kezeli, a globális optimum fellelésének megnő az esélye. Az állapotter feltárásában a következők játszanak szerepet:
 - A kezdeti populáció egyedeinek eloszlása az állapottéren
 - A keresztezések interpoláló hatása
 - A mutációk extrapoláló, a keresztezéssel kialakult belterjességből, lokalizáltságból kivezető hatása.
- A kezdeti mutációk az állapotter hatékony feltárását eredményezik. A globális optimum köré gyűlt késői populációkból viszont már csak gyorsan elhaló mutációk származnak, mivel azok életképessége a populáció többi tagjához képest alacsony.

- A **populáció fejlődése** az egyedek fejlődésén keresztül realizálódik. A szelekciónak köszönhető **látszólag spontán** fejlődés nem csak a véletlen eredménye: a jobb célfüggvény (fitness) értéket reprezentáló egyedek aránya a populációban egyre nő, míg a kevésbé életképeseké csökken. Az egyedek a **módszer memóriájaként** is felfoghatók: a jó tulajdonságú egyedek megtartásával az algoritmus **megtanulja** a jó célfüggvényértéket adó tulajdonságokat, a gyengébbeket eredményező tulajdonságok pedig **elfelejtődnek** az azokat hordozó egyedek kihalásával.



- Leképezés fenotípusról genotípusra

Feladata minden egyes állapothoz egy karaktersorozat, azaz bitminta hozzárendelése.

A leképezés kihat az alkalmazható operátorokra, végeredményben a GA hatékonyságára.

- A problémától függően a megoldásokat különböző módon lehet reprezentálni:
 - **Bináris kódolás:** Leggyakrabban használt módszer, ahol az egyedek bit-sorozatokkal vannak ábrázolva.
 - **Valós számokkal történő kódolás:** Hasznos a numerikus optimalizációs problémák esetén.
 - **Permutációs kódolás:** Alkalmazott például útvonal-optimalizációs problémákban, mint a Traveling Salesman Problem (TSP).
- Példák egyszerű esetekre:
 - Fenotípus: 12 zöld, vagy piros téglalapból álló sorozat.
Genotípus: 12 bites bináris szám, piros:1, zöld: 0.
Optimum: minden téglalap piros.
 - Fenotípus: $m \times n$ méretű fekete-fehér bittérkép alakjában adott kép.
Genotípus: $m \times n$ elemű bináris szám, fekete: 0, fehér: 1.
Optimum: két fekete átló a bittérképen.
- Megjegyzés: Az egyetlen **tulajdonság különböző értékeinek** tárolására alkalmazott több bites, egységként, génként kezelt kromoszómarszlet által felvehető értékeket **allél**-eknek nevezzük.

A szelekciós operátor

Feladata a szülőegyedek kiválasztása a keresztezés számára. A különféle szelekciós módszerekben közös, hogy a rátermettebb egyedeket **jelentősebb arányban** választják ki tulajdonságaik továbbörökítésére, de általában a gyengébb egyedek is kapnak egy **kis esélyt**.

- A szelekcióban alkalmazott technikák
 1. Rátermettség-arányos választás
 2. Párok versenyeztetése
 3. Rangsorolás

1.

1. **Rátermettség-arányos választás (Rulettkerék):** az egyed kiválasztásának valószínűsége arányos a populáció rátermettségi átlagához viszonyított rátermettséggel. A rátermettség gyakran azonos az egyedhez tartozó függvényértékkel, ritkábban azonban csak a szelekcióhoz alkalmazott egyedjellemző neve.

2. **Párok versenyeztetése:** véletlenszerűen kiválasztott két egyed közül a versenyt nyerő, azaz a nagyobb rátermettségi értékű lesz a kiválasztott. A módszer alkalmazható több, mint két versenyzős esetben is.

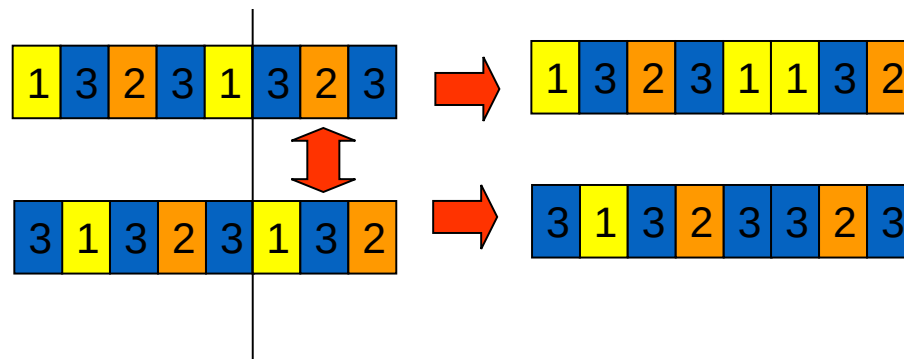
2. **Rangsorolás:** a keresztezés interpolációs hatásából eredő állapotér-feltárás hatásának lecsökkenését eredményezi, ha mindig ugyanazok az ősök szelektálódnak, öröközik tovább tulajdonságaikat. Ez a veszély fenyeget azoknál a módszereknél, amelyek a kiválasztást közvetlenül a rátermettség értékére alapozzák. Rangsorolás esetén a közvetlen **rátermettségi értékek helyett a rátermettségi sorrendre alapozott** szelekciót végzünk.

Keresztezés operátor

Szerepe utódok, azaz új egyedek előállítása.

- Leggyakoribb módszerek

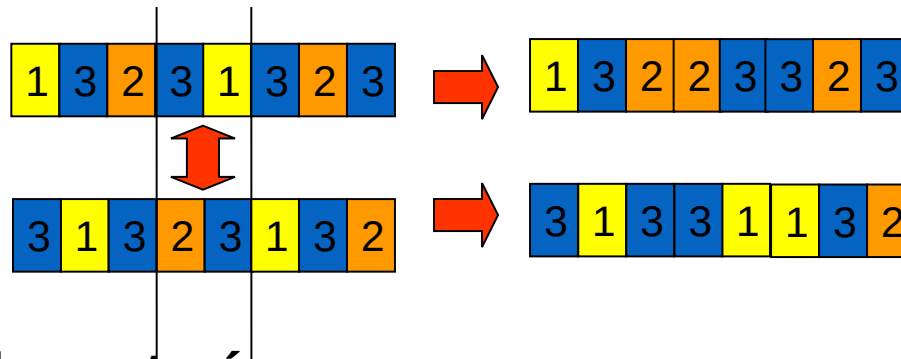
- **Egypontos keresztezés**



Véletlenszerű keresztezési pont választás.

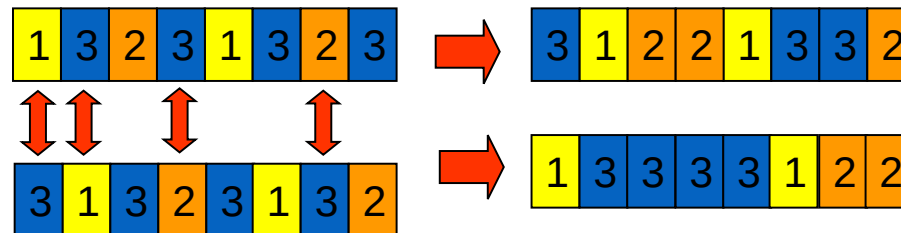
• Kétpontos keresztezés

- Két véletlenszerű keresztezési pont
- Megegyezik két különböző keresztezési pontú egypontos keresztezéssel, így a többpontos keresztezés is visszavezethető egypontos keresztezésre



• Egyenletes keresztezés

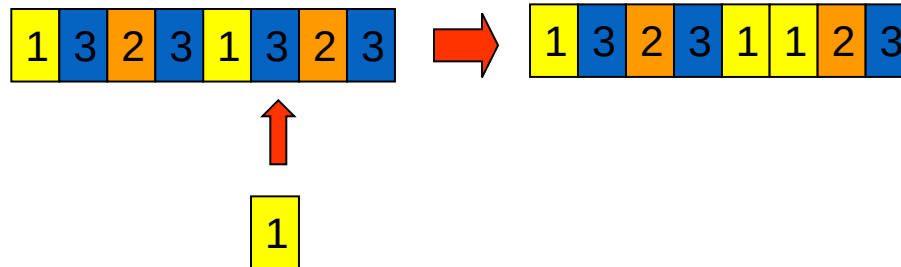
Ötven százalék eséllyel cserélődik minden egyes gén.



- **Útvonal-újrakapcsolás (Path Relinking)**
 - Két jó megoldás között köztes megoldások létrehozása (Lépésenkénti mutáció)
 - $P1 = (1,0,0,0)$, $P2 = (0,1,1,1)$
 - Akkor a gyerekek $C1 = (0,0,0,0)$, $C2 = (0,1,0,0)$, $C3 = (0,1,1,0)$
 - Távolság csökkentése a lényeg a változásnak
- Megjegyzések:
 - Nincsen általános módszer a keresztezés megválasztására, annak a feladathoz kell igazodnia.
 - Speciális keresztező operátort igényelhet az olyan eset, amikor összefüggés van a gének között (intelligens keresztezés).
 - A GA algoritmus érzékeny az operátorválasztásra.

Mutáció operátor

Olyan új egyedet hoz létre, mely mentes a keresztezés belterjességétől, azaz az ősökétől merőben elütő tulajdonságokat hordozhat. A keresztezéstől kisebb gyakorisággal alkalmazott művelet.



- A módszer: az adott egyed kromoszómájának egy véletlenszerűen választott génjét egy másik génre cseréljük ki.

A Genetikus algoritmus lépései

1. Add meg az algoritmus fő paramétereit
2. Add meg a kezdeti populációt
3. Ha a megállási kritérium teljesül, add vissza az aktuális populációt
4. Egyébként válaszd ki az egyedeket a keresztezéshez
5. Hajtsd végre a keresztezéseket
6. Válaszd ki az egyedeket a mutációhoz
7. Végezd el a mutációkat
8. Állítsd elő az új populációt az eredeti és a keresztezéssel és mutációval kapott egyedekből álló halmazból.
9. Ismételd a 3. ponttól.

- A Genetikus algoritmus paraméterei
 - A populáció számossága (pl. 16)
 - Kromoszóma hossz (pl. 12 bit)
 - A szelekciós operátor
 - A keresztezés operátor: a keresztezési ráta (új egyedek aránya, pl. 0.7), a keresztezési pontok száma, stb.
 - A mutáció operátor: a mutációs ráta (pl. 0.001), a mutációs pontok száma, stb.
- Leállási feltételek lehetnek:
 - Olyan megoldást talál, amely megfelel a minimális kritériumoknak
 - Adott generációszámot elértük
 - Elért költségvetés (számítási idő/pénz)
 - A legrátermettebb megoldás jósága eléri vagy elérte azt a platót, hogy a követő iterációk már nem hoznak jobb eredményeket (Nem javul jelentős mértékben)
 - A fentiek kombinációi

Új populáció létrehozása- Túlélési valószínűség

A leggyakoribb a rátermettségi érték (fitness érték alapú) kiválasztás, viszont más alapján is lehet túlélési valószínűséget számolni

Például:

a) Relatív rátermettség alapú:

$$P_i = \frac{f_i}{\sum_j f_j}$$

b) Diverzitás

$$diverzitás_i = \sum_j d(i, j)$$

Távolság (d) lehet:

Hamming távolság: Eltérő bitek száma

Euklideszi távolság: Összegzett kromoszómakénti különbségek négyzetének gyöke

Manhattan távolság: Kromoszómakénti különbségek abszolút értékének összege

Gyakori módszerek

1. Túlélési valószínűség-arányos választás: K-szor „megpörgetjük” a rulettkereket, majd az így kiválasztott K egyed legjobbjával térünk vissza. Ezt annyszor ismétljük, ahány egyedet szeretnénk az új populációba
2. Felső százalék elve: Az egyedeket rátermettség szerinti legjobb N százalékából választjuk ki az új populáció egyedeit
3. Valahány legjobb: Az egyedeket rátermettség szerint sorba rendezzük, és a valahány legjobbat választja ki közülük. (Elitizmus)
4. Rangsor alapú valószínűségű: Sorba rendezzük az egyedeket, paramétere a P valószínűségi változó. A legjobb egyed túlélési valószínűsége $P_1 = P$, az utolsó egyed esetében $P_n = (1 - P)^{n-1}$, a többi egyed esetén: $P_{n-1} = (1 - P)^{n-2}P$

Gyakori az értékek alapján együttesen történő rangsorolás is, ahol az új rangsor a részsorok összege, és ez alapján lesz meghatározva a túlélési valószínűsége

Például: Új rangsor = Fitnessz rangsor + diverzitás rangsor

A genetikus algoritmus alkalmazhatósága

Bár a genetikus algoritmusok nagyon igénytelenek az állapottérrel, az állapottéren értelmezett kiértékelő függvényrel szemben, van néhány követelmény, amelynek eleget kell tenniük a megoldandó feladatoknak:

- A feladat megoldási lépéseinek ismerete nem szükséges, de bizonyosnak kell lennünk a megoldhatóságában.
- A megoldások összetevői között lehet részleges függőség, de nem lehet az összes összetevő függőségi viszonyban.

Kereső eljárások

Mesterséges intelligencia alapok
2025

Dr. Dudás László diái alapján

Kereső eljárások

- Cél: egy adott problémára a szóbajöhető lehetőségek közül egy adott kritériumrendszernek eleget tévő megoldás megtalálása.
- A keresést az ismeretfeldolgozási eljárások közé soroljuk, mely révén a szemléltetett ismeret aktívvá tesszük, tudást nyerünk ki.



- Gyakran használják ismeretfeldolgozási eljárásként a keresést következtetőgráfokon következmény előállításra.

- A kereséssel történő problémamegoldás lépései
 1. A probléma beazonosítása kereséssel megoldható problémaként
 2. A problémára vonatkozó ismeretek reprezentálása, szemléltetése
 3. A kereső eljárás alkalmazása.

1. Kereséssel megoldható problémák jellemzői

Egy operátorkészlettel bejárható állapotok mindegyikén értelmezhető egy kritériumfüggvény. A probléma megoldása megfeleltethető a kritériumfüggvény adott értékének, értékhalmozának, vagy extrémumának.

2. A problémára vonatkozó ismeretek reprezentálása

Lásd a tudásszemléltetés elvárt jellemzőit, Patrick Winston szerint.

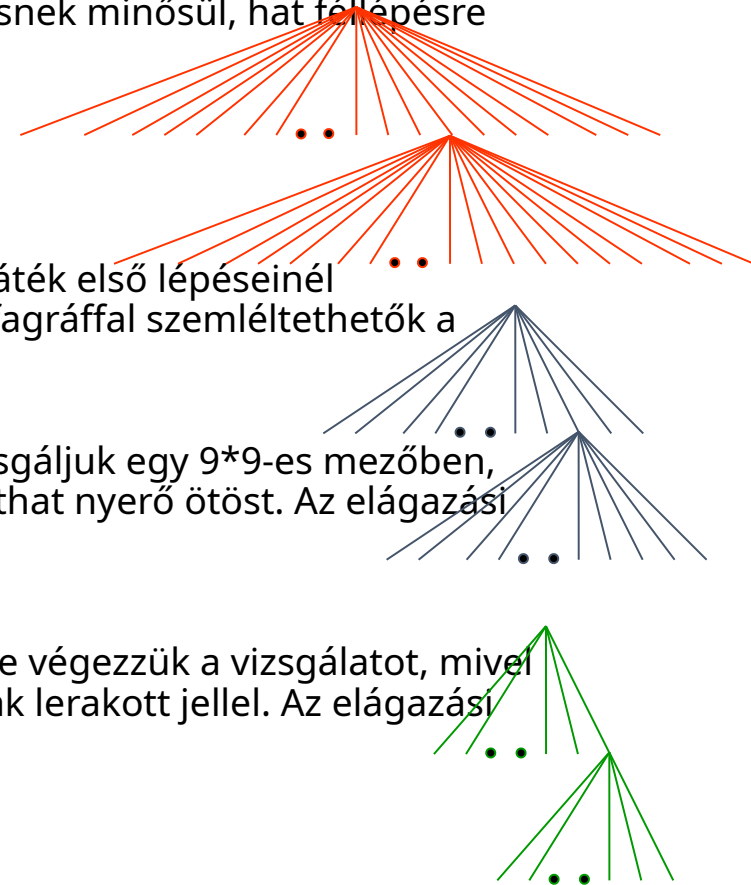
Az ismeretek reprezentálása erősen kihat az ismeretfeldolgozás, jelen esetben a keresés hatékonyságára. Ennek megvilágítására tekintsük a következő példát:

- Egy 20*20 méretű négyzetrácsos játéktéren kell az amőba játékban meghatározni azt a helyet, amely a lépésen következő szempontjából jó lépésnek minősül, hat féllépésre előre tekintve.

- Lehetséges reprezentációk**

1. Az összes üres mezőt vizsgáljuk: a lehetőségek száma a játék első lépéseinél **száz**as nagyságrendű, nagyon nagy elágazási tényezőjű fagráffal szemléltethetők a hat féllépéses részjátékokra.
2. Csak az előző lépés környezetében lévő üres helyeket vizsgáljuk egy 9*9-es mezőben, mivel az utolsónak lerakott jel csak ebben a részben alkothat nyerő ötöst. Az elágazási tényező maximum 80-as.
3. Csak az előző lépésre illesztett csillag alakzat üres helyeire végezzük a vizsgálatot, mivel ezek a helyek vannak közvetlen kapcsolatban az utolsónak lerakott jellel. Az elágazási tényező maximum 32-es.

Megj.: a reprezentációk gyengülésétől sokkal nagyobb előnyt jelent a kivitelezhető keresések hatékonyságnövekedése.
A reprezentációnak összhangban kell lennie a kereső eljárással.



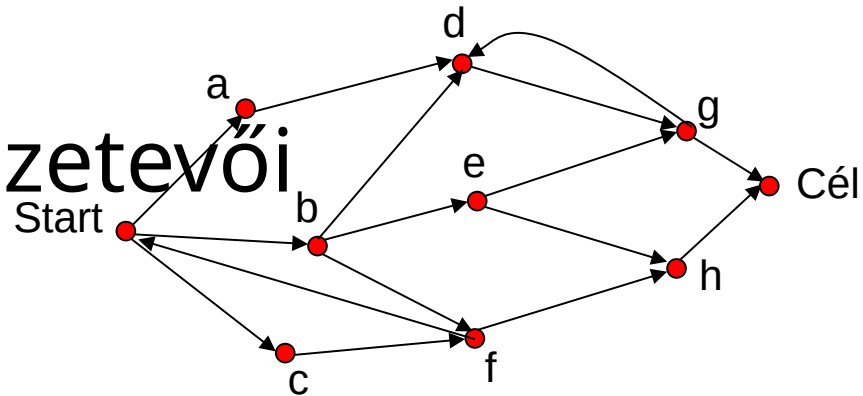
3. A kereső eljárás alkalmazása

A kereső eljárás alkalmazása a feladat megoldására az eljárások jellemzőinek ismeretében történhet. Ilyen közös jellemzők, melyek a megoldandó konkrét probléma függvényei:

- **Teljesség:** ha létezik megoldás, azt az eljárás meg is adja.
- **Optimalitás:** a Start-Cél útvonal egyben költségoptimális is.
- **Időigény:** az algoritmus lépésszámára adott felső korlát.
- **Tárigény:** a megoldás megtalálásához felhasznált memória méretére vonatkozó felső korlát.

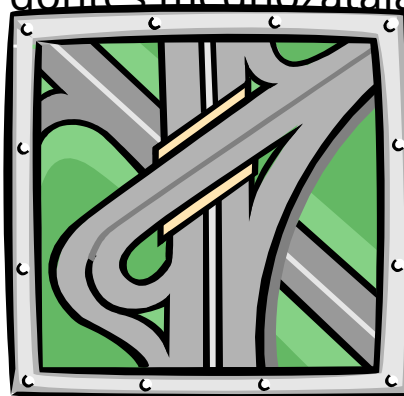


A kereső eljárások összetevői

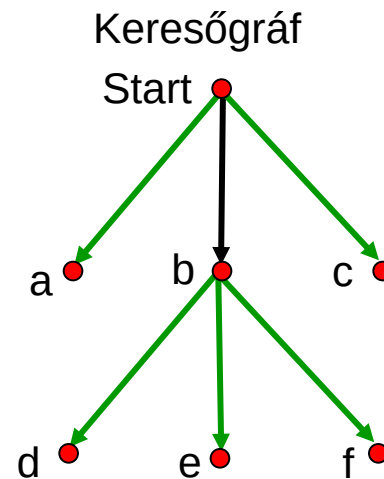
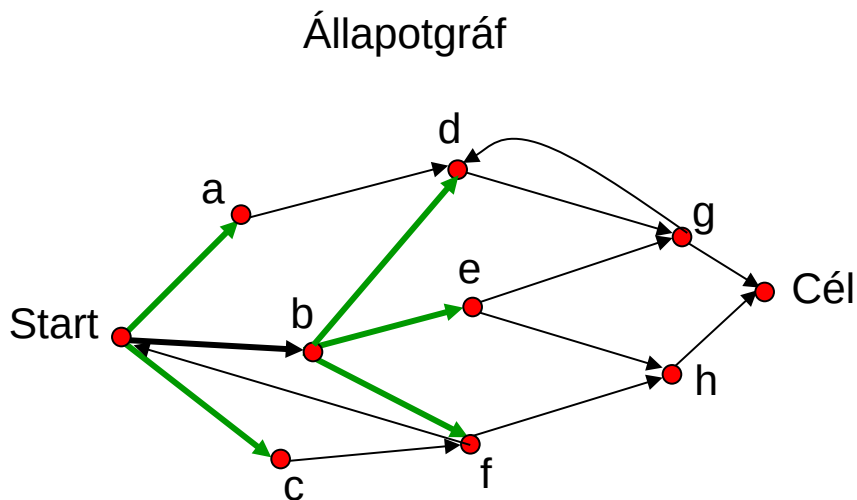


- **Állapottér**, reprezentálása az **állapotok**nak megfelelő csomópontokkal és az állapotok közötti mozgást jelentő **operátorok**nak megfelelő irányított élekkel.
- Kiinduló állapot: **Start**
- A kritériumnak megfelelő állapot/ állapotok: **Cél**.
- **Út, útvonal**: egy állapotból egy másik állapotba átvivő operátorsorozat, ill. érintett csomópontok rendezett listája.
- A keresés **leállási feltétele** előírhatja a kritérium teljesítését, vagy adott tűrésen belüli közelítését. Az utóbbi esetben *kvázioptimális* megoldásról beszélünk, mely gyakorlatilag jó és kompromisszumot jelent a keresés időigénye/költsége és a megoldás minősége között.

- **Az operátor költsége:** az operátorok a feladat valós tartalmától függő költséggel rendelkezhetnek: pl. legrövidebb út keresése városok között – az operátor költsége az útszakasz hossza. De lehet az operátor költsége érdektelen is, például bűvös kocka kitekerésénél csak a célállapot megtalálása, illetve az odavezető út fontos.
- **Az út költsége:** az úton alkalmazott operátorok költségének összege.
- **A keresés költsége:** a keresés idő- és tárigényéhez kapcsolódó költség.
- **A keresés teljes költsége:** az út költsége + a keresés költsége. Pl. városban történő útkeresés benzinköltsége: az útszakaszokon is fogy a benzin és az elágazásoknál az útválasztási döntés meghozatala ideje alatt is jár a motor.



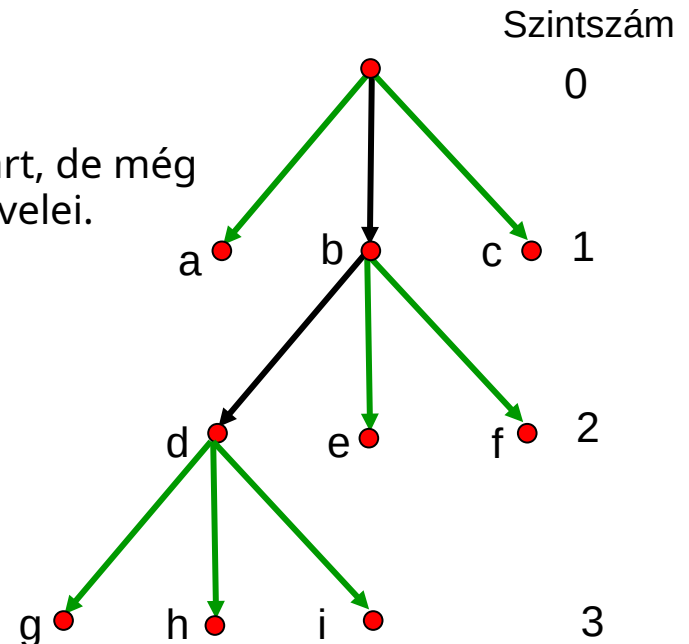
- **A keresőgráf:** egy fagráf, melynek csúcsa a Start állapot, valamelyik levele pedig, a Cél állapot, amennyiben létezik megoldás.
- **Kiterjesztés:** egy állapot kiterjesztése alatt az állapotból egyetlen lépéssel elérhető állapotok feltérképezését, azokba való betekintést, de még bele nem lépést értünk. Megfelel a keresőgráf egy adott pillanatnyi levélállapotától egy szinttel lejjebb lévő állapotok keresőgráfba való megrajzolásának.



A keresőgráf jellemzői

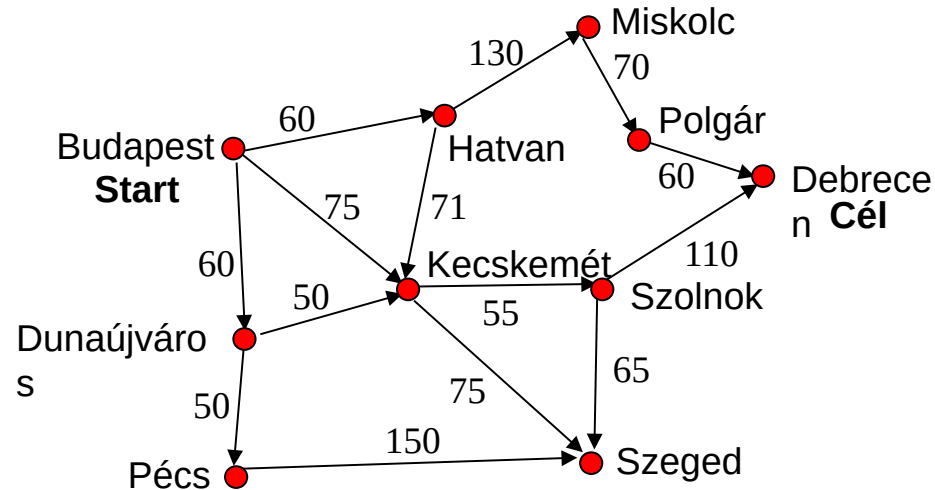
- **Elágazási tényező (branching factor, b):** egy adott csomópontból megtett kiterjesztés ágainak száma.
- **Átlagos elágazási tényező:** több csomópontra, leggyakrabban a teljes keresőgráfra értelmezett jellemző, a figyelembe vett csomópontok elágazási tényezőinek összege osztva a figyelembe vett csomópontok számával.
- A keresőgráf **mélysége** (m): a gráf legmélyebb szintjének száma, a Start a 0. szint.

- A **megoldás mélysége** (depth, d).
- Esetleges **mélységi korlát** (l).
- A keresés **frontja**: az összes, kiterjesztéssel feltárt, de még bele nem lépett csomópont, azaz a keresési fa levelei.
- Az n állapotba vezető út költsége $g(n)$
- Az n állapotból a Cél elérésének becsült költsége $h(n)$
- Az n állapoton átvezető út (becsült) költsége $f(n)$. (A^*)



- Példák kereséssel megoldható feladatokra

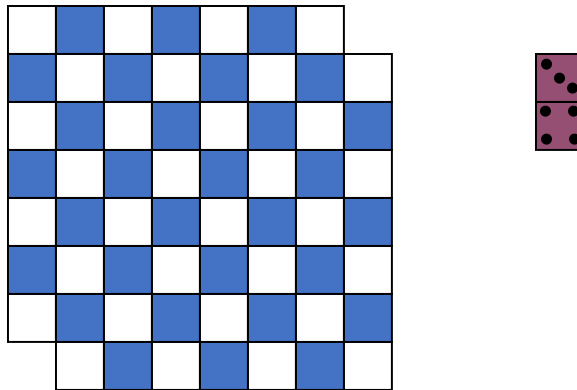
- **Útkeresés két város között**



- Probléma: melyik a legrövidebb útvonal Budapestről Debrecenbe?
- Start állapot: Budapesten vagyunk.
- Állapotok: valamelyik városban vagyunk.
- Leállási feltétel: a legrövidebb úton odajutva a Célban, Debrecenben vagyunk már? (A leállási feltétel egyben optimális utat is garantál.)
- Operátorok: utazás a szomszédos városok között.
- Költség: a megtalált Start-Cél útvonal hossza.

- Példák kereséssel megoldható feladatokra ..

- **Lefedési probléma**



- Probléma: hogyan lehet 31 dominóval lefedni egy, az átellenső sarkain csonka sakktáblát?
- Start állapot: az üres sakktábla.
- Állapotok: mindig eggyel több dominó valamilyen elrendezésben a sakktáblán.
- Leállási feltétel: mind a 62 mező le van fedve, vagy a lefedés lehetetlen voltának megállapítása.
- Operátorok: egy újabb dominó elhelyezése a sakktáblán.
- Költség: nincs.

Általános kereső eljárás

A kereső eljárások lényegi lépéseit tartalmazza az általános eljárás, amelytől a különböző eljárások csak kis részben térnek el.

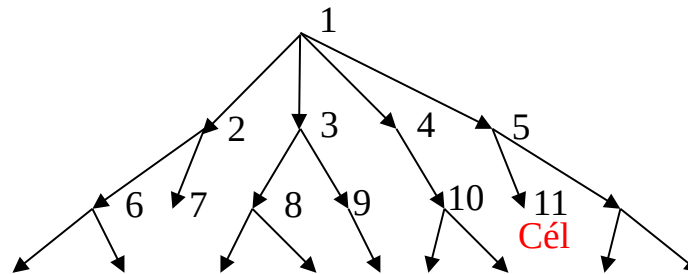
1. A keresési front legyen egyenlő a Start állapottal.
 2. Ha a front üres, akkor nincs megoldás, vége. Egyébként legyen n a front egyik állapota.
 3. Ha n kielégíti a Cél-kritériumot, akkor add vissza a hozzá vezető útvonallal együtt, vége.
 4. Egyébként a front n állapota helyére vedd fel a kiterjesztésével adódó új állapotokat és jegyezd fel a hozzájuk vezető útvonalakat. Ismételd a 2. ponttól.
- Az algoritmus szabad kezét adó pontja a 2., amelyben arról döntünk, hogy melyik állapot szomszédai irányába terjesszük ki a keresést. Ezen döntést végezhetjük gépiesen, a neminformált eljárások esetében, vagy a keresési feladatra vonatkozó információkra támaszkodva, az informált, vagy másnéven heurisztikus kereső eljárások esetében.

Neminformált kereső eljárások (vak keresés)

- Nincs információnk az aktuális állapot és a cél távolságára (költség, vagy lépésszám). Az eljárás csak arra képes, hogy észrevegye, ha a Cél állapotban van.
- Nem hatékonyak
- Általánosan használhatók.
- Típusok:
 - Szélességben először keresés (breadth-first search)
 - Elágaztatás és ugrálás (branch and bound), vagy másnéven egyenletes költségű keresés (uniform cost search)
 - Mélységben először keresés (depth-first search)
 - Mélységben először keresés mélységi korláttal (depth limited search)
 - Iteratív mélyítés (iterative depth-first search)

Szélességben először keresés

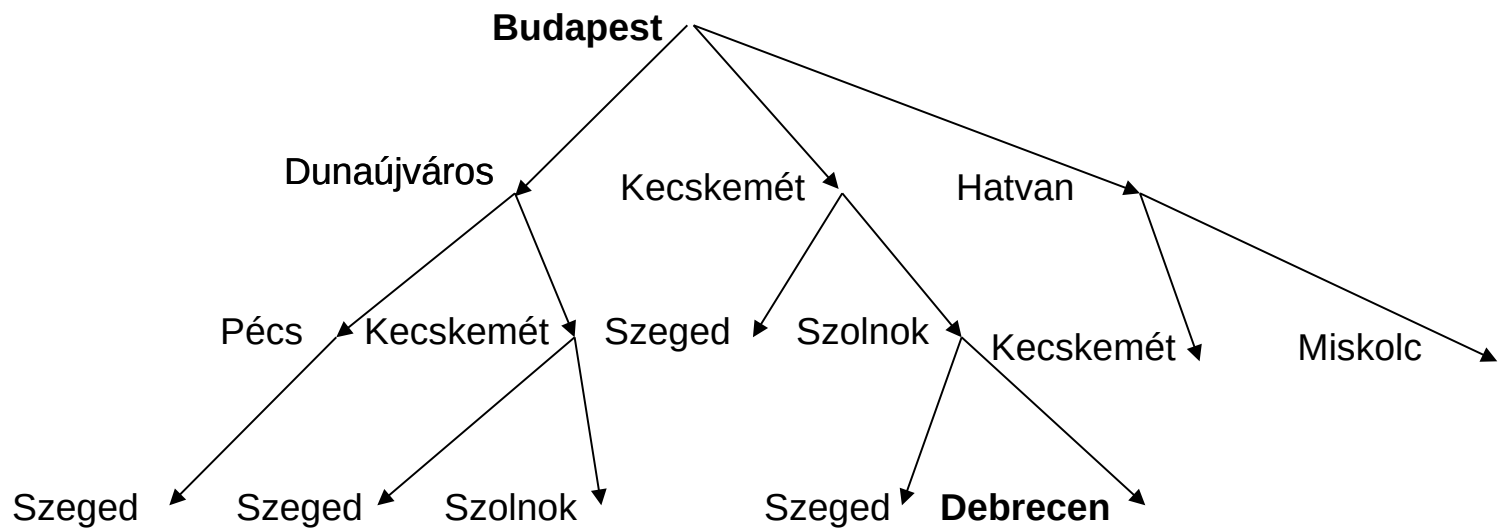
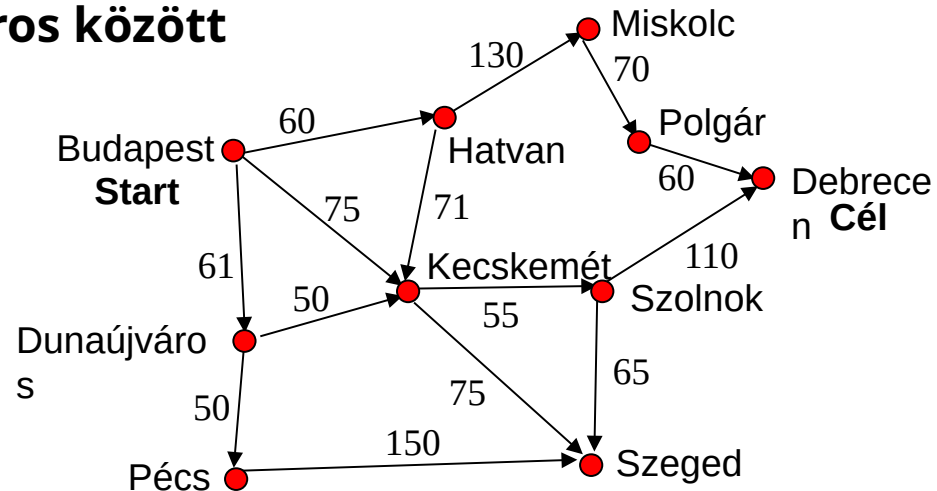
- Egy adott mélységi szint csomópontjainak mindegyikét kiterjeszti, mielőtt a következő mélységi szintre lépne.



- Az általános kereső eljárás a következőképpen módosul:
 - 2. Legyen a front első állapota az n választott állapot.
 - 4. A kiterjesztéssel kapott új állapotokat csatold a frontlista végéhez.
- Az eljárás
 - Teljes
 - Optimális (amennyiben az útszakaszok egyforma költségűek)
 - Időigénye b^d , (meredeken nő a mélységgel)
 - Tárigénye b^d , (meredeken nő a mélységgel).

- Példa a szélességben először keresésre

- **Útkeresés két város között**

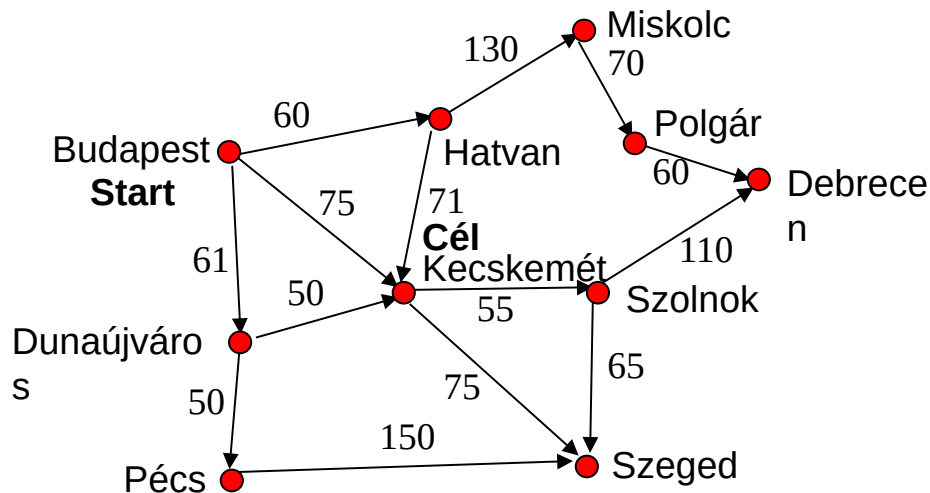


Elágaztatás és ugrálás

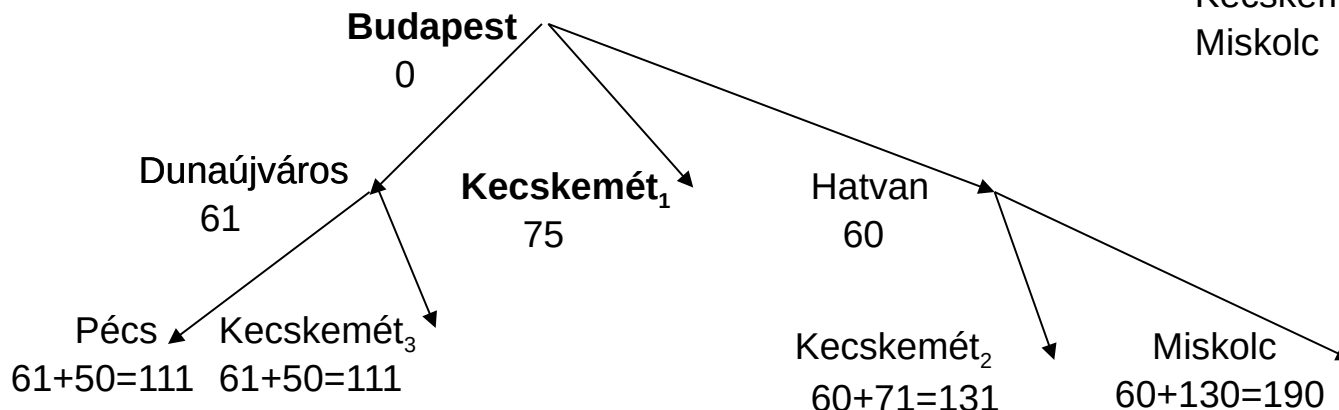
- A front azon állapotába lép, amelyikhez a legkisebb költségű út vezet a Start-tól.
- Ha az útszakaszok költsége egyforma, akkor a szélességben először keresésre vezet.
- Az általános kereső eljárás a következőképpen módosul:
 - 2. Legyen a front első állapota az n választott állapot.
 - 4. A kiterjesztéssel kapott új állapotokat add a frontlistához, majd rendezd az állapotokat növekvő költség szerint.
- Az algoritmus leáll, ha Cél-állapotba léptünk, azaz a Front összes állapotába nagyobb költségű út vezet.
- Az eljárás
 - Teljes
 - Optimális
 - Időigénye $\approx b^d$, (meredeken nő a mélységgel)
 - Tárigénye $\approx b^d$, (meredeken nő a mélységgel).

- Példa az *elágaztatás és ugrálás* keresésre

- Útkeresés két város között**



Sorsz.	Front	Választás
0.	Budapest	Budapest
1.	Dunaújváros Kecskemét ₁ Hatvan	Hatvan
3.	Dunaújváros Kecskemét ₁ Kecskemét ₂ Miskolc	Dunaújv.
4.	Pécs Kecskemét ₃ Kecskemét ₁ Kecskemét ₂ Miskolc	Kecskemét ₁

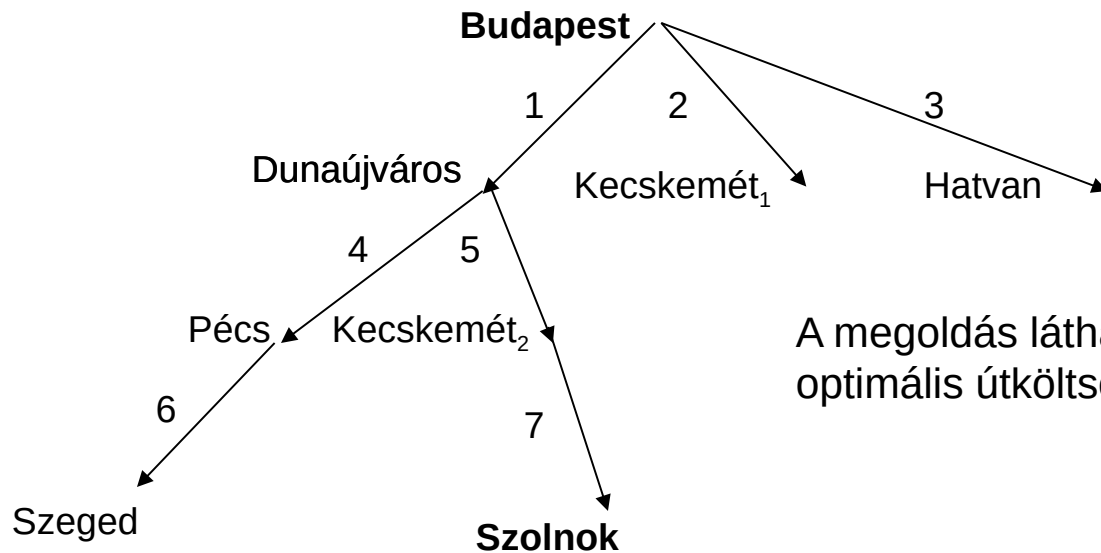
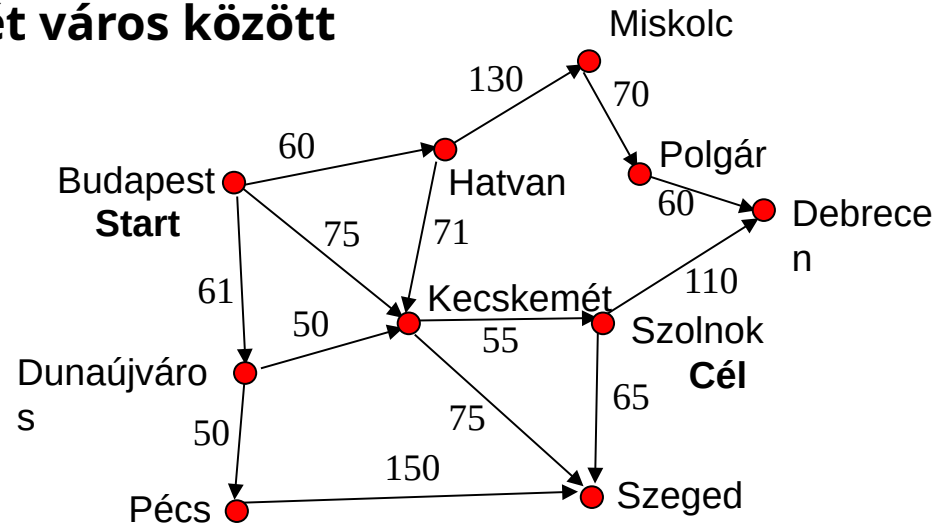


Mélységben először keresés

- A keresési fát balra lefelé tartva járja be. Ha nem tud lefelé menni tovább, visszalép a legalsó elágazásig és a következő ágot választja. Visszalépéskor a sikertelen ág állapotait ejti. (Lásd Prolog működését.)
- Az általános kereső eljárás a következőképpen módosul:
 - 2. Legyen a front első állapota az n választott állapot.
 - 4. A kiterjesztéssel kapott új állapotokat add a frontlista elejéhez.
- Az eljárás
 - Teljes, ha nincs végtelen, vagy (a memóriának) túl mély ág.
 - Nem optimális
 - Időigénye $\approx b^d$, (meredeken nő a mélységgel)
 - Tárigénye $b \cdot d$, (nagyon kis igény!).
- Csak egyetlen állapotba vezető út állapotait és az út leágazásain található állapotokat kell tárolnia.

- Példa a mélységben először keresésre

- **Útkeresés két város között**



A megoldás láthatóan nem optimális útköltségű.

Mélységben először keresés mélységi korláttal

- Cél: a mélységben először keresés végtelen, vagy gyakorlatilag végtelen mély ágainak veszélyét elkerülni egy jól megbecsült mélységi korláttal. (Átmenet az informált kereséshez, amennyiben a korlát mélységének megadásához problémafüggő információt is használ.)
- Működése azonos a mélységben először algoritmusával, de mintha az l mélységi korlát alatti állapotok nem is léteznének.
- Az eljárás
 - Teljes, ha l nagyobb, vagy egyenlő, mint a Cél-állapot mélysége.
 - Nem optimális
 - Időigénye bl , (meredeken nő a mélységi korláttal)
 - Tárigénye b^l , (nagyon kis igény!).
- Konkrét példa lehet: benzinkút keresése egy városban, ha már csak adott liternyi benzinünk van.

Iteratív mélyítés

- Cél: a korlátmegadás problémájának elkerülése azáltal, hogy nulla korlátmélységről indulva egy-egy szinttel növeli a korlátmélységet. Minden egyes korlátmélységnél elvégez egy mélységben először keresést. A korlát mélyítését addig folytatja, amíg megoldást nem talál.
- Az eljárás
 - Teljes
 - Optimális (költség a lépések száma)
 - Időigénye az újrakezdések ellenére nagyságrendileg nem változik a
 - mélységben először bd értékéhez képest.
 - Tárigénye $b*d$, nagyon kedvező

Kiterjesztés sorrendje

Belelérés sorrendje

Korlát= 0



Korlát= 1

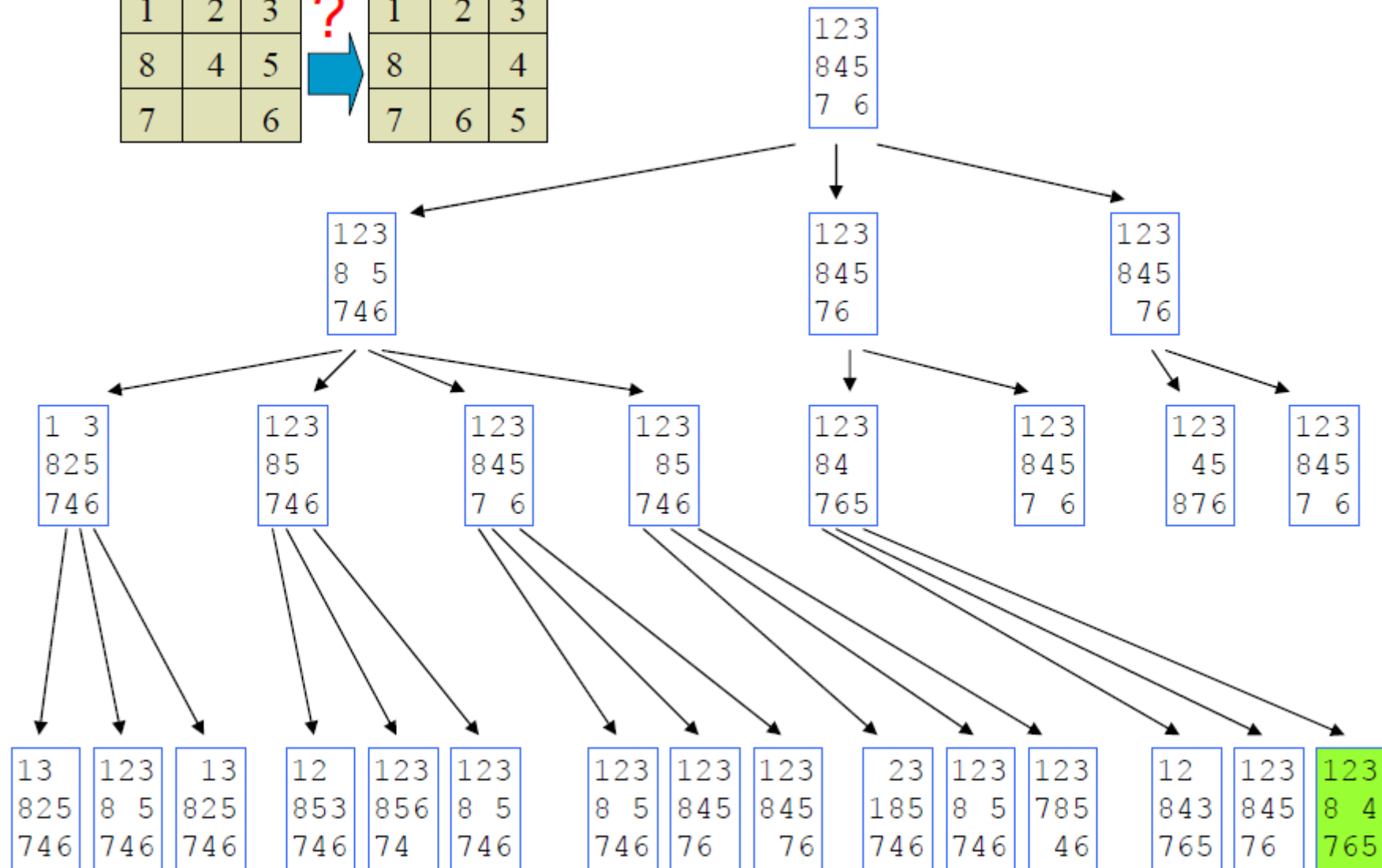
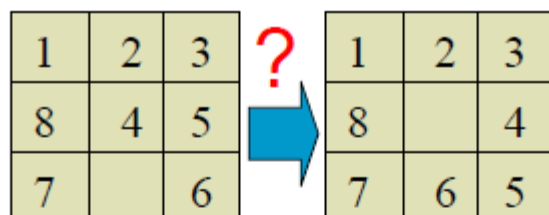


Korlát= 2



Korlát= 3





Informált kereső eljárások (heurisztikus keresés)

- A front állapotai közül belelépésre azt választjuk, amelyre vonatkozóan a konkrét megoldandó feladatra vonatkozó **információk valamilyen előnyt ígérnek**. Ilyen előny lehet: a célhoz legközelebbinek tűnik; a rajta való áthaladás a legkisebb Start-Cél költséggel kecsegtet; a legnagyobb lépést teszi a Cél irányában, stb.
- Az informált állapotválasztással a keresés hatékonysága nagyban fokozható.
- Az informált kereső eljárásokat gyakran **heurisztikus** kereső eljárásoknak nevezzük, mert a problématerületre vonatkozó tapasztalatra építenek. A heurisztika általában nagyban növeli a hatékonyságot, de tévedhet is.
- A heurisztikát a **$h(n)$ heurisztikus kiértékelő függvény** segítségével számszerűsítjük, számítjuk az n állapot esetében.
- Heurisztikus kiértékelő függvényként gyakran alkalmazzák az n állapotból a Cél elérésének költségére vonatkozó becslést. A jó függvény nem becsüli túl a költséget és a Cél állapotra 0 értéket ad.

Informált kereső eljárások fajtái

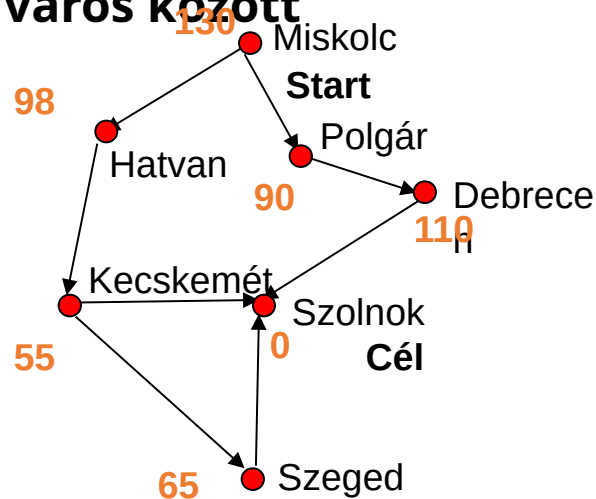
- Globális információra támaszkodó eljárások
 - Legjobbat először keresés (Best first search)
 - A és A* keresés (A search, A* search)
 - Iteratív A* algoritmus (IDA* search)
- Lokális információra támaszkodó eljárások
 - Hegymászó keresés (Hill climbing)
 - Szimulált lehűtés (Simulated annealing)
 - Tabu keresés (Tabu search)
- A globális információ az állapottér bármely két pontja között származtatható, leggyakrabban az n állapot és a Cél-állapot között számítjuk és az n állapothoz rendeljük.
Globális információ felhasználásával megtalálhatjuk a globális optimumot, az optimális költségű utat is.
- A lokális információ az n állapot és közvetlen szomszédai között számítható. Önmagában lokális információ alkalmazásával csak lokális extrémumot garantálnak az eljárások, gyakran azonban annak közelítésével is megelégszünk.

Legjobbat először keresés

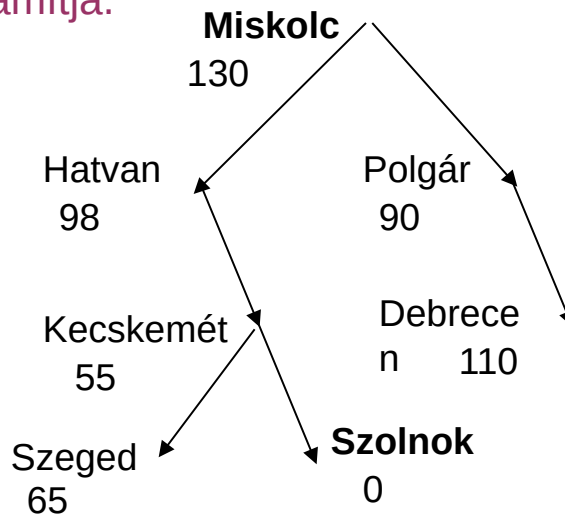
- A legjobbat először keresés egy **heurisztikus kiértékelő függvénnyel** becsüli az n állapotból **a Cél elérésének költségét** a front összes állapotára és abba az állapotba lép, amelyikre ez a költség a legkisebb. Az eljárás végződhet lokális optimumpontban is.
- Patrick **Winston** az eljárást hegymászókkal magyarázta: Egy hegymászókból álló csapat indul a hegycsúcs meghódítására. A csapat tagjai elágazásoknál szétválnak és rádiótelefonnal tartják a kapcsolatot, hogy mindig az a csapat mászhasson tovább, amely számára a csúcs elérése a legkönnyebbnek tűnik. Ha valamelyik csapat elakad, egy másik mászik tovább.
- Az általános kereső eljárás a következőképpen módosul:
 - 2. ... Legyen a front célhoz legközelebbinek becsült állapota az n választott állapot.
- Az eljárás
 - Teljes
 - Nem optimális
 - Időigénye $\approx b^m$, (meredeken nő a mélységgel)
 - Tárigénye $\approx b^m$, (meredeken nő a mélységgel).

- Példa a *legjobbat először* keresésre

- Útkeresés két város között**



A heurisztikus kiértékelő függvény a légvonalbeli távolságot számítja.



Sorsz.	Front	Választás
0.	Miskolc	Miskolc
1.	Hatvan Polgár	Polgár
3.	Hatvan Debrecen	Hatvan.
4.	Kecskemét Debrecen	Kecskemét
5.	Szeged Szolnok Debrecen	Szolnok

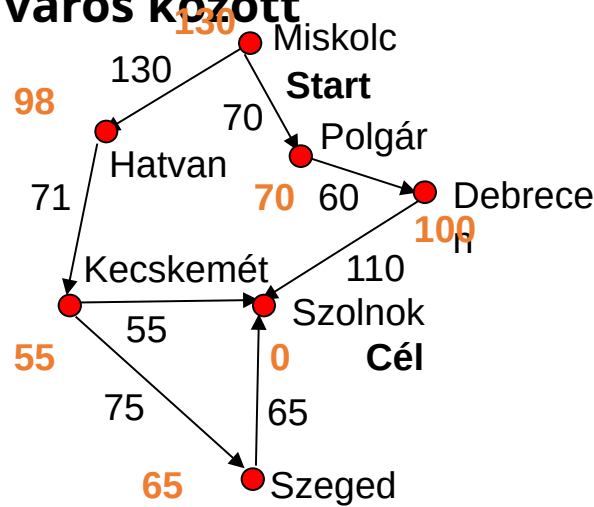
Az A és A* keresés

- Az A keresés számítja a front összes állapotára a $g(n)$ függvénnyel a Start-n távolságokat, valamint ugyanezen állapotokra egy $h(n)$ **heurisztikus kiértékelő függvénnyel** becsli az n állapotból a **Cél elérésének költségét**, majd képezi ezek összegét: $f(n) = g(n) + h(n)$.
Abba az állapotba lép, amelyikre ez az áthaladó útvonalhossz becsült költsége a legkisebb.
Az eljárás végződhet lokális optimumpontban is.
- Bizonyítható, hogy amennyiben a $h(n)$ költségbecslő függvény nem becsüli túl a Cél elérésének költségét egyik állapotra sem, akkor az eljárás által adott útvonal egyben optimális is. Az ilyen függvény jele: $f'(n)$, az eljárás neve ekkor A*.
- Az általános kereső eljárás a következőképpen módosul:
 - 2. ... Legyen a front állapotai közül kiválasztva az, amelynek legkisebb az $f'(n)$ függvényértéke.
- Az eljárás
 - Teljes
 - Optimális
 - Időigénye nagy
 - Tárigénye nagy, gyakran túlcsordítja a memóriát.

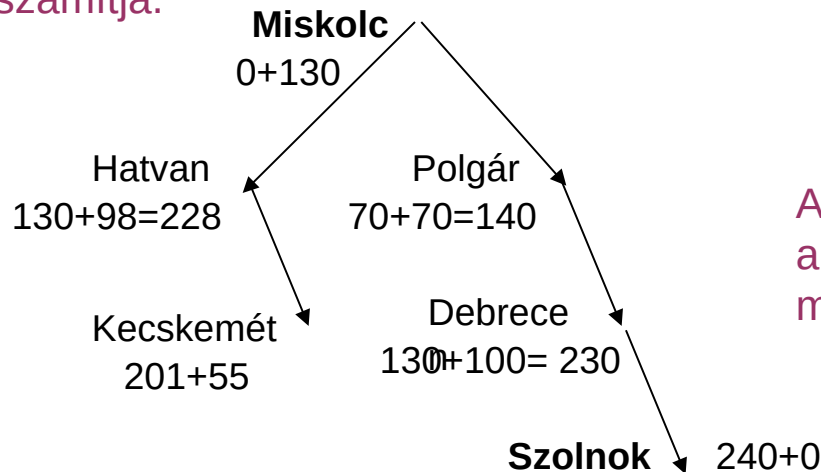


- Példa az A* keresésre

- Útkeresés két város között



A heurisztikus kiértékelő függvény a légvonalbeli távolságot számítja.



Sorsz.	Front	Választás
0.	Miskolc	Miskolc
1.	Hatvan Polgár	Polgár
3.	Hatvan Debrecen	Hatvan
4.	Kecskemét Debrecen	Debrecen
5.	Kecskemét Szolnok	Szolnok

Az elágaztatás és ugrálás algoritmus és a legjobbat először módszer egyesítése.

A hegymászó keresés

- Lokális információt használ: csak a szomszédos állapotokat nézi.
- A legjobb szomszédot választja és gyorsan feljut egy közeli lokális csúcspontra.
- Lépései:
 1. Legyen n a Start állapot.
 2. Ha n Cél-állapot, add vissza a hozzávezető úttal együtt. Vége.
 3. Egyébként állítsd elő n kiterjesztett állapotait. Legyen n új értéke ezek közül a legjobb.
Ismételd 2.-től.
- Az eljárás:
 - Teljesége: amennyiben lokális maximum a kritérium, akkor azt megtalálja
 - Nem optimális
 - Időigénye: nagyon alacsony
 - Tárigénye: nagyon kicsi, mert nem tárol keresőgráfot.
- Probléma: erősen függ a felület alakjától.

Szimulált hűtés

- Ötlet: a fémek lehűlés során rácsszerkezetbe dermednek, melynek rendezettsége és finomsága a lehűtés sebességétől függ, lassú lehűtés magasabb rendezettséget eredményez.
- Cél: a lokális csúcsokról való elkerülés azáltal, hogy megadott valószínűséggel lefelé haladó, rontó lépések is lehetnek. A rontó lépések valószínűsége a hőmérséklet csökkenésével csökken.
- Kellően lassú lehűlés esetén megtalálja a globális maximumot.
- Nagyméretű feladatokra is hatékony.
- Közelítő eredményt ad, melyet a leállási feltétel teljesülésekor ér el. A leállási feltétel általában azt írja elő, hogy az utolsó néhány hőmérsékleten a célfüggvény értéke megadott kis értéken belül maradjon.

Szimulált hűtés algoritmus minimalizálás cél esetén

1. Legyen n a kezdő állapot, T_0 a kezdő hőmérséklet, $k=0$ a ciklusváltozó kezdőértéke.

2. Amíg kilépési feltétel nem teljesül:

Legyen r az n egyik szomszédja,

ha $f(r) \leq f(n)$ akkor n vegye fel r értékét,

egyébként ha $\exp((f(n) - f(r))/T_k) > \text{Véletlenszám}$ akkor n vegye fel r értékét.

Inkrementáld a k -t, számítsd T_k új értékét k és T_{k-1} függvényében,.

3. Add vissza az n megoldást, vége.

- A hőmérséklet szigorúan csökkenő sorozat.
- *Véletlenszám* egy 0-1 intervallumbeli egyenletes eloszlású véletlenszám.
- Az $f(n)$ függvényérték globális minimumhoz való konvergálását egyre kisebb
romlási szakaszok jellemzik.